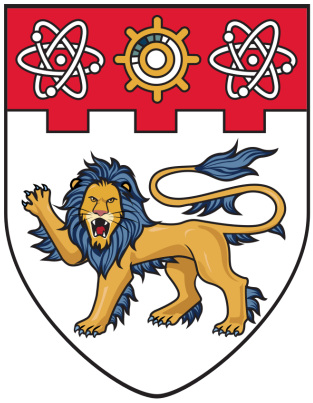


NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

Moving-Horizon-Estimation-Enabled Nonlinear Model Predictive Control for Quadrotors under Abrupt Payload Changes

[Author Name]

SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING

2026

Moving-Horizon-Estimation-Enabled Nonlinear Model Predictive Control for Quadrotors under Abrupt Payload Changes

[AUTHOR NAME]

SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING

A DISSERTATION SUBMITTED IN PARTIAL FULFILMENT OF
THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE IN [PROGRAMME]

2026

Statement of Originality

I hereby certify that the work embodied in this thesis is the result of original research and has not been submitted for a higher degree to any other University or Institution.

. . . . . .
Date

. . . . . .
[Author Name]

Supervisor Declaration Statement

I have reviewed the content and presentation style of this thesis and declare it is free of plagiarism and of sufficient grammatical clarity to be examined. To the best of my knowledge, the research and writing are those of the candidate except as acknowledged in the Author Attribution Statement. I confirm that the investigations were conducted in accord with the ethics policies and integrity standards of Nanyang Technological University and that the research data are presented honestly and without prejudice

....

Date [Supervisor Name]

Authorship Attribution Statement

This thesis does not contain any materials from papers published in peer-reviewed journals or from papers accepted at conferences in which I am listed as an author.

....

Date [Author Name]

Table of Contents

Abstract **v**

Acknowledgements **vii**

Acronyms **viii**

Symbols **ix**

List of Figures **xi**

List of Tables **xii**

1 Introduction **1**
1.1 Background 1
1.2 Problem Statement 2
1.3 Research Objectives 3
1.4 Scope 4
1.5 Contributions 5
1.6 Dissertation Organisation 7

2 Background and Related Work **9**
2.1 Model Predictive Control 9
2.2 Quadrotor NMPC 10
2.3 Uncertainty and Adaptation in MPC 10
2.4 State and Parameter Estimation 11
2.5 Inertial-Parameter Identification 12
2.6 Change Detection and Event-Triggered Estimation 12
2.7 Positioning and Research Gap 13

3 Modeling and Parameter Structure **16**
3.1 System Definition and Modelling Assumptions 17
3.1.1 Attitude representation 18
3.2 Quadrotor Dynamics with a Rigid Payload 19
3.2.1 Kinematics 19
3.2.2 Translational dynamics 20
3.2.3 Rotational dynamics 22
3.3 Payload-Induced Mass and Inertia Coupling 22

3.3.1 Inertia increment and the reduced mass 23
3.3.2 Magnitudes on the experimental platform 24
3.4 Single-Parameter Estimation Structure 25
3.4.1 One online degree of freedom 25
3.4.2 Which channel identifies the mass 26
3.4.3 What is estimated, what is derived, and what is fixed . . 27
3.5 Measured-Input Reconstruction and Model Interfaces 29
3.5.1 The physical input 29
3.5.2 The interface to the autopilot 31
3.6 Modelling Assumptions and Limitations 32
3.7 Chapter Summary 35

4 The MHE–NMPC Framework 37
4.1 Closed-Loop Architecture 38
4.2 Estimator and Controller Models 39
4.2.1 The estimator: velocity and mass 39
4.2.2 The controller's prediction model 42
4.3 Moving Horizon Estimator 43
4.3.1 The optimisation 43
4.3.2 Weights 44
4.3.3 Mass bounds and the mass-health check 44
4.3.4 Input gating and the grasp gap 46
4.3.5 Initialisation 46
4.3.6 Stage weighting 48
4.3.7 Solver failure and re-initialisation 48
4.4 Adaptive NMPC 49
4.4.1 The optimal control problem and its input baseline 49
4.4.2 Cost and weights 50
4.4.3 Residual specific force 51
4.4.4 Input constraints and the actuator envelope 52
4.4.5 Solver 55
4.5 The Continuous Payload Interface 55
4.5.1 One atomic frame 55
4.5.2 Filtering and slew limiting 56
4.6 Inner-Loop Scheduling 57
4.6.1 From inertia ratio to a command scale 57
4.6.2 Headroom limiting 58
4.6.3 Attach-time inertia bootstrap 59
4.7 The Grasp–Release Lifecycle 60
4.7.1 What release acceptance does, and why it is gated on evidence 60
4.7.2 Two state machines 62
4.7.3 No-payload evidence score 62
4.7.4 Confirmation, UNRESOLVED, and braking to hover 63
4.7.5 Force-balance release evidence and the uncommanded-loss watchdog 64

4.7.6 Limits of the lifecycle 67
4.7.7 Threshold provenance 68
4.8 Reference Trajectory 69
4.9 Online Execution Sequence 69
4.10 Chapter Summary 71

5 Implementation and Experimental Evaluation 73
5.1 Platform and Implementation 74
5.1.1 Platform parameters and task 75
5.1.2 Real-time performance 76
5.2 Evaluated Configuration 77
5.2.1 The full-state estimator and the mass-evidence lifecycle . 79
5.3 Deployed-System Validation 81
5.3.1 Design and execution 81
5.3.2 Acceptance criteria 83
5.3.3 Outcomes 83
5.3.4 Relation to the historical campaign 88
5.3.5 Scope 89
5.4 Experimental Questions and Hypotheses 89
5.4.1 Release-acceptance arms 90
5.4.2 Conditions and scenarios 91
5.5 Cohort Design, Eligibility and Metrics 92
5.5.1 Sources of evidence 93
5.5.2 Attempts and exclusions 94
5.5.3 Metrics 95
5.5.4 Experimental unit and statistical reading 96
5.6 Primary Results 97
5.6.1 Mass estimation 97
5.6.2 Release completion and delay 99
5.6.3 Delayed detachment 100
5.6.4 Command-armed versus always-on confirmation 101
5.6.5 Tracking performance 103
5.7 Historical Release Campaign with Force-Balance Evidence 104
5.7.1 Design 104
5.7.2 Release semantics and carry-phase acceptance 106
5.7.3 Uncommanded loss and a jammed gripper 108
5.7.4 Thrust-map error 109
5.7.5 Fixed, $\mathscr{L}_1$ and MHE controllers 110
5.7.6 Scope of the campaign 111
5.8 Ablations 112
5.8.1 The residual specific force 112
5.8.2 Full-state versus reduced estimator 113
5.8.3 The horizontal-offset decision 121
5.9 Failure Modes and Operating Limits 122
5.9.1 Manoeuvre-induced mass collapse: development evidence . 122
5.9.2 Grasp alignment 124

5.9.3 The operating limit 124
5.9.4 Estimator health 125
5.9.5 Configuration and default-safety 125
5.10 Threats to Validity, Missing Baselines and Reproducibility 127
5.10.1 Thrust-map calibration: an analytical sensitivity 128
5.10.2 Experiments designed but not run 128
5.10.3 Missing baselines 129
5.10.4 Reproducibility 130
5.11 Chapter Summary 132

6 Conclusion and Future Work **134**
6.1 Summary 134
6.2 Principal Findings 135
6.3 Limitations 141
6.4 Future Work 142
6.5 Closing Remark 144

References **145**

Appendix A Implemented Optimal Control Problem Configuration **150**
A.1 Identity of the deployed system in the validation campaign . . . 150

Appendix B Experiment Record Template **154**

Abstract

Delivery quadrotors change mass and roll and pitch inertia abruptly when they grasp or release a parcel, and a predictive controller that keeps the unloaded model plans with the wrong hover thrust and the wrong rotational response. This dissertation studies how far such a vehicle can adapt to a centred rigid payload from ordinary flight signals, and how it should decide that a commanded release has physically happened.

For a payload grasped on the thrust axis, rigid-body algebra makes the inertia increment the reduced mass times the square of a nominal vertical arm, so the composite mass is the only payload quantity to be identified online. A moving horizon estimator identifies it from odometry and the thrust reconstructed from measured rotor speeds. In its deployed form it keeps only velocity and mass as states and takes the measured attitude as a known input, so that the less reliable rotational model has no path to the mass. In a paired closed-loop comparison of 96 flights against a full-state estimator, this form lowered the loaded mass error from 0.035 kg to 0.013 kg, removed the collapse of the estimate in manoeuvre and solved about ten times faster, without changing tracking or release safety. Replayed on the same flights, an extended Kalman filter with the same inputs was as accurate: the gain comes from the model structure, not from the moving horizon. One health-gated estimate updates a nonlinear model predictive controller continuously; a separate lifecycle accepts a release only when evidence confirms that the payload has left — in the deployed form, a force balance of the reconstructed thrust against the specific force measured by the inertial unit, which does not pass through the mass estimate.

The system is evaluated in PX4/Gazebo software-in-the-loop simulation with acados solvers. The deployed combination of the reduced estimator and the force-balance rule was validated in 180 flights from a tagged commit, at two payloads and two speeds, under nominal and delayed release, uncommanded loss, a jammed gripper and a $\pm 10\,\%$ thrust-map error; four acceptance criteria inherited from an earlier pre-specification were fixed during the run and all met. All 72 nominal releases were confirmed, none early and none during carry, 2.2 s after physical detachment at the median; the loaded mass error was 0.013 kg, the estimate never collapsed in manoeuvre, and the estimator solved in 0.3 ms. All 24 loads lost without a command were detected and none of 24 jammed releases was accepted. The thrust map bounds the system: 10 % high, every release was missed and ended in a safe hover; 10 % low, the 0.15 kg payload read as absent, the loss watchdog never armed, and with detachment delayed by 4 s all 12 releases were accepted about 3 s before the payload left.

Historical batches with the full-state estimator show how the design was reached. In a frozen cohort of 120 flights the mass estimate was within 0.5 % in loaded hover but collapsed periodically in manoeuvre at $4\,\mathrm{m\,s^{-1}}$, and confirming the release on that mass evidence throughout carry produced 114 false acceptances in 20 of 32 flights; arming it only after the release command removed them. A campaign of 312 flights introduced the force-balance rule, which removed them without that restriction, and showed that estimating the mass gave no consistent tracking benefit over a residual specific force alone.

The evidence is from simulation only, and no hardware validity is claimed. Both the estimator and the release detector depend on thrust-map calibration, and neither was compared against a change-detection baseline.

Keywords: quadrotor; nonlinear model predictive control; moving horizon estimation; online mass estimation; payload inertia; release confirmation; software-in-the-loop.

Acknowledgements

I would like to thank my supervisor, [Supervisor Name], for guidance throughout this dissertation. I am also grateful to the members of [Laboratory or Research Group] for their technical discussions and support with the simulation and flight-control platform.

Acronyms

Acronym	Meaning
CoM	Centre of Mass
CRLB	Cramér–Rao Lower Bound
CUSUM	Cumulative Sum (change-detection procedure)
EKF	Extended Kalman Filter
ENU	East–North–Up
ESC	Electronic Speed Controller
FLU	Front–Left–Up
FSM	Finite State Machine
INDI	Incremental Nonlinear Dynamic Inversion
IQR	Interquartile Range
LPF	Low-Pass Filter
MAD	Median Absolute Deviation
MHE	Moving Horizon Estimation
MPC	Model Predictive Control
NLP	Nonlinear Programming
NMPC	Nonlinear Model Predictive Control
OCP	Optimal Control Problem
PX4	PX4 autopilot software stack
QP	Quadratic Programming
RK4	Fourth-order Runge–Kutta integration
RMS(E)	Root Mean Square (Error)
ROS	Robot Operating System
RTI	Real-Time Iteration
SITL	Software-in-the-Loop
SQP	Sequential Quadratic Programming
UAV	Unmanned Aerial Vehicle
ZOH	Zero-Order Hold

Symbols

Symbol	Definition
States and inputs	
$\boldsymbol{p}, \boldsymbol{v}$	Position and velocity in the world frame (ENU)
$\boldsymbol{q}$	Hamilton unit quaternion, scalar component first
$\boldsymbol{\omega}$	Body angular velocity (FLU body frame)
$T, \boldsymbol{\tau}$	Collective thrust and body torque
ϕ, θ	Roll and pitch angles (ZYX Euler convention)
Payload parameters	
m_B, m_P, m_T	Bare-airframe, payload and composite mass
$\boldsymbol{r}_p = [0, 0, r_z]$	Payload attachment point in the body frame (centred grasp)
r_z	Vertical arm of the payload below the rotor plane (weak prior)
$\boldsymbol{c}$	Composite centre of mass in the body frame, $\boldsymbol{c} = (m_P/m_T)\boldsymbol{r}_p$
$\boldsymbol{c}_{xy}$	Horizontal CoM offset; not modelled, held at $\mathbf{0}$
μ	Reduced mass, $\mu = m_B m_P / m_T$
J_B, J_T	Bare-airframe and composite inertia tensors
ΔJ	Payload-induced roll/pitch inertia increment, $\Delta J = \mu r_z^2$
κ	Inertia ratio $(J_{xx} + \Delta J)/J_{xx}$
Reconstructed input and actuator model	
$T_{\text{phys}}, \boldsymbol{\tau}_{\text{phys}}$	Wrench reconstructed from rotor speeds
k_f, k_m	Rotor thrust and yaw-reaction coefficients
η	Normalised throttle command
k_d, g	Linear drag coefficient, gravitational acceleration
Estimator and controller	
$N, \Delta t$	Horizon length and sampling interval
W_y, W_w, W_0	MHE measurement, process-noise and arrival weights
β_j	Per-stage weighting factor
Q, R, P	NMPC stage-state, input and terminal weights
$L_{(\cdot)}$	Bryson tolerance scale for the corresponding residual
u_{hover}	Parameter-dependent equilibrium input baseline
$\hat{\boldsymbol{d}}$	Residual specific force estimated by the controller ($\mathscr{L}_1$-type)
Lifecycle	
γ	Mass-based no-payload evidence score, (4.30); logged only
m_{acc}, f_z	Force-balance mass (4.34); specific force along the thrust axis
γ_{acc}	Force-balance evidence score, (4.36); the deployed release evidence
$\sigma(v; a, b)$	Cubic smooth-step indicator, (4.29)
ς	Body-rate command scale

List of Figures

3.1 How the payload parameter reaches the controller 33

4.1 System architecture and information flow of the deployed system. The adaptive path consumes only rotor speeds and odometry: one atomic, health-gated frame carrying the composite mass and the inertia increment derived from it drives both the NMPC model and the continuous body-rate command scheduling. The release path is separate: a force balance of the reconstructed thrust against the inertial unit's specific force feeds the release lifecycle (Section 4.7.5), which does not read the frame. No attach or release notification enters either path. 38

4.2 The deployed payload lifecycle 72

5.1 Composite-mass error through carry and release 98

List of Tables

2.1 Representative related work compared 15

3.1 Model parameters, classified by identity. Only the composite mass is a decision variable of the estimator; the inertia increment is generated from it. 28

4.1 The two models of one plant: the estimator's and the controller's prediction model. Equation numbers refer to the definitions. . . . 40
4.2 Stage weighting of the MHE. The de-weighting matrices are built on the velocity, noise and arrival blocks of (4.9). 48
4.3 Lifecycle thresholds and their provenance. 68

5.1 Main implementation components 74
5.2 Design of the deployed-system validation 82
5.3 Acceptance criteria of the deployed-system validation 84
5.4 Binary outcomes of the deployed-system validation 85
5.5 Continuous results of nominal release in the deployed system . . 86
5.6 Release-acceptance arms 91
5.7 Sources of evidence and the configuration each one tests 94
5.8 Attempts and exclusions in the frozen cohort 95
5.9 Composite-mass error by flight phase 97
5.10 Release completion and confirmation delay 100
5.11 Release confirmations during loaded carry 102
5.12 Experiments of the force-balance release campaign 105
5.13 Release acceptance under the force-balance rule (E1) 106
5.14 Faults under the force-balance rule (E4) 108
5.15 Estimation-side thrust-map error (E3) 109
5.16 Controller comparison under the force-balance rule (E2) 111
5.17 Residual specific force on against off 113
5.18 Replay of the reduced estimator on the 264 nominal-thrust flights of the release campaign, against the full-state estimate recorded online in the same flights. Flight-level medians; paired Wilcoxon signed-rank tests. 114
5.19 Full-state against reduced estimator in closed loop 117
5.20 Full-state against reduced estimator: exploratory metrics 119
5.21 Replay ablations of the reduced estimator and simple baselines . 120

5.22 Development evidence for the mass-collapse mechanism. Configuration changes are cumulative and the first two changes in the second row are confounded. 123

A.1 NMPC configuration (evaluated configuration and deployed controller). 150
A.2 MHE configuration. The full-state estimator is the evaluated configuration of the cohort and the release campaign; the reduced estimator is the deployed estimator and the only one in the mainline code base. 151
A.3 Payload interface, lifecycle and task settings (evaluated configuration). Rows marked ‡ are where the deployed system differs or adds. 152
A.4 Identity of the deployed system in the validation campaign . . . 153

Chapter 1

Introduction

1.1 Background

Quadrotors are increasingly used for inspection, transport and delivery because they can hover and manoeuvre in confined three-dimensional environments. Their agility, however, is obtained from a light airframe with limited thrust and torque margins. A package pickup can therefore change the vehicle dynamics on the same time scale as the control loop. The total mass changes the thrust-to-acceleration map, and a load hanging below the airframe multiplies the rotational inertia that the attitude loop must move.

Model predictive control is attractive for this problem because it predicts future motion while handling actuator limits and coupled dynamics explicitly [1, 2]. For a quadrotor the relevant equations are nonlinear and the attitude evolves on the unit-quaternion manifold; nonlinear MPC therefore avoids the loss of fidelity caused by repeated linearisation over aggressive motion [3, 4]. Its performance nevertheless depends on the model supplied to the prediction. If the controller continues to use the empty-airframe parameters after grasping a parcel, even a numerically converged optimisation solves the wrong problem.

This dissertation couples moving horizon estimation with nonlinear model predictive control. MHE estimates states and parameters by optimising over a re-

cent window of measurements while enforcing the system dynamics and physical bounds [5]. Within that window the estimator identifies a single payload-dependent scalar, the composite mass, and from it the controller reconstructs every payload-dependent term in its own model.

1.2 Problem Statement

Consider a quadrotor that follows a three-dimensional figure-eight path, grasps a payload from above, transports it and releases it at a selected phase of the trajectory. The controller has no access to the true payload parameters. Only ordinary onboard flight signals are available: odometry, attitude, body rate and rotor-speed telemetry.

Two model errors appear at attachment:

1. the composite mass changes abruptly, shifting the hover thrust and the translational acceleration gain;
2. the payload lever arm increases roll and pitch inertia several-fold, reducing effective inner-loop bandwidth.

At release both changes occur in the opposite direction. The payload is grasped on the thrust axis, so the composite centre of mass moves only vertically and no standing trim moment arises; a residual offset left by imperfect alignment is treated as an unmodelled disturbance (Chapter 3).

The central research question is:

> Can a quadrotor identify its payload online from its ordinary flight signals alone — through grasp, loaded manoeuvre and release —

and feed that estimate to a constrained predictive controller consistently enough to remain a well-tracking controller throughout?

Release adds one decision that the continuous model does not make by itself: when the controller may accept that a delivery has happened. Accepting a release declares the task complete and resets auxiliary estimates learned against the loaded vehicle; it does not change the model's mass or inertia, which follow the estimate continuously. The controller in this work issues the release command itself, but a command is not a detachment — a gripper can be slow to open or can jam. The dissertation therefore studies *estimator-confirmed detachment*: once a release has been commanded, the release is accepted only when the estimate shows that the payload has actually gone. This costs a confirmation delay, and it is evaluated against accepting at the command, under nominal release and under a release whose physical detachment is held back. It is not a detector of load loss in general: a load lost without any command is outside what the evaluated mechanism can establish.

This question has a specific technical obstacle, which Chapter 3 addresses and which shapes the whole design. The inertia increment is several times larger, relative to its bare-airframe value, than the mass increment, so the rotational channel carries a strong payload signature — but it is also the channel this platform models least well. An estimator that lets the mass enter its rotational model will use the mass to absorb rotational model error. The design adopted here identifies the composite mass on the translational channel only, derives the inertia increment from it by rigid-body algebra, and accepts — and measures — the loss of information that this costs in manoeuvre.

1.3 Research Objectives

The dissertation has five objectives:

1. derive a consistent quadrotor model with explicit payload mass and inertia entry points, and reduce a centred rigid payload to one online degree of freedom from which every payload-dependent term follows (Chapter 3);

2. formulate a moving horizon estimator that identifies the composite mass from a wrench reconstructed from measured rotor speeds, on the translational channel, while keeping the mass from absorbing rotational model error (Chapters 3 and 4);

3. formulate a predictive controller whose prediction model, input baseline and inner-loop command scaling are updated continuously from one health-gated estimate (Chapter 4);

4. define a release lifecycle that separates the release command from the physical detachment by gating release acceptance on measured evidence, and that holds an unconfirmed release as UNRESOLVED rather than assuming it complete (Chapter 4);

5. evaluate the complete interface in software-in-the-loop simulation with a randomised, fully reported cohort and targeted ablations, and characterise where it fails (Chapter 5).

1.4 Scope

The work covers simulation and software-in-the-loop validation of a rigidly attached parcel grasped on the thrust axis. Airframe inertia, rotor geometry and the thrust and yaw-reaction coefficients are treated as calibrated plant quantities. The calibrated airframe mass appears in the rigid-body model, in the derived payload mass $m_P = (m_T - m_B)^+$ and in offline scoring.

The information boundary is stated precisely, because a looser statement would be false. *No task-specific payload mass enters the estimator or the controller, and*

no attach or release notification enters the estimator. The controller uses its own release command only to start the confirmation timeout, to end the attach-time inertia floor, to decide whether an acceptance also resets the residual specific force, and — under the command-armed and force-balance policies — to arm the confirmation count. The command never sets the model's mass or inertia. One mass-valued platform constant does remain in the closed loop — the airframe's 0.30 kg payload envelope, used for an attach-time inertia floor — and it is disclosed as a specification of the airframe rather than as information about the parcel. Chapter 3 states exactly where it appears.

The scope does not include eccentric grasps beyond the gripper's alignment tolerance, deformable or sloshing loads, cable-suspended payload dynamics, wind, motor failure, hardware flight, or a formal closed-loop stability proof for the estimator–controller interconnection. All system-level evidence is from PX4/Gazebo software-in-the-loop simulation. These limits are revisited in Chapter 6.

1.5 Contributions

The contributions lie in the architecture and its evaluation rather than in the rigid-body algebra on which they rest. That basis is stated first because the rest depends on it: for a payload grasped on the thrust axis, the parallel-axis theorem taken about the moving composite centre of mass gives a roll and pitch inertia increment equal to the reduced mass times the square of a nominal vertical arm, so the composite mass is the only payload quantity to be identified online, and the inertia follows from it and vanishes with it, with no event needed to remove it (Chapter 3). This is a standard result applied to the task, not a contribution in its own right. The contributions are:

1. **A continuous estimate-driven controller interface with an evidence-**

confirmed release lifecycle. One health-gated frame updates the controller's prediction model, input baseline and inner-loop scaling continuously; release acceptance is a separate decision, taken only on evidence that the payload has left. In the deployed form that evidence is a force balance of the thrust reconstructed from rotor speeds against the specific force measured by the inertial unit, which does not pass through the mass estimate, and it is opened by the release command or by a watchdog armed on airborne load. In a 180-flight validation of the deployed system it produced no false acceptance during carry, never accepted a release early under a nominal thrust map, detected every load lost without a command and accepted no jammed release. Its price is the thrust map: when the map under-reads by more than the payload's share of the composite mass, a loaded vehicle reads as empty, and a delayed release was then accepted before the payload left in every such flight. In the historical cohort, the same lifecycle on the estimator's own mass evidence was sound only when armed by the release command: run throughout carry, it accepted releases that had not happened during high-speed flight (Chapter 4 and Sections 5.3, 5.6 and 5.7).

2. **A mass estimator driven by a reconstructed physical input that does not integrate the rotational dynamics.** The estimator is fed the thrust reconstructed from measured rotor speeds rather than the controller's intended input, which would make the mass unobservable by construction. In its deployed form it keeps only the world-frame velocity and the composite mass as states and takes the measured attitude as a known input, so that errors of the poorly modelled rotational channel have no path to the mass; a mass-health check flags an estimate that leaves a physical band instead of hiding it behind a hard bound. Replayed on the 312 recorded campaign flights, it had a smaller loaded mass error than the full-state estimator in every nominal flight. The evaluation campaigns were flown with that earlier full-state form, which separated the channels more weakly by carrying the payload inertia as a parameter lagged by one

solve. In a paired closed-loop comparison of 96 flights with everything else fixed, the deployed form met every pre-specified acceptance criterion: it lowered the loaded mass error from 0.035 kg to 0.013 kg, removed the manoeuvre-induced collapse of the estimate, solved about ten times faster and changed neither tracking nor the lifecycle. Replay against an extended Kalman filter and simpler estimators on the same inputs shows that this gain comes from the model structure — reconstructed thrust and measured attitude as known inputs, no rotational channel — rather than from the moving-horizon solution, which matched a filter on accuracy (Chapters 3, 4 and Section 5.8.2).

3. **A software-in-the-loop evaluation with randomised blocks, complete reporting, ablations and failure cases.** A 180-flight validation of the deployed system from a tagged commit, with acceptance criteria inherited from an earlier pre-specification; and, as the historical record of how the design was reached, a frozen cohort of 120 flights and a release campaign of 312, both with the full-state estimator, and a paired estimator comparison of 96. Every flight is reported, with the configuration each result applies to stated explicitly, together with an ablation of the residual specific force and an account of the failure modes — in particular the collapse of the mass estimate under sustained manoeuvre and the false acceptances it causes in an always-on lifecycle, and, for the deployed system, the early acceptance caused by an under-reading thrust map (Chapter 5).

1.6 Dissertation Organisation

Chapter 2 reviews predictive control, online estimation, inertial-parameter identification and payload-adaptive aerial control, and positions this work against representative approaches. Chapter 3 derives the model and the payload parameter structure, and is where the one-degree-of-freedom reduction and the channel

assignment of the mass are developed in full. Chapter 4 presents the coupled architecture, the numerical form of both optimal control problems, the continuous payload interface and the release lifecycle. Chapter 5 describes the implementation and reports the validation of the deployed system and the historical experiments that led to it, including the conditions under which the interface fails. Chapter 6 answers the research objectives against that evidence and states what remains open.

Chapter 2

Background and Related Work

2.1 Model Predictive Control

MPC repeatedly solves a finite-horizon optimal control problem, applies the first input and shifts the horizon forward. Its practical value comes from combining prediction, multivariable coordination and explicit constraints within one optimisation [1, 2]. Linear MPC is mature and computationally efficient, but a single linear model is poorly suited to the large attitude changes and thrust-direction coupling of a quadrotor. NMPC retains the nonlinear dynamics and nonlinear tracking errors directly [3].

The price is computation. Multiple shooting, sequential quadratic programming, Gauss–Newton Hessian approximations and real-time iteration schemes are widely used to make NMPC practical [6]. CasADi provides symbolic differentiation and a high-level optimisation interface [7]; acados generates efficient optimal-control solvers around tailored SQP and QP components [8], with HPIPM [9] as the quadratic subproblem solver.

2.2 Quadrotor NMPC

A quadrotor is differentially flat under common assumptions [10], and many successful controllers exploit position and yaw references with a cascaded inner attitude loop [11]. NMPC can instead optimise collective thrust and body torque against the complete nonlinear rigid-body model, and this formulation becomes advantageous as trajectory curvature and operating envelope increase [4, 12].

The controller in this dissertation predicts position, velocity, unit quaternion and body rate. The attitude residual is the vector part of the relative quaternion, which avoids treating the four quaternion components as independent Euclidean coordinates. Input limits represent collective thrust and body-torque authority. This formulation is nominally standard; the distinguishing issue is that the equilibrium input and the plant parameters change at payload attachment and again at release.

2.3 Uncertainty and Adaptation in MPC

Robust MPC protects against bounded uncertainty, while stochastic MPC describes uncertainty probabilistically. Both can preserve margins without identifying the current plant, but conservatism grows when the uncertainty set must cover both an empty and a loaded vehicle. Adaptive MPC instead updates a parameterised model online: certainty-equivalent schemes use the current estimate directly, set-membership schemes maintain a feasible parameter set, and dual-control schemes excite the plant to learn [13, 14].

Learning-based MPC extends this using Gaussian processes, neural networks or learned residual models. Data-driven MPC for quadrotors can compensate complex aerodynamic effects [15], and neural approximations can reduce online computation [16]; broader surveys of learning for control under safety con-

straints are available [17]. Those methods are valuable when the uncertainty is poorly structured. Payload pickup has stronger prior structure: rigid-body mechanics already specifies how mass and attachment geometry enter the equations. The present work therefore estimates a small physical parameter set rather than learning a general residual map.

A distinct and influential alternative is to decline to identify the payload at all, and to absorb its effect as a lumped matched disturbance. The $\mathscr{L}_1$-adaptive augmentation of a quadrotor NMPC [18] is the representative of this family, and incremental nonlinear dynamic inversion [19] is its inner-loop counterpart. Such schemes are robust and require no payload model, but their internal quantity is a force or an acceleration, not a mass and a moment. They therefore cannot answer the question this dissertation poses — whether a payload is still attached — because that question is not represented in their state. The comparison is implemented here (an additive lumped translational and rotational disturbance in the prediction model) but is not among the frozen results.

2.4 State and Parameter Estimation

Recursive filters such as the extended Kalman filter propagate a local Gaussian approximation. They are efficient and central to flight-state estimation, but inequality constraints are not naturally hard constraints and abrupt parameter changes violate the local linearisation assumption. MHE estimates a window of past states and disturbances by solving a constrained optimisation; it can impose bounds such as a physically admissible mass interval and can attribute residuals using the same nonlinear model as the controller [1, 5]. Recent work learns the MHE weights themselves [20], which is the principled version of the stage-weight question that Chapter 4 settles negatively for this problem.

A moving window introduces a trade-off that this work encounters directly. A

longer horizon improves noise rejection, but the constraint that makes an augmented parameter identifiable — that it is constant across the window — becomes less defensible as the window spans more of the trajectory. The implemented estimator uses 20 stages at 0.1 s.

2.5 Inertial-Parameter Identification

That the rigid-body equations are *linear* in a particular set of inertial parameters — mass, first mass moment and the six inertia elements — is a classical result in manipulator identification [21]. The present work identifies only the first of them online. For a payload grasped on the thrust axis the first moment vanishes and the inertia increment reduces to a known function of the mass, so the classical set collapses to one scalar; the argument of Chapter 3 is about which measurement channel should carry that scalar on a vehicle whose rotational model is less trustworthy than its translational one.

Payload-aware estimation for multirotors has previously used force–torque sensing, external observers or explicit mass estimators [22–25], and slung-load and aerial-manipulation work treats the payload as a dynamic subsystem of its own [26–28]. In the works reviewed here, whether a payload is present is either assumed known or not represented as a quantity of its own. This dissertation studies a setting in which the estimator is not told when a payload is attached or released.

2.6 Change Detection and Event-Triggered Estimation

Deciding that a plant has changed, from a residual, is the subject of classical change-detection theory [29], and event-triggered estimation is an established

response to it [30]. The predecessor of the present system used exactly this structure: a persistent step in the reconstructed collective thrust triggered a de-weighting of the pre-event stages of the window.

Chapter 4 reports why that structure was retired for this problem. A collective-thrust residual cannot distinguish a payload change from a trajectory-mode change, a sustained vertical gust, or an aggressive manoeuvre, because all of them move the same scalar. Any decision resting on that signal alone inherits the ambiguity. Two replacements were evaluated, and they should be kept apart. The *historical mass-evidence rule*, used in the cohort of Chapter 5, did not detect the change at all: it waited for the estimated mass itself to persist at the bare-airframe value, and accepted both the delay and the dependence on mass accuracy that this implies. The *deployed force-balance rule* takes the evidence from outside the estimator: the composite mass implied by the thrust reconstructed from rotor speeds and the specific force measured by the inertial unit, tested for persistence once a release command or a watchdog armed on airborne load opens the test. The first answers whether the *estimate* has returned to the empty airframe; the second whether the *measured force balance* has, and only the second is independent of the estimator's accuracy in manoeuvre. Neither is a change detector in the sense of [29], and neither has been compared with one: a CUSUM or generalised-likelihood-ratio detector on the same signals, tuned to a matched false-alarm rate, is the natural baseline, and its absence is one of the principal limitations of this dissertation (Section 5.10.3).

2.7 Positioning and Research Gap

Table 2.1 places representative works from the families above along the dimensions that matter for this dissertation. The summaries are necessarily simplified to the formulation each work emphasises, and a dash means that the aspect is

not the subject of that work rather than that the work fails at it.

Within the works reviewed here, three aspects receive comparatively little attention, and they define the scope of this dissertation.

First, **a payload model that does not need an event to switch off.** Payload-adaptive formulations commonly estimate parameters conditioned on attachment, so that some event must tell the model when the payload has gone. A model whose payload-dependent terms are continuous functions of an estimated quantity, and vanish as that quantity returns to the bare airframe, does not need such an event for its dynamics.

Second, **the difference between a release command and a physical detachment.** Systems that grasp and release usually treat the gripper command, or a contact sensor, as the moment the payload leaves, and the mismatch between the two is rarely measured. This dissertation uses the estimate to separate the two once a release has been commanded, and measures both the delay this costs and what happens when the same evidence is consulted before any command.

Third, **an explicit account of the costs.** Confirming a release on the estimate delays it; estimating an absolute mass from an absolute thrust makes the interface sensitive to a calibration constant; and resting every lifecycle decision on one estimate makes the interface only as reliable as that estimate in manoeuvre. Chapter 5 reports these costs rather than asserting them to be small.

The dissertation consequently treats estimation and control as one architecture rather than two algorithms. Estimation is judged by whether its outputs are identifiable and useful for control; control is judged by whether it remains consistent with the quantities being estimated, and by whether it knows when it does not know.

Table 2.1: Representative related work compared along the dimensions relevant to payload grasp and release. Entries summarise the formulation emphasised in each cited work; "—" means the aspect is not its subject.

Work	Payload representation	Signals used	Estimated quantities	Attach / release known?	Release confirmation	Controller	Evidence	Principal limitation for this problem
$\mathcal{L}_1$-adaptive NMPC [18]	lumped matched force and torque disturbance	state estimate	disturbance (adaptation law)	not required	—	NMPC with $\mathcal{L}_1$ augmentation	flight experiments	no payload quantity; presence not represented
Adaptive INDI [19]	lumped inner-loop disturbance	angular acceleration, actuator feedback	control effectiveness	not required	—	incremental attitude control	flight experiments	attitude loop only; no payload quantity
NeuroMHE [20]	external disturbance wrench	state measurements	disturbance wrench; learned MHE weights	not required	—	disturbance-compensated tracking	simulation and flight	payload not separated from other disturbances
Online parameter estimation [24, 25]	vehicle mass, centre of mass and inertia	onboard inertial and actuator signals	mass, CoM, inertia (and actuator parameters)	parameters assumed slowly varying	—	not the focus	flight data	no release event; not coupled to a predictive controller
IMU-based inertia estimation [23]	rigid-body inertia	IMU and rotor inputs	inertia	parameters assumed constant	—	not the focus	experiments	inertia only; no grasp or release
External wrench estimation [22]	external wrench	inertial and rotor signals	external force and torque; collisions	not required	contact events from the wrench	reflex reaction	flight experiments	payload not represented as mass and inertia
Aerial grasping [10]	rigid grasped object	external motion capture	object parameters after grasp	grasp commanded and known	—	cascaded tracking control	flight experiments	attachment assumed known
This dissertation	centred rigid point mass on a nominal vertical arm	rotor speeds, odometry	composite mass; inertia derived	not given to the estimator	command-armed, estimator-confirmed detachment; force-balance confirmation with loss watchdog	NMPC over PX4 rate loop	software-in-the-loop only	mass collapses in manoeuvre; thrust-map sensitive; no hardware

Chapter 3

Modeling and Parameter Structure

A quadrotor that picks up a parcel does not merely become heavier. It also becomes harder to rotate: on the platform of this work, a 0.30 kg parcel hanging about a quarter of a metre below the rotor plane adds fifteen percent to the mass but more than doubles the roll and pitch inertia (Section 3.3.2). These are two distinct changes to the equations of motion, and a controller that compensates only the first loses inner-loop bandwidth in a way that mass adaptation cannot explain.

This chapter derives the model that the estimator and the controller share and makes explicit which of its parameters are identified online. Its structural claim is that, for a payload grasped on the thrust axis, both changes are governed by a *single* online scalar. The composite mass m_T is identified from the translational channel; the roll and pitch inertia increment then follows from m_T by rigid-body algebra and the nominal vertical arm of the gripper. No payload mass is supplied to either optimiser, and no attach or release notification to the estimator. The nominal arm is a geometric prior, and Section 3.4.3 states exactly what that prior is and what it is not.

Section 3.1 defines the system and its assumptions, Section 3.2 its dynamics, and Section 3.3 the coupling between payload mass and inertia. Section 3.4 reduces the payload to one online degree of freedom and explains why the estimator keeps away from the rotational channel. Section 3.5 describes the

measured input and the interface to the autopilot, and Section 3.6 collects the modelling assumptions and their limits. The estimator and controller models built on this chapter are collected in Section 4.2; their numerical form, initialisation and constraints, the residual specific force and the inner-loop scheduling belong to Chapter 4; the experimental evidence for the design choices belongs to Chapter 5.

3.1 System Definition and Modelling Assumptions

Two reference frames are used throughout. The world frame $\mathscr{W}$ is East–North–Up (ENU), so that gravity acts along $-z$. The body frame $\mathscr{B}$ is Front–Left–Up (FLU) with its origin at the centre of mass of the *bare airframe*, on the thrust axis. This is the point to which the autopilot's odometry and the inertial unit refer; in the simulated airframe the rotor plane lies 0.032 m above it. The origin is fixed in the body, so once a payload is attached it is *not* the centre of mass of the composite vehicle (Section 3.2.2). The thrust vector lies on the body z axis through the origin.

The state vector and the control input are

$$x = \left[\boldsymbol{p}; \boldsymbol{v}; \boldsymbol{q}; \boldsymbol{\omega}\right] \in \mathbb{R}^{13}, \qquad u = \left[T; \boldsymbol{\tau}\right] \in \mathbb{R}^{4}, \tag{3.1}$$

where $\boldsymbol{p}$ and $\boldsymbol{v}$ are the position and velocity in $\mathscr{W}$, $\boldsymbol{q}$ is the unit quaternion encoding the attitude of $\mathscr{B}$ relative to $\mathscr{W}$ (Section 3.1.1), $\boldsymbol{\omega}$ is the body rate in $\mathscr{B}$, T is the collective thrust along the body z axis, and $\boldsymbol{\tau}$ is the body torque. We abbreviate $\boldsymbol{e}_3 = [0,0,1]^\top$, and write m_B, m_P and $m_T = m_B + m_P$ for the bare-airframe, payload and composite masses.

Three assumptions define the system.

1. *The bare airframe is balanced about the body origin*, so that its centre of

mass and the body origin coincide when no payload is attached. This is an assumption about the vehicle, not a consequence of the frame choice.

2. *The payload is a point mass m_P rigidly attached on the thrust axis*, at the body-frame offset $\boldsymbol{r}_p = [0, 0, r_z]^\top$ with $r_z < 0$: the load hangs below the airframe. The gripper aligns the vehicle over the parcel before attaching; the residual horizontal offset that alignment leaves is treated as an approximation in Section 3.6.

3. *Rotor thrust follows the static quadratic law* of Section 3.5, with calibrated coefficients.

3.1.1 Attitude representation

Attitude is represented by a unit quaternion under the *Hamilton* convention with the scalar part first,

$$\boldsymbol{q} = [q_w, q_x, q_y, q_z]^\top, \qquad \|\boldsymbol{q}\| = 1, \tag{3.2}$$

encoding the rotation from $\mathscr{B}$ to $\mathscr{W}$. The corresponding rotation matrix is

$$R(\boldsymbol{q}) = \begin{bmatrix} q_w^2+q_x^2-q_y^2-q_z^2 & 2(q_xq_y-q_wq_z) & 2(q_xq_z+q_wq_y) \\ 2(q_xq_y+q_wq_z) & q_w^2-q_x^2+q_y^2-q_z^2 & 2(q_yq_z-q_wq_x) \\ 2(q_xq_z-q_wq_y) & 2(q_yq_z+q_wq_x) & q_w^2-q_x^2-q_y^2+q_z^2 \end{bmatrix}. \tag{3.3}$$

Three conventions are fixed here and used throughout: the Hamilton product, the scalar-first component order of (3.2), and $\boldsymbol{\omega}$ resolved in $\mathscr{B}$.

One scalar derived from (3.3) is used repeatedly: the fraction of the collective thrust that acts vertically. With roll ϕ and pitch θ in the ZYX Euler convention $R = R_z(\psi)R_y(\theta)R_x(\phi)$, it is the $(3,3)$ entry of the rotation matrix,

$$\cos\phi\cos\theta \;=\; \boldsymbol{e}_3^\top R(\boldsymbol{q})\,\boldsymbol{e}_3 \;=\; 1-2\big(q_x^2+q_y^2\big). \tag{3.4}$$

The implementation evaluates the right-hand side directly from the quaternion.

3.2 Quadrotor Dynamics with a Rigid Payload

3.2.1 Kinematics

Translational kinematics is the definition of velocity,

$$\dot{\boldsymbol{p}} = \boldsymbol{v}. \tag{3.5}$$

With the body rate resolved in $\mathscr{B}$, attitude kinematics is the Hamilton product of the attitude with the pure quaternion of the body rate,

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\,\boldsymbol{q} \otimes [0, \boldsymbol{\omega}] = \tfrac{1}{2}\,\Xi(\boldsymbol{q})\,\boldsymbol{\omega}, \qquad \Xi(\boldsymbol{q}) = \begin{bmatrix} -q_x & -q_y & -q_z \\ q_w & -q_z & q_y \\ q_z & q_w & -q_x \\ -q_y & q_x & q_w \end{bmatrix}. \tag{3.6}$$

The body rate enters on the *right* of the product because it is resolved in the body frame; a world-frame rate would enter on the left. The implementation uses the matrix form, which is linear in $\boldsymbol{\omega}$. One property of Ξ matters for prediction: $\boldsymbol{q}^\top \Xi(\boldsymbol{q}) = \mathbf{0}^\top$ for every quaternion, so $\boldsymbol{q}^\top \dot{\boldsymbol{q}} = 0$ and the *continuous* kinematics preserve $\|\boldsymbol{q}\|$ exactly. The discretised kinematics do not: one explicit fourth-order Runge–Kutta step reproduces the exact rotation only to fourth order in $\|\boldsymbol{\omega}\|\Delta t$, and the norm drifts by a term of sixth order. The controller integrates (3.6) across its horizon without renormalising and evaluates (3.3) on the unnormalised prediction, so a drift would scale the predicted thrust direction by $\|\boldsymbol{q}\|^2$. At the controller's step of 0.05 s the drift over the twenty-step horizon is 3×10^{-11} at $1\,\mathrm{rad\,s^{-1}}$, 2×10^{-8} at $3\,\mathrm{rad\,s^{-1}}$ and 9×10^{-6} at $8\,\mathrm{rad\,s^{-1}}$. Integrated along the measured body rates of all 96 flights of Section 5.8.2 (largest com-

ponent $1.6\,\mathrm{rad\,s^{-1}}$), the largest drift over any horizon was 3×10^{-10}. Each solve starts from the measured unit quaternion, so the drift does not accumulate between solves, and renormalisation was omitted as numerically irrelevant at these rates rather than as unnecessary in principle. The estimator, whose attitude is a measured input, normalises it before use (Section 4.2.1).

Neither (3.5) nor (3.6) contains a mass, an inertia, or any other physical parameter. *The payload therefore enters the model only through the dynamics.*

3.2.2 Translational dynamics

Newton's second law in $\mathscr{W}$, with thrust rotated from the body frame and a linear aerodynamic drag term, gives

$$\dot{\boldsymbol{v}} = \frac{1}{m_T}\Big(R(\boldsymbol{q})\,\boldsymbol{e}_3\,T - k_d\,\boldsymbol{v}\Big) - g\,\boldsymbol{e}_3, \tag{3.7}$$

where m_T is the *composite* mass of airframe plus payload, k_d is a fixed drag coefficient, and g is the gravitational acceleration. The mass — and only the mass — enters here, as the scalar gain $1/m_T$ from thrust to acceleration. If the thrust is known independently of the model, the measured motion determines m_T through (3.7) in ordinary near-hover flight, with no dedicated excitation manoeuvre; the qualification "independently of the model" is the subject of Section 3.5.

Projecting (3.7) onto $\boldsymbol{e}_3$ and neglecting the vertical component of the drag term gives the vertical force balance

$$m_T\big(g + a_z\big) \;=\; T\cos\phi\,\cos\theta, \tag{3.8}$$

with $\cos\phi\cos\theta$ from (3.4) and $a_z = \dot{v}_z$. It is used to initialise the estimator (Section 4.3.5).

Reference point of the translational state. Equation (3.7) is Newton's law for the centre of mass of the composite body, whereas $\boldsymbol{p}$ and $\boldsymbol{v}$ in (3.1) are the position and velocity of the body origin, the point the odometry reports. The two coincide only for the bare airframe. With a payload attached the composite centre of mass lies at the body-frame offset $\boldsymbol{c}$ of (3.12), and the transport theorem for a rigid body gives

$$\boldsymbol{v}_C = \boldsymbol{v} + R(\boldsymbol{q})\,\big(\boldsymbol{\omega}\times\boldsymbol{c}\big), \qquad \dot{\boldsymbol{v}}_C = \dot{\boldsymbol{v}} + R(\boldsymbol{q})\,\big(\dot{\boldsymbol{\omega}}\times\boldsymbol{c} + \boldsymbol{\omega}\times(\boldsymbol{\omega}\times\boldsymbol{c})\big). \tag{3.9}$$

Writing (3.7) for the origin therefore omits the second term of $\dot{\boldsymbol{v}}_C$. With $\boldsymbol{c} = c\,\boldsymbol{e}_3$ and $c = (m_P/m_T)\,r_z$, its body-frame components are $c\,\big[\dot{\omega}_y + \omega_x\omega_z,\ -\dot{\omega}_x + \omega_y\omega_z,\ -(\omega_x^2+\omega_y^2)\big]^\top$: the horizontal part is driven by the angular acceleration and averages out over a manoeuvre, and the part along the thrust axis, which is what biases a mass estimate, is quadratic in the roll and pitch rates. Both scale with $|c|$, at most $0.034\,\mathrm{m}$ here. On the $4\,\mathrm{m\,s^{-1}}$ figure-eight of Chapter 5 the measured body rates and angular accelerations had root-mean-square values of $0.57\,\mathrm{rad\,s^{-1}}$ and $0.60\,\mathrm{rad\,s^{-2}}$; the omitted acceleration then has a root-mean-square value of $0.010\,\mathrm{m\,s^{-2}}$, about $0.1\,\%$ of g, and a mean component along the thrust axis of $-2\times10^{-4}\,\mathrm{m\,s^{-2}}$, about $0.002\,\%$ of g (medians over the 24 flights at that speed in Section 5.8.2). That is two to three orders of magnitude below the manoeuvre-induced mass bias reported there, and replaying the estimator on the composite-centre velocity of (3.9) instead of the origin velocity left its mass error unchanged (Section 5.8.2). The model therefore keeps the origin as its reference point and omits the transport term; it is listed with the other approximations in Section 3.6. The omission is specific to this geometry and these manoeuvres: the term grows with m_P/m_T, with the arm and with the square of the attitude rate, and a longer suspension or aggressive attitude manoeuvres would have to model it.

3.2.3 Rotational dynamics

Let $J_B = \mathrm{diag}(J_{xx}, J_{yy}, J_{zz})$ denote the inertia of the bare airframe about its own centre of mass, and J_T the composite inertia derived in Section 3.3.1. Euler's equation in the body frame is

$$\dot{\boldsymbol{\omega}} = J_T^{-1}\Big(\boldsymbol{\tau} - \boldsymbol{\omega} \times J_T\boldsymbol{\omega}\Big), \tag{3.10}$$

where $\boldsymbol{\omega} \times J_T\boldsymbol{\omega}$ is the gyroscopic coupling. Equation (3.10) holds in this form only when the inertia and the applied moments refer to the same point, and for a free-flying body that point must be the centre of mass. The composite inertia must therefore be taken about the composite centre of mass, which *moves* when a payload is attached (Section 3.3.1). For the centred payload of assumption 2 that centre of mass stays on the thrust axis, so thrust contributes no moment to (3.10).

Taken together, (3.7) and (3.10) give each payload-dependent quantity exactly one point of entry: the composite mass appears only as $1/m_T$ in the translational dynamics, and the inertia increment only inside J_T in the rotational dynamics. This separation is what allows the two effects to be attributed to distinct physical channels, and it is used throughout Section 3.4.

3.3 Payload-Induced Mass and Inertia Coupling

With the payload of assumption 2, define the composite mass and the *reduced mass*

$$m_T = m_B + m_P, \qquad \mu = \frac{m_B m_P}{m_B + m_P} = \frac{m_B m_P}{m_T}. \tag{3.11}$$

3.3.1 Inertia increment and the reduced mass

The inertia increment requires more care than the parallel-axis theorem applied naively, because *the centre of mass itself moves when the payload is attached*. With the airframe centre of mass at the body origin, the composite centre of mass lies on the thrust axis at

$$\boldsymbol{c} = \frac{m_P}{m_T}\,\boldsymbol{r}_p = \left[0,\, 0,\, \frac{m_P}{m_T}\,r_z\right]^{\top}. \tag{3.12}$$

The theorem must be applied about this *new* centre of mass, and both bodies contribute, since the airframe no longer sits at the centre of mass either.

Consider a roll or pitch axis and let $d = |r_z|$ denote the lever arm. From (3.12), the distances of the two bodies from the composite centre of mass are

$$\underbrace{\big\|\mathbf{0} - \boldsymbol{c}\big\| = \frac{m_P}{m_T}\,d,}_{\text{airframe}} \qquad \underbrace{\big\|\boldsymbol{r}_p - \boldsymbol{c}\big\| = \Big(1 - \frac{m_P}{m_T}\Big)d = \frac{m_B}{m_T}\,d\,.}_{\text{payload}} \tag{3.13}$$

The payload's distance is *smaller* than d. Adding the two parallel-axis contributions and simplifying,

$$\begin{aligned}
\Delta J &= m_B\Big(\frac{m_P}{m_T}d\Big)^2 + m_P\Big(\frac{m_B}{m_T}d\Big)^2 \\
&= \frac{d^2}{m_T^2}\Big(m_B\,m_P^2 + m_P\,m_B^2\Big) = \frac{d^2}{m_T^2}\,m_B\,m_P\big(m_P + m_B\big) \\
&= \frac{m_B\,m_P}{m_T}\,r_z^2 = \mu\,r_z^2.
\end{aligned} \tag{3.14}$$

The factor $(m_B + m_P)$ produced by the sum cancels one power of m_T in the denominator, and the two contributions collapse onto the reduced mass (3.11) — the same quantity that governs the two-body problem, and for the same reason: the inertia of a rigid pair about its centre of mass depends only on the *relative* position of the two bodies.

Equation (3.14) also identifies a natural error. Writing $\Delta J = m_P r_z^2$ — treating the payload as orbiting a fixed body origin, and forgetting both the airframe's own contribution and the displacement of the centre of mass — overestimates the increment by the factor m_T/m_B, which is 14.5 % at the heavier payload of this work.

The same computation carried out for the full tensor gives, for an on-axis point mass,

$$J_T \;=\; J_B + \mu\Big(\|\boldsymbol{r}_p\|^2 I_3 - \boldsymbol{r}_p\,\boldsymbol{r}_p^{\top}\Big) \;=\; J_B + \operatorname{diag}\big(\mu r_z^2,\, \mu r_z^2,\, 0\big). \tag{3.15}$$

Three consequences are used below. The increment is *isotropic* in roll and pitch, so a single scalar $\Delta J = \mu r_z^2$ describes both. It is *diagonal*: every off-diagonal product of inertia contains a horizontal arm and vanishes. And the yaw increment is exactly zero for a point mass on the yaw axis; the parcel's own rotational inertia about its centre is the only yaw contribution, and Section 3.6 declares its omission.

3.3.2 Magnitudes on the experimental platform

The factor by which the payload slows the roll and pitch response to a given torque is the inertia ratio

$$\kappa \;=\; \frac{J_{xx} + \Delta J}{J_{xx}}. \tag{3.16}$$

For the two payloads used in the experiments, attached at the nominal arm $r_z = -0.27\,\mathrm{m}$ on the platform of this work ($m_B = 2.0643\,\mathrm{kg}$, $J_{xx} = 0.0142\,\mathrm{kg\,m^2}$), equations (3.11), (3.14) and (3.16) give

	μ [kg]	ΔJ [kg m^2]	κ
$m_P = 0.15\,\mathrm{kg}$	0.140	0.0102	1.72
$m_P = 0.30\,\mathrm{kg}$	0.262	0.0191	2.34

(3.17)

A 0.30 kg parcel on a 2.06 kg airframe is a 15 % change of mass but a more than *two-fold* change of roll and pitch inertia. Adapting the mass alone, in the thrust channel, accounts for the first and not the second. The increment is governed by how far *below* the airframe the load hangs, through r_z^2, which is why the gripper geometry, and not only the parcel, sets the difficulty of the task.

3.4 Single-Parameter Estimation Structure

3.4.1 One online degree of freedom

Equations (3.11)–(3.15) express every payload-dependent quantity of the model through two numbers, the payload mass m_P and the vertical arm r_z, and the two are not on an equal footing.

The mass changes from flight to flight and within a flight: it is zero before a grasp, takes the parcel's value after it, and returns to zero at release, possibly in several steps if a load is lost piecewise. It is also *observable*, through the gain $1/m_T$ of (3.7). It is therefore the quantity estimated online, in the form of the composite mass m_T, and the payload mass follows as

$$m_P = (m_T - m_B)^+, \tag{3.18}$$

where $(\cdot)^+$ denotes the positive part, because the mass estimate is permitted to fall slightly below m_B to leave room for calibration error.

The vertical arm, by contrast, is a property of the gripper rather than of the parcel. It is fixed by where the gripper holds the load relative to the rotor plane, and it enters the equations only through the inertia increment, which is weakly excited in delivery flight. It is therefore held at a nominal value rather

than estimated. The inertia increment consumed by the model is then a function of the single online scalar,

$$\boxed{\Delta J(m_T) \;=\; \frac{m_B\,(m_T - m_B)^{+}}{m_T}\; r_z^2} \tag{3.19}$$

which vanishes when the mass estimate returns to the bare-airframe value. No attach or release event is needed to switch the inertia on or off: the algebra does it, continuously, as the estimate moves. The mass is estimated; the inertia is derived from it and is never an independent decision variable.

3.4.2 Which channel identifies the mass

Equation (3.19) makes the rotational dynamics depend on m_T. Should the *estimator* also be allowed to use that dependence, so that the angular-rate residual contributes information about the mass?

In principle it should: the increment is larger than the airframe inertia itself (Section 3.3.2), so any roll or pitch manoeuvre carries a strong mass signature. In practice the rotational channel of this platform is not modelled well enough to carry an absolute quantity. Three independent errors were found in it: the calibrated airframe inertia differs substantially from the simulator's model description; rotor drag acting through the vertical offset of the rotor plane produces a velocity-dependent body moment that (3.10) does not contain; and the reconstructed torque of an unloaded vehicle carries a small constant bias (Section 3.6). None prevents the vehicle from flying, but an optimiser that is allowed to move m_T inside the rotational model will use the mass to absorb all three. In development flights with that freedom the estimator overestimated the loaded mass by 28 % to 64 % at $4\,\mathrm{m\,s^{-1}}$ and the flight diverged; with the airframe inertia corrected to the simulator's value it still overestimated the mass by 29 % in the figure-eight and by 39 % after release, so that the release was never confirmed.

The estimator therefore does not integrate the rotational dynamics at all. It keeps the translational dynamics (3.7), supplies the measured attitude as a known input instead of estimating it, and holds the composite mass and the velocity as its only states (Section 4.2.1). The mass then enters the estimator's model only where the model — a thrust, a mass and gravity — is trustworthy, and no inertia, torque or arm enters it.

Removing only the *direct* sensitivity of the rotational dynamics to the mass would not be enough. An estimator that integrates the rotational dynamics keeps two indirect couplings. Within a window, the angular-rate residual shapes the estimated attitude, and the attitude sets the direction of the thrust in (3.7), so an error in the rotational model reaches the mass through the attitude. And any payload inertia it carries in its rotational model must come from a mass estimate, so a biased estimate changes the next window's rotational model. The estimator of this work has neither path: it has no estimated attitude through which an error could pass, and no inertia into which a mass could be fed back. Section 5.8.2 compares it with a full-state estimator that removed only the direct path (Section 5.2.1).

3.4.3 What is estimated, what is derived, and what is fixed

Table 3.1 records the identity of every parameter of the model. One scalar is an online degree of freedom; everything else is derived from it by the algebra of Section 3.3, calibrated once for the airframe, or held at a nominal value.

Three entries of the table invite objection.

Why not augment the inertia increment as well? Because it is nearly unobservable in the flight regime of interest, and because the channel through which it would be observed is the one this platform models least well. The incre-

Table 3.1: Model parameters, classified by identity. Only the composite mass is a decision variable of the estimator; the inertia increment is generated from it.

Symbol	Meaning	Dim.	Identity	Value / enters
m_T	composite mass	1	**MHE decision variable**	(3.7)
ΔJ	roll/pitch inertia increment	1	derived, (3.19); absent from the estimator	(4.6)
m_P	payload mass	1	derived, $(m_T - m_B)^+$	(3.19)
r_z	payload vertical arm	1	nominal gripper geometry	$-0.27\,\mathrm{m}$
$\boldsymbol{c}_{xy}$	CoM horizontal offset	2	**not modelled**; held at $\mathbf{0}$	—
m_B	bare-airframe mass	1	calibration	$2.0643\,\mathrm{kg}$
J_B	bare-airframe inertia	3	calibration	$\mathrm{diag}(0.0142, 0.0142, 0.0210)$ $\mathrm{kg\,m^2}$
k_f, k_m	thrust and yaw-reaction coefficients	2	calibration	$8.55 \times 10^{-6}\,\mathrm{N\,s^2}$, $0.016\,\mathrm{m}$
k_d, g	drag coefficient, gravity	2	constants	$0.05\,\mathrm{kg\,s^{-1}}$, $9.81\,\mathrm{m\,s^{-2}}$

ment acts only through $\dot{\omega}$, which a delivery task spent largely in near-hover, low-rate flight excites weakly; a free ΔJ would let the optimiser explain measurement noise and the rotational model errors of Section 3.4.2 with a quantity the data barely constrain. Moreover J_T appears *inverted* in (3.10) and is propagated across the horizon, so an error in it is amplified rather than damped, and inertia elements are two orders of magnitude smaller than the mass, which would worsen the scaling of the estimator's Hessian.

Why not estimate a horizontal offset? Because at the operating condition of this work there is none to estimate, and carrying one is costly. The gripper aligns over the parcel before attaching, so the offset is bounded by the alignment tolerance; an estimated offset would be identifiable only through the rotational residual, the channel that Section 3.4.2 keeps away from absolute quantities; and an offset is undefined once the payload has gone, so any parameterisation that carries one must also say what it becomes at release.

Why is a nominal r_z acceptable? Because the arm is a property of the gripper, fixed by its design, and not information about the parcel. It is a geometric prior, and the method does use it: it enters the inertia increment (3.19) of the controller, where its error propagates quadratically — a 10 % error in r_z is a 21 % error in ΔJ, which mis-sizes the predicted attitude response and the inner-loop ratio κ. It does not enter the estimator at all. The difference between the nominal and the realised arm is quantified in Section 3.6.

The information boundary. The precise claim of this dissertation is that *no task-specific payload mass enters the estimator or the controller, and no attach or release notification enters the estimator*. It is not the stronger claim that the flight stack contains no payload-related constant. Two such constants remain and are disclosed: the nominal arm r_z above, and the platform's 0.30 kg payload *envelope*, which sets the inertia floor of the attach-time bootstrap (Section 4.6.3). Both are specifications of the airframe and its gripper, in the same category as J_B or the actuator envelope, and neither is the mass of the parcel carried on a particular flight. The controller's own release command is used to open the release confirmation, to start its timeout, to end the inertia floor and to classify an acceptance as a delivered rather than a lost load (Chapter 4); it does not set the payload mass.

3.5 Measured-Input Reconstruction and Model Interfaces

3.5.1 The physical input

The thrust and torque in (3.7) and (3.10) must be known to the estimator. They are reconstructed from measured rotor speeds ω_i through the quadratic rotor law

$F_i = k_f \omega_i^2$ and the linear map from the four rotor thrusts to the wrench. For the X configuration of this platform, with rotor positions (x_i, y_i), $|x_i| = |y_i| = l = 0.174\,\mathrm{m}$, and spin directions $d_i \in \{\pm 1\}$,

$$\begin{bmatrix} T_{\text{phys}} \\ \tau_{\text{phys}} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ -l & l & l & -l \\ -l & l & -l & l \\ -k_m & -k_m & k_m & k_m \end{bmatrix}}_{M} \begin{bmatrix} F_1 \\ F_2 \\ F_3 \\ F_4 \end{bmatrix}, \qquad F_i = k_f \omega_i^2, \tag{3.20}$$

whose rows are $T_{\text{phys}} = \sum_i F_i$, $\tau_x = \sum_i y_i F_i$, $\tau_y = -\sum_i x_i F_i$ and $\tau_z = -k_m \sum_i d_i F_i$, with $k_f = 8.55 \times 10^{-6}\,\mathrm{N\,s^2}$ and yaw-reaction coefficient $k_m = 0.016\,\mathrm{m}$. The first two moment components are lever-arm products; the third is the aerodynamic reaction torque, proportional to rotor thrust and opposed to the rotor's own spin.

This is a wrench, not a state estimate. Equation (3.20) returns the wrench the *actuators generate*. It excludes $J_T \dot{\omega}$ and rotor gyroscopic terms, which is what makes it usable as the known input u^{phys} of the estimator rather than as a second, competing estimate of the state. The estimator uses only its first row, T_{phys}. Rotor speeds are sampled at 250 Hz and averaged over each 10 Hz estimation interval, consistent with a zero-order-hold discretisation of (3.7).

Why the commanded input cannot be used. Let the controller compute its thrust command from an assumed mass $\tilde{m}$, so that in hover $T_{\text{cmd}} = \tilde{m} g$. Feeding T_{cmd} into (3.7) and solving for the mass yields $m_T = T_{\text{cmd}}/g = \tilde{m}$ for *any* $\tilde{m}$: the relation is satisfied identically, the residual carries no information, and the estimate would reproduce whatever the controller already believed. Any inversion error, saturation or motor lag between command and rotor is likewise invisible. Equation (3.20) breaks this circularity because rotor speeds are a physical measurement into which no model state enters, taken downstream of allocation and the motor dynamics. The principle recurs throughout the design:

an adaptive loop must be fed by a signal that is not itself a product of the adaptation. The difference is not academic; the executed yaw torque recorded in flight is about a third of the intended value (Section 5.1).

3.5.2 The interface to the autopilot

The deployed system is a cascade: the predictive controller is the outer loop, and the autopilot's body-rate controller and mixer are the inner loop. Three torque-like quantities exist in it and must not be confused.

1. The *predicted torque* τ is a decision variable of the controller's problem. It shapes the predicted state trajectory through (4.6) and never leaves the solver.

2. The *body-rate setpoint* is what the autopilot receives. After each solve the controller publishes, at 50 Hz, a collective thrust and a body-rate setpoint interpolated along the latest predicted trajectory: the thrust from the predicted input sequence, converted to a normalised throttle by (4.21), and the body rate from the predicted state sequence one stage ahead, starting at its second node. The roll and pitch rate setpoints are then scaled for the payload inertia (Section 4.6). The autopilot's rate loop and mixer turn this setpoint into rotor commands.

3. The *physical torque* τ_{phys} is what the rotors actually produced, reconstructed from measured rotor speeds by (3.20). No optimiser uses it; it is logged as a diagnostic.

The division is deliberate: the rate loop runs at a bandwidth the 20 Hz solver cannot match, and leaving allocation on the autopilot keeps saturation handling next to the hardware. It is also why the estimator's thrust is evaluated in the *forward* direction from measured rotor speeds rather than read off the solver:

M applied to what the motors did is a measurement, whereas the solver's input is an intention. The ROS layer never evaluates the inverse allocation; its role in sizing the controller's input bounds is discussed with the actuator envelope in Section 4.4.4.

Figure 3.1 traces the complete accounting: what is measured, what is estimated, what is generated by algebra, where each quantity is consumed, and how the loop closes.

3.6 Modelling Assumptions and Limitations

The relations of this chapter fall into two classes, and the distinction matters for how the results should be read.

Relations that hold exactly. For a payload on the thrust axis, the vertical displacement (3.12) of the centre of mass is collinear with the thrust, so the thrust produces no moment about the composite centre of mass; this is a consequence of geometry, not an omitted term. The increment (3.14) is exact for a rigid pair of point masses, and so is the zero yaw increment noted after (3.15).

The remaining relations are approximations whose validity depends on the operating condition.

The payload is assumed centred. The model sets the horizontal attachment offset to zero. A residual offset r_{xy} left by imperfect alignment produces a constant trim moment $m_P g\, r_{xy}$ and a roll/pitch inertia error μr_{xy}^2. At the 0.03 m attachment tolerance of the gripper and 0.30 kg these are 0.09 N m, or 18 % of the roll authority, and 0.4 % of the increment. The first is supplied by the integral action of the autopilot's rate loop; the second is negligible.

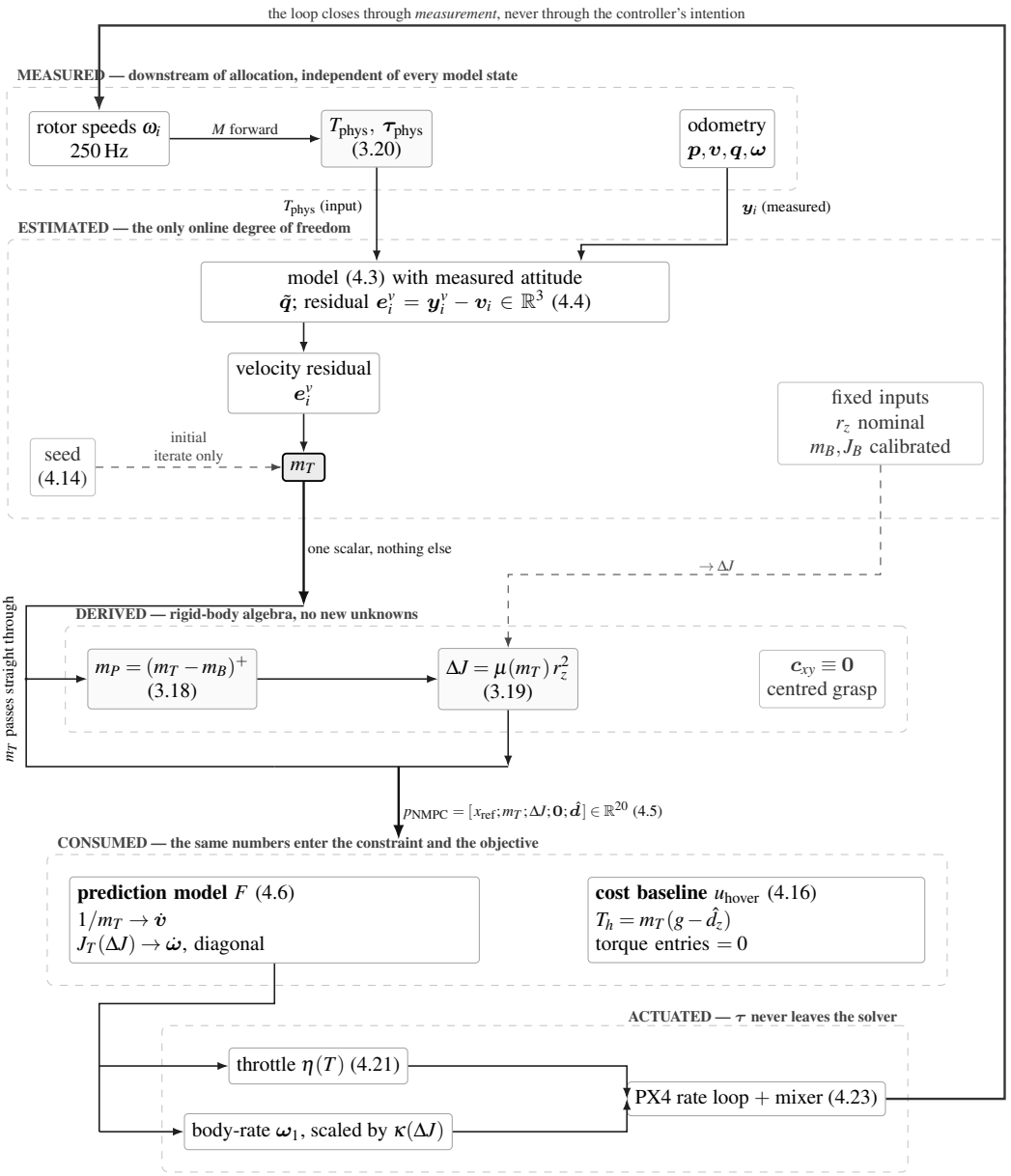


Figure 3.1: How the payload parameter reaches the controller. In the estimator the window's measured velocities and its known inputs — the measured attitude and the reconstructed thrust — meet in one velocity residual (4.4)**, which identifies the single boxed decision variable. Everything in the DERIVED band follows from m_T by the algebra of Section 3.3 and introduces neither a new unknown nor a new sensor; the inertia increment vanishes by itself when the estimate returns to the bare airframe, so no release event is needed to remove it. The estimator has no rotational model. The horizontal-offset parameters are held at zero by the operating condition, not estimated. The loop closes through measured rotor speeds, never through the moment the solver intended. The residual specific force $\hat{d}$ in the controller's parameter vector is estimated from the same measurements but is not a payload parameter (Section 4.4.3).**

The arm is nominal, not measured. Both optimisers use $r_z = -0.27\,\mathrm{m}$ for every flight. The realised arm is set by the height at which the vehicle grasps the parcel, which is commanded in the autopilot's local frame whose vertical zero drifts from flight to flight; it varies between about 0.25 m and 0.30 m. Over that range the true increment for 0.30 kg lies between 0.0164 and $0.0236\,\mathrm{kg\,m^2}$ against the modelled $0.0191\,\mathrm{kg\,m^2}$ — an error of -14 to $+23$ percent, and a true inertia ratio between 2.15 and 2.66 against the modelled 2.34. Every result in this dissertation carries an arm error of this size, unmeasured on a per-flight basis.

The translational state refers to the body origin. Equation (3.7) is applied to the body origin rather than to the composite centre of mass, which omits the transport acceleration of (3.9). Its thrust-axis component is about 0.002 % of g in the $4\,\mathrm{m\,s^{-1}}$ figure-eight, and removing the omission in replay did not change the mass error (Section 3.2.2).

The parcel's own inertia is neglected. The payload is treated as a point mass. A box of side 0.15 m has a central inertia of about $0.004\,m_P$, about 2 % of the roll/pitch increment and the only yaw contribution of a centred load.

The airframe rotational model is approximate. The calibrated airframe inertia $J_{xx} = 0.0142\,\mathrm{kg\,m^2}$ is about 60 % of the value in the simulator's model description, and a least-squares fit of bare-airframe pitch dynamics recovered an effective inertia 1.07–1.18 times the latter; and the rotor-drag moment and the constant torque bias of order 0.01 N m described in Section 3.4.2 are not modelled. These errors are the reason the mass is kept off the rotational channel, and the reason the estimator does not integrate that channel at all (Section 3.4.2). They remain in the controller's prediction model, where they appear as an unmodelled moment that the rate loop absorbs.

The inertia is treated as constant across a transfer. Equation (3.10) omits the $\dot{J}_T\omega$ term that a time-varying inertia would contribute. The omission is exact on either side of a grasp or release and wrong only for the duration of the transfer itself, which the controller's prediction shares with the plant's rate loop.

The rotor model is static. The thrust coefficient k_f is a constant, and is known to lose validity under large induced inflow during fast translation [31]; the mass channel recovers m_T from an absolute thrust and inherits any error in k_f in full. The yaw reaction of (3.20) is the steady-state term only, omitting the rotor angular-acceleration contribution $\sum_i J_r\dot{\omega}_i$, and rotor gyroscopic moments are not modelled. These terms are negligible in hover and mild manoeuvre but not during rapid yaw. Because m_T is recovered from an absolute thrust, a relative error in k_f passes one-for-one into $\hat{m}$; this sensitivity is analytical and has not been measured in the evaluated configuration (Section 5.10). Translational data cannot separate the two: the estimator flags a thrust-map error that drives its estimate outside a physical band (Section 4.3.3) but does not correct it.

3.7 Chapter Summary

This chapter addressed the first research objective and the structural part of the second. For a rigid payload grasped on the thrust axis, the roll and pitch inertia increment is the reduced mass times the square of the vertical arm, so the composite mass is the only payload quantity that has to be identified online; the inertia follows from it algebraically and vanishes with it. The mass is identified on the translational channel, from a thrust reconstructed from measured rotor speeds. The estimator does not integrate the rotational dynamics at all — the measured attitude enters as a known input — so the optimiser cannot use the mass to absorb rotational model error. The structure rests on stated assumptions — a centred grasp, a nominal arm and a calibrated rotor model — whose limits

are collected in Section 3.6. What remains open is the numerical form of the estimator and the controller, and how a release is recognised; Chapter 4 takes these up.

Chapter 4

The MHE–NMPC Framework

Chapter 3 established *what* is estimated and why. This chapter states *how*. Section 4.2 states the two models side by side — the velocity–mass estimator and the controller's prediction model. Sections 4.3 and 4.4 give the two optimal control problems built on them, Section 4.5 the interface that carries the estimate from one to the other, Section 4.6 the inner-loop scheduling that consumes the same estimate, and Section 4.7 the logic that decides when the controller may believe that its payload has gone.

The estimator receives no attach or release notification. The controller's model parameters follow the estimate continuously, and a release is *accepted* — declared complete, with auxiliary states reset — only on persistent evidence that the payload has left. In the deployed system that evidence is a force balance formed from rotor speeds and the inertial unit, independent of the mass estimate (Section 4.7.5). Section 4.7.1 separates what acceptance does from what the continuous adaptation does, and Section 4.7.6 states the limits of the life-cycle.

4.1 Closed-Loop Architecture

Figure 4.1 shows the information flow. Three layers are involved: the physics and flight stack (Gazebo, PX4 software-in-the-loop, and a magnetic detachable-joint gripper that produces genuine mass and inertia changes); the inner rate-control loop on the autopilot; and a ROS outer loop containing the estimator and the predictive controller.

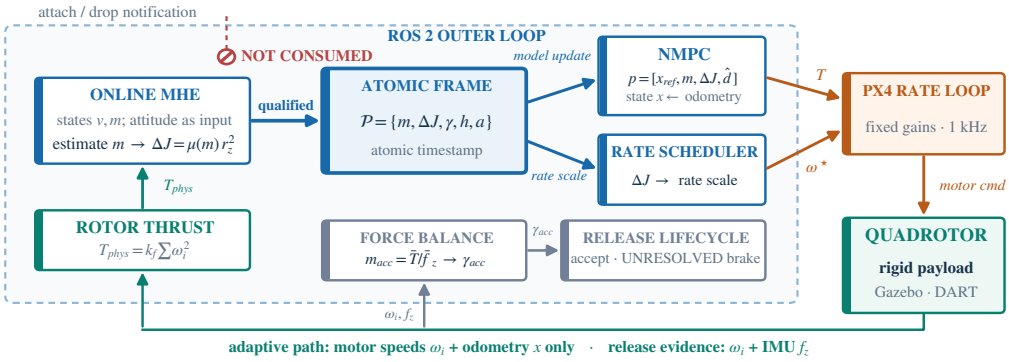


Figure 4.1: System architecture and information flow of the deployed system. The adaptive path consumes only rotor speeds and odometry: one atomic, health-gated frame carrying the composite mass and the inertia increment derived from it drives both the NMPC model and the continuous body-rate command scheduling. The release path is separate: a force balance of the reconstructed thrust against the inertial unit's specific force feeds the release lifecycle (Section 4.7.5), which does not read the frame. No attach or release notification enters either path.

Two properties of this diagram carry the argument of the dissertation.

First, **the only inputs of the adaptive path are odometry and rotor speeds.** The estimator does not read the controller's intended thrust or torque (Section 3.5), does not read the gripper's attach or release record, and does not read the simulator's joint state. The payload truth available in simulation is used exclusively for offline scoring. The release lifecycle additionally reads the specific force measured by the inertial unit (Section 4.7.5); like the rotor speeds it is a measurement, and it does not reach the estimator or the controller's model.

Second, **there is exactly one interface.** The estimator publishes a single atomic

frame (Section 4.5) containing the mass, the derived inertia, a no-payload evidence score and a health flag. Both consumers — the NMPC model parameters and the inner-loop command scaling — read that one frame. An earlier design in which mass and inertia travelled on separate topics, each with its own latency and validity, made it possible for the controller to hold a mutually inconsistent parameter set; the atomic frame makes that class of fault unrepresentable.

The two optimisers run at different rates and do not share a solve clock. The estimator solves at 10 Hz over a 2 s window; the controller solves at 20 Hz over a 1 s preview; setpoints are published at 50 Hz by interpolating along the most recent predicted trajectory.

4.2 Estimator and Controller Models

Chapter 3 derived one plant model: the kinematics (3.5)–(3.6), the translational and rotational dynamics (3.7) and (3.10), and the payload algebra that makes the inertia increment a function of the composite mass, (3.19). The estimator and the controller each keep only the part of that model they need, and they keep different parts. This section states the two models side by side, so that the optimisation problems of Sections 4.3 and 4.4 can refer to one place. Table 4.1 summarises them.

4.2.1 The estimator: velocity and mass

The estimator keeps the composite mass and the world-frame velocity, and nothing else,

$$x_{\mathrm{R}} = \left[\boldsymbol{v};\, m_T\right] \in \mathbb{R}^4, \qquad \dot{m}_T = 0. \tag{4.1}$$

Table 4.1: The two models of one plant: the estimator's and the controller's prediction model. Equation numbers refer to the definitions.

	Estimator	**NMPC prediction model**
State	$[\boldsymbol{v}; m_T] \in \mathbb{R}^4$ (4.1)	$[\boldsymbol{p}; \boldsymbol{v}; \boldsymbol{q}; \boldsymbol{\omega}] \in \mathbb{R}^{13}$ (3.1)
Known inputs and parameters	measured attitude $\tilde{\boldsymbol{q}}$, T_{phys}; $\mathbb{R}^5$ (4.2)	reference, m_T, ΔJ, $\boldsymbol{c}_{xy} = \boldsymbol{0}$, $\hat{\boldsymbol{d}}$; $\mathbb{R}^{20}$ (4.5)
Dynamics	translational only, attitude supplied (4.3)	(4.6)
Where the payload enters	$1/m_T$ in $\dot{\boldsymbol{v}}$; m_T a decision variable	$1/m_T$ and $\Delta J(m_T)$, runtime parameters
Rotational dynamics	not integrated	integrated with $J_T(\Delta J)$
Residual	velocity $\boldsymbol{e}^v \in \mathbb{R}^3$ (4.4)	tracking and input deviation (4.17)
Rate, horizon	10 Hz, $N = 20$, $\Delta t = 0.1$ s	20 Hz, $N = 20$, $\Delta t = 0.05$ s

Its known parameters at stage j of the window are the measured attitude and the reconstructed collective thrust,

$$p_{\text{R},j} = \left[\, \tilde{\boldsymbol{q}}_j;\; T_{\text{phys},j} \,\right] \in \mathbb{R}^5, \qquad \tilde{\boldsymbol{q}}_j = \frac{\boldsymbol{q}_j^{\text{meas}}}{\|\boldsymbol{q}_j^{\text{meas}}\|}, \tag{4.2}$$

where $\boldsymbol{q}_j^{\text{meas}}$ is the odometry attitude at the start of the stage and $T_{\text{phys},j}$ is (3.20) averaged over the stage interval. The model is the translational dynamics (3.7) with the attitude supplied rather than integrated,

$$\dot{\boldsymbol{v}} = \frac{1}{m_T}\left(\frac{R(\tilde{\boldsymbol{q}})\,\boldsymbol{e}_3}{\|\tilde{\boldsymbol{q}}\|^2}\,T_{\text{phys}} - k_d\,\boldsymbol{v}\right) - g\,\boldsymbol{e}_3 + \boldsymbol{w}, \qquad \dot{m}_T = 0, \tag{4.3}$$

with velocity process noise $\boldsymbol{w} \in \mathbb{R}^3$. The stage residual is the velocity block alone,

$$\boldsymbol{e}_j^v = \boldsymbol{y}_j^v - \boldsymbol{v}_j \in \mathbb{R}^3. \tag{4.4}$$

The mass is held constant across the window, so that a window of measurements constrains a single value rather than a free trajectory, and process noise is admitted on the velocity only; noise on the mass channel would let it drift within the window and dissolve the constraint that makes the estimate meaningful. The residual contains no acceleration: the trajectory is produced by integrating (4.3) forward under the known input, so the comparison is between velocities, never between a measured and a modelled acceleration. Differentiating the odometry velocity would amplify its noise by $1/\Delta t$, to the order of the

very effect being estimated.

Three properties of the attitude input are deliberate. First, every entry of (3.3) is a quadratic form in $\boldsymbol{q}$, so $R(\boldsymbol{q}) = \|\boldsymbol{q}\|^2 R(\boldsymbol{q}/\|\boldsymbol{q}\|)$; dividing by $\|\tilde{\boldsymbol{q}}\|^2$ in (4.3) normalises the attitude inside the model as well as before it, and a non-unit odometry quaternion cannot rescale the thrust. Second, the same quadratic form makes $R(\boldsymbol{q}) = R(-\boldsymbol{q})$ exactly, so the double cover of the rotation group needs no hemisphere alignment. Third, the attitude is an input and not a state: the cost contains no quaternion residual, neither a component-wise difference nor a norm penalty.

Why is this enough? The composite mass enters the plant at exactly one place, the gain $1/m_T$ of (3.7) (Section 3.2.2). Position kinematics, attitude kinematics and Euler's equation contain no mass, so given the direction and the magnitude of the thrust, the velocity response carries all the information about m_T that the window holds. What an estimator that integrates the full vehicle model adds is an *estimated* attitude trajectory, and that is precisely the indirect path by which errors in the rotational model reach the mass (Section 3.4.2). Supplying the measured attitude closes that path by construction: no inertia, no torque and no vertical arm enters the estimator at all. The price is that attitude measurement noise enters the thrust direction unfiltered. At an attitude noise of $\sigma_q = 0.01$ (about 1.1°) it perturbs the vertical thrust component by less than $0.02\,\%$ and the horizontal components by up to $2\,\%$ per sample, zero-mean, which the window averages. The simulator's odometry is close to ideal, so this argument is not tested by the experiments of Chapter 5.

With four states and three noise channels the quadratic subproblem is small; Section 5.8.2 reports the solve times.

4.2.2 The controller's prediction model

The predictive controller carries the payload parameters as runtime values,

$$p_{\mathrm{NMPC}} = \left[x_{\mathrm{ref}}(13);\, m_T;\, \Delta J;\, \boldsymbol{c}_{xy}(2);\, \hat{\boldsymbol{d}}(3)\right] \in \mathbb{R}^{20}, \tag{4.5}$$

where m_T is the published mass estimate, ΔJ is (3.19) evaluated at it, the two horizontal-offset entries $\boldsymbol{c}_{xy}$ are held at zero (assumption 2), and $\hat{\boldsymbol{d}}$ is the residual specific force of Section 4.4.3. The offset entries are held at zero by the controller; the evidence that modelling the offset buys nothing at the operating condition is in Section 5.8.3. The prediction model is the plant model of Section 3.2 with these parameters substituted,

$$\begin{aligned}
&\dot{\boldsymbol{p}} = \boldsymbol{v}, \qquad \dot{\boldsymbol{q}} = \tfrac{1}{2}\,\Xi(\boldsymbol{q})\,\boldsymbol{\omega},\\
&\dot{\boldsymbol{v}} = \frac{1}{m_T}\Big(R(\boldsymbol{q})\,\boldsymbol{e}_3\,T - k_d\,\boldsymbol{v}\Big) - g\,\boldsymbol{e}_3 + \hat{\boldsymbol{d}},\\
&\dot{\boldsymbol{\omega}} = J_T^{-1}\Big(\boldsymbol{\tau} - \boldsymbol{\omega}\times J_T\,\boldsymbol{\omega}\Big), \quad J_T = \mathrm{diag}\big(J_{xx}+\Delta J,\, J_{yy}+\Delta J,\, J_{zz}\big).
\end{aligned} \tag{4.6}$$

Apart from the additive residual $\hat{\boldsymbol{d}}$, nothing in (4.6) is new: the pair $(m_T, \Delta J)$ are the only payload quantities that change from one solve to the next, each enters at exactly one place, and the second is a function of the first. The controller is the only one of the two models that needs the inertia, and it obtains it from the estimate rather than estimating it.

The published mass is not the raw estimate. The estimator publishes $m_B + m_P^{\mathrm{out}}$, where m_P^{out} is the positive part (3.18) passed through a first-order filter, so the published mass never falls below m_B; the controller filters and rate-limits it again (Section 4.5.2). The estimator carries no inertia, so the inertia is never a function of a decision variable, and it has no counterpart of $\hat{\boldsymbol{d}}$, so a persistent residual cannot be used there to explain away a mass change.

4.3 Moving Horizon Estimator

4.3.1 The optimisation

The estimator is a constrained least-squares problem. At each 10 Hz tick it solves

$$\begin{aligned} \min_{x_{0:N},\, w_{0:N-1}} \quad & \left\|x_0 - \bar{x}_0\right\|_{W_0}^2 + \sum_{j=0}^{N-1} \beta_j \left(\left\|y_j - C x_j\right\|_{W_y}^2 + \left\|w_j\right\|_{W_w}^2 \right) \\ \text{s.t.} \quad & x_{j+1} = F_{\mathrm{M}}\left(x_j, p_j\right) + G w_j, \\ & \underline{m} \le m_T \le \overline{m}, \end{aligned} \tag{4.7}$$

where x is (4.1), F_{M} is one explicit fourth-order Runge–Kutta step of (4.3) driven by the known inputs p_j of (4.2), y_j is the odometry measurement, $C = \begin{bmatrix} I_3 & \mathbf{0} \end{bmatrix}$ selects the velocity, and

$$G = \begin{bmatrix} I_3 \\ 0_{1\times 3} \end{bmatrix} \tag{4.8}$$

confines the process noise to the velocity, leaving m_T noise-free within the window.

Stage j pairs the measurement y_j taken at the start of the interval with the input averaged over that interval; the newest measurement y_N enters only as the terminal node of the trajectory and carries no cost. This is an implementation convention inherited from the full-state estimator rather than a design choice, and it costs the estimate about one sample of delay: the last input interval is constrained by the dynamics but not by a measurement. A replay with y_N penalised at the weight of the other stages left the accuracy unchanged and shortened the convergence after lift-off and after detachment by 0.2 s to 0.3 s (Section 5.8.2); the flown estimator does not yet include it.

The horizon is $N = 20$ stages at $\Delta t = 0.1\,\mathrm{s}$. The choice is a compromise: a longer window averages more measurement noise, but the constraint that makes the mass identifiable — that it is *constant* over the window — becomes less defensible the longer the arc of trajectory the window spans. In development flights with an earlier estimator, on a $4\,\mathrm{m\,s^{-1}}$ figure-eight a ten-stage window recovered the mass $3.7\times$ more accurately than a twenty-stage one, while a forty-stage window sat on the lower bound for the entire flight. The horizon was not varied in the reported experiments.

4.3.2 Weights

All weight matrices follow Bryson's rule, with each diagonal entry set by a representative residual magnitude:

$$W_y = \sigma_v^{-2} I_3, \qquad W_w = 10^4 I_3, \qquad W_0 = \mathrm{diag}\big(10^3 I_3,\ \sigma_{m,0}^{-2}\big), \tag{4.9}$$

with $\sigma_v = 0.05\,\mathrm{m\,s^{-1}}$ and $\sigma_{m,0} = 3.16\,\mathrm{kg}$. The process-noise weight is deliberately *tighter* than the measurement weight: it forces persistent disagreement between measured motion and reconstructed thrust to be explained by the mass rather than by unstructured process noise. The arrival cost anchors the start of the window, with $\bar{x}_0$ taken from the previous window shifted by one stage. Its mass entry is deliberately four orders of magnitude softer than the velocity entries: it carries negligible prior information but remains non-zero so that a poorly excited window retains positive curvature in the mass direction. Setting it to zero produced 472 solver failures in an earlier configuration.

4.3.3 Mass bounds and the mass-health check

The estimator separates the numerical bound from the physical judgement. A hard physical bound would act as a prior under a thrust-map error: with a

lower bound at $0.95\,m_B$, a map reading 10 % low requires an empty-airframe estimate of $0.9\,m_B = 1.858\,\mathrm{kg}$, below the bound, and the estimate is held at the bound rather than revealing the error (Section 5.8.2).

Numerical box and health band. The box, $[0.70\,m_B,\ 5.0\,\mathrm{kg}] = [1.445\,\mathrm{kg}, 5.0\,\mathrm{kg}]$, only prevents the $1/m_T$ singularity and run-away iterates and is wide enough for a thrust-map error of about 30 % to appear in the estimate. Whether the estimate is *physically credible* is decided after each solve by a separate check against a health band,

$$m_B - 0.10\,\mathrm{kg} \;\le\; \hat{m} \;\le\; m_B + m_P^{\mathrm{env}} + 0.10\,\mathrm{kg}, \qquad \text{i.e.} \quad [1.964\,\mathrm{kg},\ 2.464\,\mathrm{kg}], \tag{4.10}$$

where $m_P^{\mathrm{env}} = 0.30\,\mathrm{kg}$ is the platform's payload envelope (Section 3.4.3). An estimate below the band is flagged as a possible thrust-map *under*-estimate, one above it as out of envelope or a thrust-map *over*-estimate; an estimate within 2 g of the numerical box, or a non-finite one, is flagged as well. The flag takes effect on the first offending solve and is cleared only after ten consecutive solves inside the band (1 s). The margins come from a replay: under the nominal thrust map the estimate ranged over 1.985 kg to 2.368 kg across 264 flights and never left (4.10). No simulator truth is used by the check.

A flagged estimate is not forwarded. The frame is marked unhealthy, so the controller keeps the last accepted model (Section 4.5.1), and the estimator records the reason. The check *detects*; it does not correct. Translational data alone cannot separate the mass from a scale error in the thrust map — (4.3) depends on them only through T_{phys}/m_T — so the estimator does not attempt to estimate both, and a thrust-map error is detected only when it drives the estimate out of (4.10). A -10 % map on the empty airframe ($\hat{m} \approx 1.86\,\mathrm{kg}$) and a 10 % map with the heavier payload ($\hat{m} \approx 2.60\,\mathrm{kg}$) are detected; a -10 % map with the heavier payload ($\hat{m} \approx 2.13\,\mathrm{kg}$) is not.

4.3.4 Input gating and the grasp gap

The reconstructed wrench is admitted as a known input only while the odometry height exceeds 0.5 m and the collective thrust exceeds 1 N; near the ground the support force decouples T_{phys} from $m_T g$. If any input is older than 0.30 s the window is discarded, the frame is marked unhealthy, and the window is rebuilt once fresh data return. The grasp hover at 0.35 m lies below the gate. In each of the ten flights inspected the window was therefore discarded during the final descent, 1–3 seconds before the gripper registered the attach, and rebuilt about six seconds later, after lift-off had begun. The grasp itself is never inside a window: the loaded estimate starts afresh from the window-balance seed below, and the attach-time convergence reported in Chapter 5 is dominated by this gap and the subsequent window fill, not by the estimator's own dynamics.

4.3.5 Initialisation

The estimator requires an initial iterate for the mass channel. Supplying the calibrated airframe mass would be a prior about the answer; the implementation instead constructs the seed from measurements.

The simplest form is the hover approximation obtained by setting $a_z = 0$ and $\phi = \theta = 0$ in the vertical force balance (3.8),

$$m^{(0)} = \text{clip}\left(\frac{T_{\text{phys}}}{g}, \underline{m}, \overline{m}\right), \tag{4.11}$$

which is prior-free because T_{phys} comes from (3.20) and involves no model state. It is valid only near hover: in development flights on a $4\,\text{m}\,\text{s}^{-1}$ figure-eight the instantaneous ratio T_{phys}/g swung from $-42\,\%$ to $30\,\%$ of the true mass, because the collective thrust also supplies centripetal acceleration and is tilted away from the vertical.

The deployed seed therefore uses the whole window. Summing (3.8) over the N stages,

$$m_T \sum_{i=0}^{N-1} \left(g + a_{z,i}\right) \;=\; \sum_{i=0}^{N-1} T_i \cos\phi_i \cos\theta_i, \tag{4.12}$$

and eliminating the accelerations by the telescoping identity

$$\sum_{i=0}^{N-1} a_{z,i} = \sum_{i=0}^{N-1} \frac{v_{z,i+1} - v_{z,i}}{\Delta t} = \frac{v_{z,N} - v_{z,0}}{\Delta t}, \tag{4.13}$$

gives a seed that requires no numerical differentiation of the velocity signal:

$$m^{(0)} \;=\; \frac{\displaystyle\sum_{i=0}^{N-1} T_i \cos\phi_i \cos\theta_i}{Ng + \dfrac{v_{z,N} - v_{z,0}}{\Delta t}}. \tag{4.14}$$

Only the two velocity endpoints of the window enter, so the seed is immune to the sample-to-sample noise a finite-difference acceleration would amplify; the tilt factor restores the vertical component that (4.11) discards; and every input is either T_{phys} or odometry, so the seed is decoupled from the estimator's own state. The implementation falls back to (4.11), and then to a neutral box value, when the window is not yet full or the denominator of (4.14) degenerates.

The seed enters the solver in two places: the initial iterate of the sequential quadratic programme, and the mean of the arrival cost. The former affects only the number of iterations, never the optimum. The latter affects the optimum in proportion to its weight, which by (4.9) corresponds to a standard deviation of several kilograms — far wider than the range of masses the vehicle can carry. The seed therefore influences where the optimiser starts, not where it converges. The same expression is used on re-initialisation after repeated solver failure (Section 4.3.7).

4.3.6 Stage weighting

The constant-payload model is invalid when a grasp or release lies inside the window. Table 4.2 gives the stage weighting in effect.

Table 4.2: Stage weighting of the MHE. The de-weighting matrices are built on the velocity, noise and arrival blocks of (4.9)**.**

Treatment	Status	Reason
Uniform, $\beta_j \equiv 1$	in effect in almost every flight	no event logic; accepts a transient of roughly one window
Residual-triggered de-weighting (pre-event stages $\times 10^{-4}$ after a collective-thrust step above 4.4 N for two frames)	enabled, rarely effective	collective-thrust steps also occur at trajectory changes; a grasp detection mostly falls inside the gap of Section 4.3.4, so the de-weighted stages have left the window before solving resumes

The de-weighting row needs a number. Across the 312 flights of the release campaign the trigger fired in 81 flights, but the stage weights of a solved window were changed in only 10; in the others the detection fell inside the grasp gap or was overtaken by the window rebuild. The design therefore effectively tolerates the change transient rather than detecting it, and the lifecycle below is designed around that delay.

4.3.7 Solver failure and re-initialisation

A solve that ends with a non-zero status, or with a non-finite iterate, is discarded. The published estimate is not updated from it and the frame is marked unhealthy. After five consecutive failures the window is discarded and rebuilt: the physical states are reset to the current measurement and the mass is re-seeded from (4.14). Re-seeding from a measurement rather than from the last accepted estimate matters because a run of failures is itself evidence that the last estimate may be corrupt; the window balance is the only mass value avail-

able at that instant that does not depend on the estimator's own history.

The seed is used only as the arrival mean and initial iterate of the rebuilt window: the published estimate stays at the last successful solution and the frame stays unhealthy until a solve succeeds, because a seed is not a solution. The estimator logs the solver status, the number of SQP iterations and the running and cumulative failure counts of every failed solve.

4.4 Adaptive NMPC

4.4.1 The optimal control problem and its input baseline

Over a prediction horizon of N stages the controller solves

$$\begin{aligned} \min_{u_{0:N-1}} \quad & \sum_{k=0}^{N-1} \Big(\big\| e_{\text{track}}(x_k, x_{\text{ref},k}) \big\|_Q^2 + \big\| u_k - u_{\text{hover}}(p) \big\|_R^2 \Big) \\ & \qquad + \big\| e_{\text{track}}(x_N, x_{\text{ref},N}) \big\|_P^2 \\ \text{s.t.} \quad & x_{k+1} = F(x_k, u_k; p_{\text{NMPC}}), \\ & \underline{T} \le T_k \le \overline{T}, \quad |\tau_{k,i}| \le \overline{\tau}_i, \end{aligned} \tag{4.15}$$

where F is one explicit fourth-order Runge–Kutta step of the prediction model (4.6) with the parameters (4.5), and the input bounds are those of Section 4.4.4. The problem is re-solved at every step; what is sent to the autopilot is not u_0 itself but the thrust and body-rate setpoints read from the predicted trajectory, as described in Section 3.5.2.

The second term penalises the deviation of the input from a *baseline* u_{hover}, recomputed from the live parameters at every solve rather than fixed when the

solver is generated:

$$T_h = m_T\left(g - \hat{d}_z\right), \qquad u_{\text{hover}} = \left[T_h,\, 0,\, 0,\, 0\right]^\top. \tag{4.16}$$

With the additive residual of (4.6), vertical equilibrium requires $T/m_T - g + \hat{d}_z = 0$, which is (4.16); $\hat{d}_z$ is a specific force, so it is scaled by the mass. Penalising $\|u\|_R^2$ outright would treat the thrust that keeps the vehicle airborne as an expense; the baseline recentres the penalty on the equilibrium input, so that R charges only genuine control effort [1]. What is particular here is that the baseline is a function of the *estimated* mass. A stale baseline — for example one computed from the unloaded mass while a parcel is carried — pulls the thrust persistently below equilibrium, and the optimum settles at a non-zero steady-state error where the marginal tracking and baseline costs balance. The signature is that increasing R makes the error worse, because R is the strength of the incorrect pull; in development flights with a heavy load and an empty-airframe baseline the tracking error stalled above 0.85 m.

The torque entries of (4.16) are zero because a centred payload requires no trim moment. The small trim left by imperfect alignment is not represented in the controller's model or cost; it is supplied by the integral action of the autopilot's rate loop (Section 5.8.3).

4.4.2 Cost and weights

The controller solves (4.15) with $N = 20$ stages at $\Delta t = 0.05\,\text{s}$, preserving a 1.0 s preview while running at 20 Hz. The residual vector is

$$\begin{aligned} y_k = \big[\, & \boldsymbol{e}_p;\ \boldsymbol{e}_v;\ \boldsymbol{e}_{q,\text{vec}};\ \boldsymbol{e}_\omega; \\ & u_k - u_{\text{hover}}(p)\,\big] \in \mathbb{R}^{16}, \end{aligned} \tag{4.17}$$

where the attitude entry is the vector part of the relative quaternion $\boldsymbol{q}_{\text{ref}}^{*} \otimes \boldsymbol{q}$, which keeps the residual on the tangent space of the rotation manifold instead of treating four quaternion components as independent Euclidean coordinates.

Weights again follow Bryson's rule, $Q_{ii} = L_i^{-2}$, with tolerances

$$L_p = 0.1\,\text{m}, \quad L_v = 0.3\,\text{m}\,\text{s}^{-1}, \quad L_q = 0.05, \quad L_\omega = 0.3\,\text{rad}\,\text{s}^{-1}, \tag{4.18}$$

and, on the inputs, $L_T = 2\,\text{N}$, $L_{\tau,xy} = 0.5\,\text{N}\,\text{m}$, $L_{\tau,z} = 0.2\,\text{N}\,\text{m}$; the torque scales are taken to be the constraint bounds themselves, since the input cannot exceed them in any case. The terminal weight rescales the stage blocks by 20 (position), 1 (velocity), 2 (attitude) and 0.2 (rate): the endpoint of the prediction is pulled tightly onto the reference in position and attitude, while relaxing the terminal rate demonstrably helps the solver.

This non-dimensionalisation is not cosmetic. Before it was applied, position and torque weights differed by a factor of tens at their natural magnitudes, the Gauss–Newton Hessian was correspondingly ill-conditioned across directions, and the stationarity residual was observed to reach 6.26×10^5 — a complete numerical breakdown rather than a slow convergence.

4.4.3 Residual specific force

The translational model at the published mass is still wrong whenever the mass estimate is: under sustained manoeuvre the loaded estimate is biased low (Section 5.6.1), and the thrust feed-forward implied by (4.6) then falls short by the same proportion, leaving a steady-state height error that only the position error can push against. The controller therefore carries an additive specific force $\hat{\boldsymbol{d}}$ [$\text{m}\,\text{s}^{-2}$], estimated at the odometry rate by the state predictor and adaptation

law of $\mathscr{L}_1$-adaptive control [18]:

$$\dot{\hat{\boldsymbol{v}}} = \boldsymbol{a}_{\text{nom}} + \hat{\boldsymbol{d}}_{\text{raw}} - a(\hat{\boldsymbol{v}} - \boldsymbol{v}), \qquad \dot{\hat{\boldsymbol{d}}}_{\text{raw}} = -\Gamma(\hat{\boldsymbol{v}} - \boldsymbol{v}), \qquad \hat{\boldsymbol{d}} = \frac{\omega_c}{s + \omega_c}\,\hat{\boldsymbol{d}}_{\text{raw}}, \quad (4.19)$$

with $a = 10\,\text{s}^{-1}$, $\Gamma = (a/2)^2$ (critically damped) and low-pass cut-off $\omega_c = 0.5\,\text{rad}\,\text{s}^{-1}$. The nominal acceleration $\boldsymbol{a}_{\text{nom}}$ is (3.7) evaluated with the *measured* collective thrust of (3.20) and the *published* mass $\hat{m}$, so $\hat{\boldsymbol{d}}$ absorbs only the part of the measured acceleration that the model at $\hat{m}$ fails to explain. It enters (4.6) additively and the baseline (4.16) through its vertical component.

Four properties keep it out of the payload accounting. It is a specific force, not a mass, so it needs no assumption about what the mass is. It is used by the controller only: it does not reach the estimator, the evidence scores (4.30) and (4.36) or any lifecycle decision. It is reset to zero whenever the controller accepts a release — a commanded release, or a load lost without a command and detected by the watchdog of Section 4.7.5 — since a residual learned against the loaded vehicle becomes a phantom force once the payload has gone. And it is held at zero while the measured thrust is below $0.4 m_B g$, because on the ground the support force would otherwise be learned as an upward disturbance. Its cost is equally plain: because it compensates the translational consequence of a mass bias, tracking results obtained with it do not measure the accuracy of $\hat{m}$, and the mass error is reported directly wherever it matters (Chapter 5). Section 5.8.1 isolates its contribution, and Section 5.7.5 compares the controller with an $\mathscr{L}_1$-type controller that uses the same law with a fixed nominal mass and no payload model.

4.4.4 Input constraints and the actuator envelope

The rotor model of Section 3.5 also defines what the vehicle can actually produce, and therefore the input constraint set of (4.15). The autopilot accepts

a normalised throttle η, which the electronic speed controllers map affinely to rotor speed before the quadratic thrust law applies:

$$\omega(\eta) = \omega_{\min} + \Delta\omega\,\eta, \qquad T(\eta) = K\,\omega(\eta)^2, \qquad K = 4k_f, \tag{4.20}$$

with $\omega_{\min} = 150\,\mathrm{rad\,s^{-1}}$ and $\Delta\omega = 850\,\mathrm{rad\,s^{-1}}$. Inverting (4.20) gives the command actually published,

$$\eta(T) = \frac{\sqrt{T/K} - \omega_{\min}}{\Delta\omega}, \tag{4.21}$$

which is exact at every mass, unlike a linear hover-referenced heuristic. Because η is clipped to $[0.05, 0.95]$ — the upper limit reserving 5 % of the command range so that the rate loop retains differential authority when all four rotors are near saturation — the attainable collective thrust is

$$T \in \big[T(0.05),\, T(0.95)\big] = [\,1.27\,\mathrm{N},\, 31.35\,\mathrm{N}\,], \tag{4.22}$$

and this is the thrust box of (4.15). The box must come from the actuators rather than from the vehicle's weight: a weight-based box such as $[0.5\,\mathrm{N}, 2m_Bg = 40.5\,\mathrm{N}]$ over-states the available thrust by 29 %, the surplus is removed silently by the clip in (4.21), and a planned recovery acceleration is then never delivered while the next solve asks for still more.

The torque bounds follow from the allocation. The ROS layer never evaluates the inverse of (3.20) — the autopilot's mixer does — but its form sizes the bounds. The rows of M are mutually orthogonal, so $M^{-1} = M^{\top}\mathrm{diag}\big(\frac{1}{4}, \frac{1}{4l^2}, \frac{1}{4l^2}, \frac{1}{4k_m^2}\big)$ and

$$F_i = \frac{T}{4} + \frac{y_i}{4l^2}\,\tau_x - \frac{x_i}{4l^2}\,\tau_y - \frac{d_i}{4k_m}\,\tau_z. \tag{4.23}$$

A unit roll or pitch moment costs $1/(4l) = 1.44\,\mathrm{m^{-1}}$ of differential rotor thrust, whereas a unit yaw moment costs $1/(4k_m) = 15.6\,\mathrm{m^{-1}}$, a factor of 10.9. At a

hover thrust of 20 N each rotor carries 5 N and has 2.84 N of headroom to the 7.84 N per-rotor ceiling implied by (4.22); that headroom buys 1.98 N m of roll authority but only 0.18 N m of yaw. Against the bounds $\overline{\tau}_{x,y} = 0.5\,\mathrm{N\,m}$ and $\overline{\tau}_z = 0.2\,\mathrm{N\,m}$ used in (4.15), the roll and pitch bounds are conservative by a factor of four while the yaw bound is marginally *optimistic*, which is consistent with the executed yaw torque falling well short of the intended one (Section 5.1).

The torque box is a surrogate constraint. Two facts limit what the bounds of (4.15) mean. First, the thrust and torque boxes are independent, whereas (4.23) couples them: a combination is realisable only if every F_i lies within the per-rotor range $[0.32\,\mathrm{N}, 7.84\,\mathrm{N}]$. The roll and pitch bounds remain realisable up to 28 N of collective thrust, but the largest realisable yaw moment falls from 0.18 N m at 20 N to 0.13 N m at the 23.2 N hover thrust with the heavier payload and 0.05 N m at 28 N, and 55 % of the volume of the full box is not realisable by any rotor setting. Second, the torque is never sent: the autopilot receives the collective thrust and the predicted body rate, and the moment actually produced is the output of its rate loop and mixer. The optimiser's torque is a virtual input of the prediction model, and its box is a surrogate that keeps the planned rotational accelerations within the authority the vehicle roughly has, not a model of the actuator limits. In the 48 deployed-system flights of Section 5.8.2 the surrogate was rarely active: at $4\,\mathrm{m\,s^{-1}}$ the planned yaw moment was at its bound in a median 2.4 % of loaded samples — planned values that, at that thrust, no rotor setting realises — the planned roll and pitch moments at theirs in at most 2.3 %, and the rotors themselves saturated in at most 0.12 %. Replacing the boxes by the four linear constraints $F_{\min} \le F_i(T, \boldsymbol{\tau}) \le F_{\max}$ of (4.23), which are polytopic in u and cost nothing structurally in the solver, would make the input set exact; it would still constrain a virtual input, and it has not been implemented.

4.4.5 Solver

Both optimal control problems are transcribed by direct multiple shooting with explicit fourth-order Runge–Kutta integration, a Gauss–Newton Hessian approximation, and partial-condensing HPIPM [9] as the quadratic subproblem solver, generated and solved with acados [8]. Both use *full* sequential quadratic programming with merit-function backtracking rather than the real-time-iteration scheme [6]: RTI was observed to diverge silently in this configuration, and a controller that reports success while producing a diverging input is worse than one that takes an extra millisecond. Generated C code is cached and reused until the model or problem dimensions change.

4.5 The Continuous Payload Interface

4.5.1 One atomic frame

After each solve the estimator publishes

$$\mathscr{P} = \Big\{ m_T,\ \Delta J_{xx}, \Delta J_{yy}, \Delta J_{zz},\ J_{xx}, J_{yy}, J_{zz},\ \gamma,\ h,\ a \Big\}, \tag{4.24}$$

in which the inertia entries are computed from m_T by (3.19), with $\Delta J_{zz} = 0$ for a centred point mass; $\gamma \in [0,1]$ is the no-payload evidence score of Section 4.7.3, h is a health bit and a is the age of the most recent successful solution. The message layout additionally retains four horizontal-offset fields for compatibility with the generated solver; they are not consumed by the evidence score, by the lifecycle logic, or by the controller, which holds $\boldsymbol{c}_{xy} = \mathbf{0}$.

Health is a conjunction: the latest solve must have succeeded, the odometry, rotor-speed and known-input streams must all be fresher than 0.30 s, and the latest estimate must pass the mass-health check (4.10). The consumer applies

one further test of its own, on the receive age of the frame itself (0.35 s), because a frame can be healthy at the moment it is produced and stale by the time it is used.

The asymmetry in how an unhealthy frame is treated is deliberate and is the safety property of the interface: *an unhealthy or stale frame freezes the last accepted model and resets the evidence-score persistence timer*. A degraded estimate can therefore delay a release confirmation, but it can never cause one.

4.5.2 Filtering and slew limiting

A valid frame is not consumed directly. Each channel passes a first-order low-pass filter and a separate rate limiter before it reaches the solver:

$$\alpha = 1 - e^{-\Delta t/\tau}, \qquad \hat{\theta} \leftarrow \hat{\theta} + \alpha\big(\theta^{\mathrm{tgt}} - \hat{\theta}\big), \qquad \hat{\theta} \leftarrow \hat{\theta} + \mathrm{clip}\big(\cdot,\ \pm\dot{\theta}_{\max}\Delta t\big), \tag{4.25}$$

with $\tau = 0.20\,\mathrm{s}$ on the controller side (0.25 s on the estimator's own output stage) and rate limits $0.60\,\mathrm{kg\,s^{-1}}$ for the mass and $0.080\,\mathrm{kg\,m^2\,s^{-1}}$ for the inertia increment.

The reason for a limiter in addition to a filter is that the two protect against different things. The filter attenuates noise; the limiter bounds how fast the *model* underneath a receding-horizon optimisation may change. A parameter step between consecutive solves invalidates the warm start, and an optimiser that must re-converge from scratch at 20 Hz is an optimiser that occasionally does not converge in time. The limiter converts any step — including a genuine one at grasp — into a ramp the solver can track.

4.6 Inner-Loop Scheduling

Model consistency in the outer loop cannot restore inner-loop bandwidth. The autopilot's rate controller is tuned for the bare airframe; a payload that multiplies the roll and pitch inertia by κ of (3.16) lowers its effective bandwidth by the same factor, and for heavy loads the resulting attitude oscillation grows until the torque saturates. At the nominal arm κ is 2.34 for the heavier payload, by (3.17); because it grows with r_z^2, the arm and the payload mass together define a feasibility boundary, characterised in Section 5.9.3. The controller does not modify the autopilot's rate-loop parameters; it scales the body-rate *command* instead, which keeps the adaptation inside the ROS layer and makes it continuous and reversible.

4.6.1 From inertia ratio to a command scale

The deployed response scales the body-rate *command* around the current measured rate,

$$\omega'_{\mathrm{cmd}} = \omega + \varsigma\left(\omega_{\mathrm{cmd}} - \omega\right), \qquad \varsigma \geq 1, \tag{4.26}$$

applied to the roll and pitch channels only, since ΔJ enters only J_{xx} and J_{yy}. Equation (4.26) exaggerates the rate *error* the inner loop sees, which is equivalent to raising its proportional gain, without modifying any autopilot parameter. Because it is a common scale on the roll/pitch pair it preserves the direction of the rate-error vector.

The requested scale is the inertia ratio (3.16), clipped at a cap of 5.0, evaluated from the live ΔJ of the payload frame. Three filters stand between the request and its application, each addressing a distinct failure:

- a **dead band** of 0.08 on the target, so that estimator noise cannot move it, with explicit snapping to 1.0 and to the cap so that hysteresis cannot

strand the empty-airframe steady state permanently above unity;

- a **first-order filter** with $\tau = 0.5\,\mathrm{s}$, chosen far slower than the inner loop (hundreds of hertz) and still fast enough to follow the second-scale progressive load transfer during a grasp;
- **asymmetric rate limits**, $4.0\,\mathrm{s}^{-1}$ rising against $1.5\,\mathrm{s}^{-1}$ recovering, encoding the asymmetry of the consequences: being late to add authority risks saturation, being late to remove it does not.

Because ΔJ is a function of $\hat{m}$, the scale is too: the inner loop receives exactly the authority the mass estimate implies, and a mass estimate that collapses toward the bare airframe during manoeuvre withdraws that authority at the moment it is needed (Section 4.7.6).

4.6.2 Headroom limiting

Scaling a command that is already close to the rate limit accomplishes nothing except saturation, at which point the scaling has silently stopped acting. The implementation therefore solves for the largest scale that keeps the scaled command inside a guard band $\pm L$, with L equal to 95 % of the hard rate limit $\overline{\omega} = 2.0\,\mathrm{rad\,s}^{-1}$.

For one channel, write the rate error $e = \omega_{\mathrm{cmd}} - \omega$. Requiring $|\omega + \varsigma e| \le L$ gives

$$\varsigma \;\le\; \begin{cases} \dfrac{L-\omega}{e}, & e > 0, \\[2ex] \dfrac{-L-\omega}{e}, & e < 0, \end{cases} \tag{4.27}$$

and the applied scale is the smallest such bound over the roll and pitch channels, floored at unity:

$$\varsigma_{\mathrm{eff}} = \max\Big(1,\; \min_{i\in\{x,y\}} \varsigma_i^{\max}\Big). \tag{4.28}$$

The floor at unity guarantees that the scheduler can never make the raw controller command *less* authoritative than it would have been without scheduling — the scheduler is allowed to help and not allowed to hurt. When (4.28) binds, it is logged, so that saturation avoidance is an observable event rather than a silent degradation.

4.6.3 Attach-time inertia bootstrap

One interval resists estimate-driven adaptation: the second or so between a load beginning to take weight and the estimator accumulating enough evidence to say so. During that interval the vehicle is physically loaded and the model is not, and it is the interval in which the predecessor implementation lost eleven of sixteen flights at the heaviest condition.

The deployed bootstrap addresses it with the narrowest intervention that works. When the grasp is enabled, the *inertia* parameter alone is floored at the value implied by the platform's 0.30 kg payload envelope; the mass is untouched, and no payload is fabricated in it. At the nominal arm the floor is $\Delta J = 0.0191\,\mathrm{kg\,m^2}$, an inner-loop ratio of 2.34; the controller uses the larger of the floor and the estimate-derived increment, so a heavier estimate still raises the inertia. The floor is *held until command*: loaded evidence does not end it. It is ended by the controller's release command, or by the acceptance of an uncommanded loss (Section 4.7.5), and is then faded over 0.5 s to the estimate-derived increment. If the estimate becomes stale or unhealthy while the handover is in progress, the fade freezes at the last safe inertia rather than continuing toward zero. Loaded evidence is not allowed to end the floor because the inertia would then follow the loaded estimate, which can collapse in sustained manoeuvre (Section 5.9.1) while the parcel is still attached.

The hold rule is the one place where the release command touches the controller's payload model, and it should be stated exactly. The command ends an

overlay; it does not assert that the payload has gone. After the fade the inertia is whatever the estimate implies, which for a payload still attached is the loaded value, and the release is still accepted only by (4.31). The command is waited for because, after a real release, the estimator can no longer produce the loaded evidence that would end the floor, and the floor would otherwise remain on an empty airframe indefinitely. A load lost without any command issues no command; the loss watchdog of Section 4.7.5 then ends the floor (Section 5.7.3).

Three properties keep this consistent with the rest of the design. It is *inertia-only*, so it cannot influence the mass channel on which every release decision depends. It can only *raise* the inertia above what the estimate implies, never lower it, so the release path remains estimate-only. And its magnitude is the airframe's payload envelope, which Section 3.4.3 has already disclosed as a platform specification rather than task information.

The bootstrap also has a known failure of its own. It is armed when the grasp is enabled, whether or not the gripper actually catches the parcel. A missed grasp therefore leaves an empty airframe flying with the envelope inertia floor — a 2.34-fold inner-loop scale at the nominal arm — until the release command. The loss watchdog cannot end it earlier, because the watchdog arms only on airborne loaded evidence, which a missed grasp never produces.

4.7 The Grasp–Release Lifecycle

4.7.1 What release acceptance does, and why it is gated on evidence

Four things happen around a release, and the design keeps them apart.

1. *The dynamic parameters* — the mass, the inertia increment and the inner-loop scale — follow the estimate frame by frame through the continuous interface of Section 4.5, in every lifecycle state. When the payload leaves, they return to the bare-airframe values because the estimate does, not because any decision switches them.

2. *The release lifecycle state* — carry, release pending, release accepted, or UNRESOLVED — is owned by the controller.

3. *The residual specific force* of Section 4.4.3 is reset when a release is accepted.

4. *Task completion* — the delivery is declared done — coincides with acceptance.

Release acceptance is the transition of item 2; it triggers items 3 and 4 and never sets the mass or the inertia. Acceptance is the only lifecycle decision of the controller, and the rest of this section concerns when it may be taken.

The controller issues the gripper-release command itself, and it would be simple to accept the release at that instant. The command and the physical detachment can, however, differ: a gripper can be slow to open, can jam, or can release only part of a load. A controller that accepts at the command declares the delivery complete and discards the residual it learned against the loaded vehicle while the load may still be on the hook. Gating acceptance on measured evidence — *evidence-confirmed detachment* — separates "the command was issued" from "the payload has left", at the cost of a confirmation delay while the evidence accumulates.

The evidence is a force-balance test that does not pass through the mass estimate, enabled either by the command or by a watchdog armed on airborne load (Section 4.7.5); the watchdog also lets it detect a load lost without a command. An earlier rule that gated acceptance on the estimator's own mass evidence is

evaluated in Chapter 5 (Section 5.2.1).

4.7.2 Two state machines

The lifecycle is split as shown in Figure 4.2. The estimator tracks payload *presence*. The controller owns the *release lifecycle* and accepts a release only on *confirmation*. The separation exists so that no single estimator-side transition can, by itself, declare a delivery complete.

Presence is a hysteretic EMPTY↔LOADED machine. It enters LOADED when the estimated payload mass exceeds 0.09 kg for 2 s; it returns to EMPTY when the payload mass stays below 0.02 kg for 5 s *while the vehicle is quiet* ($\|\boldsymbol{\omega}\| \leq 0.15\,\mathrm{rad\,s^{-1}}$, $\|\boldsymbol{v}_{xy}\| \leq 0.20\,\mathrm{m\,s^{-1}}$). The quiet-flight condition is there because the mass is least reliable in manoeuvre (Section 4.7.6). The thresholds differ by a factor of four-and-a-half on purpose: the estimate is naturally low while it converges after a grasp, and a single threshold would toggle at the start of every lift. Presence is a diagnostic: it is logged with every transition and affects no published field.

4.7.3 No-payload evidence score

The frame carries an evidence score that is a function of the published mass alone. It is logged but not used to accept a release; its form is reused by the force-balance score of Section 4.7.5. Let $\sigma(v;a,b)$ be the smooth indicator that is 1 for $|v| \leq a$, 0 for $|v| \geq b$, and in between

$$\sigma(v;a,b) = 1 - \left(3\chi^2 - 2\chi^3\right), \qquad \chi = \frac{|v| - a}{b - a}, \tag{4.29}$$

which is the standard cubic smooth-step: continuous with continuous first derivative at both ends. Let $\bar{m}_{P,k}$ be the estimator's filtered payload mass and

let $\gamma_k^\star$ be its instantaneous empty-payload score. The published score is filtered once more at the output stage:

$$\begin{aligned} \gamma_k^\star &= \sigma\big(\bar{m}_{P,k};\, 0.015,\, 0.060\big), \\ \gamma_k &= \gamma_{k-1} + \alpha_\gamma\big(\gamma_k^\star - \gamma_{k-1}\big), \qquad \alpha_\gamma = 1 - e^{-\Delta t/0.25\,\mathrm{s}}. \end{aligned} \tag{4.30}$$

Thus γ is a deterministic evidence score, not a posterior probability or a statistical confidence level. The score is 1 at or below 0.015 kg, 0 at or above 0.060 kg, and changes smoothly between them. The instantaneous score exceeds 0.90 below approximately 0.024 kg; the additional output filter determines when the published score crosses that threshold. The inertia increment is deliberately absent: in the mass-only model it is a deterministic function of the same mass estimate and therefore cannot provide independent release evidence.

4.7.4 Confirmation, UNRESOLVED, and braking to hover

The controller accepts a release through exactly one route, a persistence test on the force-balance score γ_{acc} of (4.36):

$$\gamma_{\mathrm{acc}} \geq 0.90 \quad \text{held for} \quad 1.0\,\mathrm{s} \quad \text{on fresh evidence}, \tag{4.31}$$

where fresh means that the rotor-speed and inertial signals are younger than 0.2 s. The persistence must be contiguous in valid evidence time: stale evidence breaks the count rather than pausing it. There is no separate estimator declaration, and the release command does not enter the score; it only decides when the count of (4.31) may start (Section 4.7.5).

If no confirmation arrives within 12 s of the release command, the controller enters UNRESOLVED. This state is what makes confirmation on the estimate deployable, and its semantics deserve emphasis: **a timeout is not a release**. Nothing is accepted, the residual is not reset, and the model continues to follow the es-

timate. What changes is the vehicle's exposure: it stops flying an aggressive trajectory it may no longer have the authority to fly, and it gives the estimator a quiet operating point in which the mass is best observed.

The braking manoeuvre is a finite-time reference rather than an instantaneous switch to hover, because at $4\,\mathrm{m\,s^{-1}}$ the latter would command a step of several metres per second. Let $\boldsymbol{p}_0, \boldsymbol{v}_0$ be the state at the declaration and $T_b = 3\,\mathrm{s}$. The stopping point is chosen as the displacement of a constant deceleration from $\boldsymbol{v}_0$ to rest,

$$\boldsymbol{p}_1 = \boldsymbol{p}_0 + \tfrac{1}{2} T_b \, \boldsymbol{v}_0, \tag{4.32}$$

and the reference between the two is the unique quintic satisfying $\boldsymbol{p}(0) = \boldsymbol{p}_0$, $\dot{\boldsymbol{p}}(0) = \boldsymbol{v}_0$, $\ddot{\boldsymbol{p}}(0) = \mathbf{0}$, $\boldsymbol{p}(T_b) = \boldsymbol{p}_1$, $\dot{\boldsymbol{p}}(T_b) = \mathbf{0}$, $\ddot{\boldsymbol{p}}(T_b) = \mathbf{0}$:

$$\begin{aligned} \boldsymbol{p}(t) = \boldsymbol{p}_0 + \boldsymbol{v}_0 t + a_3 t^3 + a_4 t^4 + a_5 t^5, \qquad & a_3 = \big(10\,\Delta\boldsymbol{p} - 6 T_b \boldsymbol{v}_0\big)/T_b^3, \\ & a_4 = \big(-15\,\Delta\boldsymbol{p} + 8 T_b \boldsymbol{v}_0\big)/T_b^4, \\ & a_5 = \big(6\,\Delta\boldsymbol{p} - 3 T_b \boldsymbol{v}_0\big)/T_b^5, \end{aligned} \tag{4.33}$$

with $\Delta\boldsymbol{p} = \boldsymbol{p}_1 - \boldsymbol{p}_0$. Zero acceleration at both ends means the manoeuvre begins and ends without a jump in commanded thrust, which is what keeps the transition from itself perturbing the estimator whose evidence is being awaited. Attitude is blended over the same interval by the smooth-step of (4.29).

4.7.5 Force-balance release evidence and the uncommanded-loss watchdog

This is the release rule of the controller, called arm D in Chapter 5. Release evidence should not pass through the mass estimate, the one quantity of the system that degrades in sustained manoeuvre (Section 4.7.6); it should not pass through the attitude, the velocity history or the rotational model either. The vertical force balance in the body frame provides it. The specific force measured

by the inertial unit along the thrust axis, f_z, contains no gravity term, and with the rotor-drag term neglected it satisfies $f_z \approx T/m$. A mass can therefore be formed from two measured signals alone,

$$m_{\mathrm{acc},k} = \frac{\bar{T}_k}{\max\left(\bar{f}_{z,k},\, 1\,\mathrm{m\,s^{-2}}\right)}, \qquad T_k = \textstyle\sum_i k_f\, \omega_{i,k}^2, \tag{4.34}$$

where $\bar{(\cdot)}$ denotes a first-order low-pass with a time constant of 0.5 s, applied to the thrust reconstructed from rotor speeds and to f_z alike. The lower limit on $\bar{f}_z$ guards against division in free fall. No estimator state enters (4.34); in particular, it is unaffected by any manoeuvre-induced bias of $\hat{m}$, and it remains available when the estimator's frames are unhealthy. Its weakness is the one the mass channel also has: a relative error in k_f scales m_{acc} one-for-one.

What (4.34) **assumes.** The relation $f_z \approx T/m$ holds for the specific force at the composite centre of mass, with all external forces other than thrust and gravity neglected. An accelerometer at the body-frame position $\boldsymbol{r}_I$ measures instead

$$\boldsymbol{f}_I = \boldsymbol{f}_C + \dot{\boldsymbol{\omega}} \times (\boldsymbol{r}_I - \boldsymbol{c}) + \boldsymbol{\omega} \times \big(\boldsymbol{\omega} \times (\boldsymbol{r}_I - \boldsymbol{c})\big) + \boldsymbol{b}_a + \boldsymbol{\nu}_a, \tag{4.35}$$

with $\boldsymbol{f}_C$ the specific force of the composite centre, $\boldsymbol{b}_a$ a bias and $\boldsymbol{\nu}_a$ noise and vibration; $\boldsymbol{f}_C$ itself contains, besides $T\boldsymbol{e}_3/m$, the rotor and airframe drag and any induced-inflow change of the thrust. The thrust-axis component of the lever-arm terms is $-(\omega_x^2 + \omega_y^2)(r_{I,z} - c)$ plus products of $\dot{\boldsymbol{\omega}}$ with the horizontal offset of the sensor. In simulation the inertial unit sits at the body origin, $\boldsymbol{r}_I = \boldsymbol{0}$, so only the composite offset c of Section 3.2.2 remains, at about 0.002 % of g, and the plant generates thrust by exactly the static law that (4.34) inverts. On hardware none of this is given: a bias $b_{a,z}$ shifts m_{acc} by the fraction $-b_{a,z}/g$, so $0.05\,\mathrm{m\,s^{-2}}$ is 0.5 %, or 0.01 kg, against a lower score threshold of 0.015 kg in (4.36); a sensor 0.05 m off the thrust axis adds a term proportional to the angular acceleration that the 0.5 s low-pass attenuates but does not remove; rotor-speed latency relative to the inertial unit turns every thrust tran-

sient into a transient of m_{acc}; and motor dynamics, induced inflow and vertical drag break the static thrust law the detector shares with the estimator. Only the thrust-coefficient error was tested (Section 5.7.4). Equation (4.34) is therefore a mass measurement for this simulated vehicle, not a generally valid one; on hardware it would need the sensor position, a bias estimate in empty hover and a time alignment of the two streams before its thresholds mean what they mean here.

The evidence score has the form of (4.30) with the same pair of thresholds,

$$\gamma^{\star}_{\text{acc},k} = \sigma\big((m_{\text{acc},k} - m_B)^{+}; 0.015, 0.060\big), \tag{4.36}$$

filtered at the output with a 0.25 s time constant. The score starts at zero, because no evidence is not evidence of an empty vehicle, and it is held rather than advanced while either signal is stale (older than 0.2 s); a stale frame also breaks the persistence count of (4.31), which is otherwise applied to γ_{acc} unchanged: $\gamma_{\text{acc}} \geq 0.90$ held for 1.0 s. Under this rule the controller's lifecycle reads the inertial unit and the rotor speeds directly and does not depend on the published frame, by design. The inertial unit is the one sensor the deployed system uses beyond odometry and rotor speeds, and only the lifecycle reads it; the adaptive path of Figure 4.1 does not.

What may consume the score is decided by *dual enabling*. The score is computed throughout the flight, but a confirmation counts only when one of two conditions holds:

1. a release command is pending — the command opens confirmation at once; or
2. the *uncommanded-loss watchdog* is armed. It arms after the lift has started, once fresh evidence of load, $\gamma_{\text{acc}} < 0.90 - 0.15 = 0.75$, has persisted for 2.0 s. Once armed it is not disarmed by a rising score — that rise is the event it exists to detect — and it is cleared only at the next

pickup.

During grasp and ground support neither condition holds, so a parcel that is attached but still resting on the ground, for which the force balance correctly reads an empty vehicle, cannot produce an acceptance.

An acceptance with a command pending is a normal release, with the actions of Section 4.7.1. An acceptance with no command pending and the watchdog armed is declared an *uncommanded loss*. In addition to resetting the residual specific force, the controller then sets the model's inertia increment to zero, starts the 0.5 s fade of the attach-time inertia floor (Section 4.6.3), and suppresses any later planned release, since there is nothing left to release. The inner-loop scale, which follows the inertia increment, returns to unity at its recovery rate limit; afterwards the increment again follows the estimate, which by then reads the empty airframe. The controller issues no gripper command, because the gripper's state is unknown and it may be the gripper that failed.

4.7.6 Limits of the lifecycle

The release evidence inherits the thrust map. A relative error in k_f scales m_{acc} one-for-one, so a map that reads high makes the vehicle look loaded after a real release, and one that reads low can make a light payload look absent, so that the watchdog never arms (Section 5.7.4). In that case the score already reads an empty vehicle when the command opens the rule, and under a delayed detachment the release is accepted before the payload has left; the deployed-system validation observed exactly this with a 10 % under-reading map and the lighter payload (Section 5.3). The force balance also assumes $f_z \approx T/m$, neglecting the vertical rotor drag. And the attach-time floor is ended only by a command or a detected loss, so a missed grasp leaves it on the empty airframe until the command (Section 4.6.3).

The mass estimate is deliberately not asked the release question. In sustained manoeuvre a low bias can carry $\hat{m}_P$ below the 0.024 kg threshold of (4.30) while the payload is attached, and no rule admitting only that estimate can distinguish such a collapse from a real release (Sections 5.9.1 and 5.6.4).

4.7.7 Threshold provenance

Table 4.3 lists every lifecycle threshold and where its value came from. None was selected against the evaluation data; all are engineering choices whose closed-loop sensitivity has not been swept, and they are reported as such.

Table 4.3: Lifecycle thresholds and their provenance.

Threshold	Value	Provenance
presence entry	$m_P > 0.09$ kg, 2 s	below the smallest payload tested; persistence suppresses the post-grasp convergence transient
presence exit	$m_P < 0.02$ kg, 5 s, quiet flight	slow backstop; hysteresis against the entry threshold
mass evidence score	(4.30); logged, not used to release	engineering choice
confirmation	$\gamma_{\text{acc}} \geq 0.90$ held 1.0 s, (4.31)	engineering choice
force-balance score	(4.34)–(4.36); low-pass 0.5 s/0.25 s; fresh within 0.2 s	thresholds shared with (4.30); time constants an engineering choice, checked by offline replay of the earlier cohort and frozen before the campaign of Section 5.7
loss-watchdog arming	$\gamma_{\text{acc}} < 0.75$ for 2.0 s, after lift start	engineering choice; frozen before that campaign
UNRESOLVED timeout / brake	12 s / 3 s	engineering choice

4.8 Reference Trajectory

The evaluation trajectory is a three-dimensional Gerono lemniscate,

$$x_r = r\sin a, \qquad y_r = r\sin a\cos a = \tfrac{r}{2}\sin 2a, \qquad z_r = z_h + d_z\sin a, \qquad a = w t_c, \tag{4.37}$$

with yaw commanded along the velocity vector. The crossing is not a cusp: $\boldsymbol{v} = rw(\cos a, \cos 2a)$ has magnitude $\sqrt{2}rw$ there while the planar acceleration vanishes. The curve therefore excites coupled translation and yaw without a velocity reversal, and the vertical term keeps the workspace compact.

After the initial hover, position amplitude and heading are introduced with the quintic smooth step $\alpha(s) = 10s^3 - 15s^4 + 6s^5$. The ramp duration satisfies $t_{\text{ramp}} \geq 2.65/w$, limiting the extra velocity due to amplitude growth, and heading is blended from the hover direction to the trajectory tangent. This yaw blend was retained because, without it, six development flights at $4\,\mathrm{m\,s^{-1}}$ spent 19–25 % of their first 5 s at the yaw-moment constraint.

4.9 Online Execution Sequence

At every control instant the implementation performs the following sequence.

1. Receive and frame-convert the latest odometry.
2. Reconstruct the collective thrust from rotor speeds, (3.20), averaging the 250 Hz samples over the estimation interval.
3. Append the measurement and the reconstructed input to the 10 Hz window; re-seed by (4.14) if the window must be rebuilt.
4. Solve (4.7) for m_T and apply the mass-health check (4.10).

5. Update the presence machine and the no-payload evidence score (4.30).
6. Assemble and publish the atomic frame (4.24) with its health bit and solution age.
7. In the controller: validate freshness, low-pass filter and slew-limit the frame (4.25), and write m_T and ΔJ into the model parameters and the hover baseline (4.16).
8. Update the force-balance score (4.34)–(4.36) from the latest rotor speeds and inertial specific force, and the loss watchdog; evaluate confirmation (4.31) on it, if a release command is pending or the watchdog is armed, and the UNRESOLVED timeout; switch the reference to the braking quintic (4.33) if the timeout expires.
9. Shift the previous solution forward as a warm start and solve (4.15). Between solves, publish at 50 Hz the thrust and the body rate interpolated along the predicted trajectory (Section 3.5.2); invert thrust to throttle by (4.21) and apply the body-rate scaling (4.26)–(4.28).
10. Log estimates, residuals, constraint activity, solver statistics and every lifecycle transition.

Estimator and controller failures are handled separately and asymmetrically. An invalid estimator solve, or an estimate outside the health band, retains the last accepted parameter set and marks the frame unhealthy, which — by Section 4.5.1 — freezes the controller's model and resets its confirmation timer. An invalid controller solve falls back to the configured safe action rather than publishing an unconstrained command.

4.10 Chapter Summary

This chapter addressed the third and fourth research objectives: turning the single-parameter model of Chapter 3 into an estimator and a controller that run together, and defining the release lifecycle. The two models are collected in one section: the controller predicts with the full vehicle model and the estimate-derived inertia, while the estimator keeps only the translational dynamics, with the measured attitude as a known input, so that no rotational model error can reach the mass. The estimator is a constrained moving-horizon problem whose only payload decision variable is the composite mass, initialised from a window force balance, driven by the measured thrust, and checked against a physical health band before its estimate is released. One atomic, health-gated frame carries the mass and the inertia increment derived from it to the controller, which updates its prediction model, its input baseline and its inner-loop command scaling continuously from that frame. Release acceptance is kept separate from that adaptation: it changes the lifecycle state, resets the residual force and declares completion, and it is gated on a force balance of the measured thrust against the inertial specific force, which does not pass through the mass estimate; it is opened by the release command or by a watchdog armed on airborne load, so a load lost without a command is also detected. Two properties of the design are known before any experiment: the release evidence inherits any error of the thrust map, and the attach-time inertia floor protects the inner loop at the price of being released only by a command or a detected loss, not by evidence of load. Chapter 5 measures how the complete system behaves.

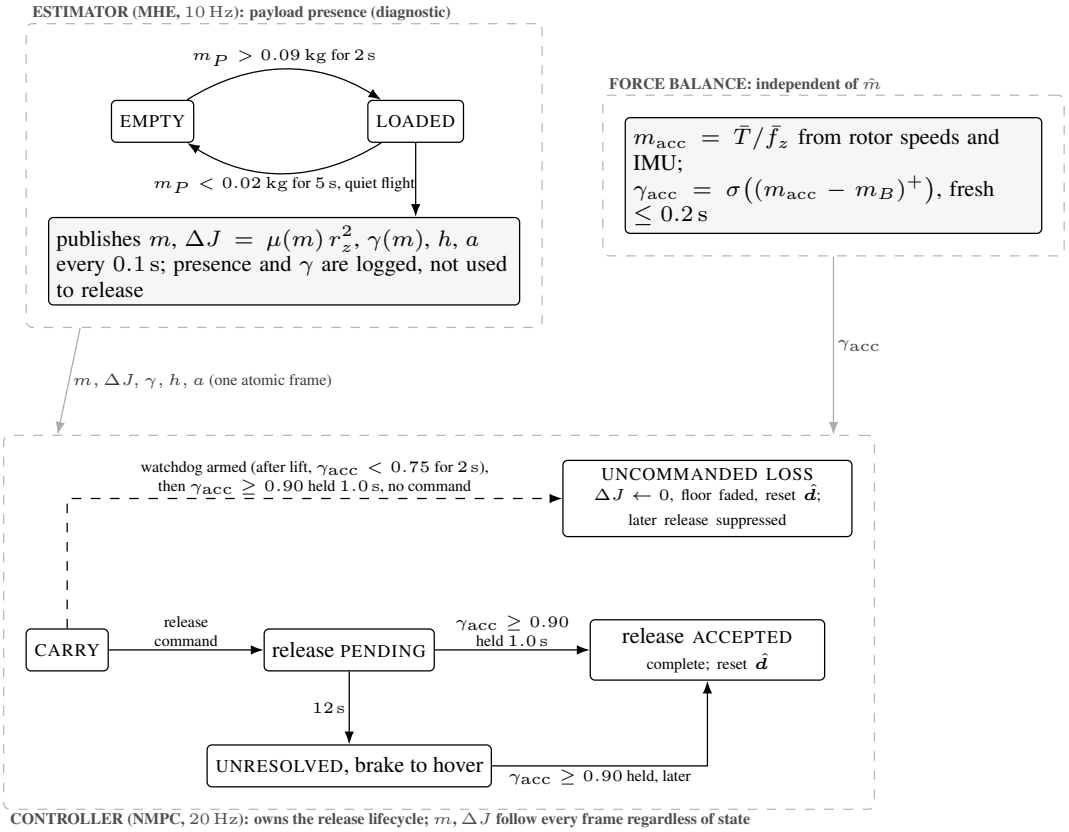


Figure 4.2: The deployed payload lifecycle. The estimator tracks payload presence, driven by sustained payload-mass evidence with hysteresis, and publishes the evidence score γ; neither is used to accept a release. The controller owns the release lifecycle: the release command moves it from carry to release pending and starts the 12 s timeout, and a release is accepted on confirmation by the force-balance score, $\gamma_{\mathrm{acc}} \geq 0.90$ held for 1.0 s (Section 4.7.5). UNRESOLVED is a hold, not an acceptance: the vehicle brakes to hover and keeps waiting for evidence. The dashed path, acceptance without a command, is open only once the loss watchdog has armed on airborne load, and is classified as an uncommanded loss. In every state the mass and inertia increment follow the estimate frame by frame; no transition sets them, except that an uncommanded loss zeroes the inertia increment once.

Chapter 5

Implementation and Experimental Evaluation

This chapter addresses the fifth research objective: evaluating the complete interface in software-in-the-loop simulation and characterising where it fails. The evidence comes from sources of different standing, which are kept apart throughout (Section 5.5.1). The primary evidence for the deployed system — the reduced estimator with the force-balance release rule — is a campaign of 180 flights of that system under nominal release, delayed detachment, uncommanded loss, a jammed gripper and thrust-map error (Section 5.3). The remaining sources concern the *historical* configuration in which the design was developed, with the full-state estimator: a frozen, block-randomised cohort of 120 flights on the estimator's mass evidence (Sections 5.5–5.6); a second frozen campaign of 312 flights that introduced the force-balance rule and compared fixed and $\mathscr{L}_1$ baselines (Section 5.7); a smaller frozen batch that isolates the residual specific force; and the paired comparison of 96 flights on which the reduced estimator was chosen (Section 5.8.2). Development flights in earlier configurations are used only to show mechanisms. The frozen campaign of an earlier revision used a different estimator and eccentric grasps; none of its numbers is reused here.

5.1 Platform and Implementation

The implementation is organised as ROS 2 packages around PX4 software-in-the-loop [32] and Gazebo [33]. The controller node owns the reference generator, the predictive optimal control problem, the mission sequence and the release-confirmation logic; the estimator node owns the sliding measurement window, the physical-input reconstruction, the mass optimisation, the presence machine and the published payload frame. The two communicate through the single atomic frame of Section 4.5.

Table 5.1: Main implementation components. Rows marked * exist only in the campaign code base; the others form the deployed pipeline.

Component	Role	Principal signals
Gazebo (Harmonic)	Rigid-body physics; detachable-joint gripper	pose, rotor speed, attach/detach record (logging only)
PX4 SITL	State estimation, rate loop, control allocation	odometry, actuator commands
`mhe_node`	moving-horizon mass estimation (full-state in the campaigns, reduced deployed); presence machine	odometry, rotor speed → payload frame (4.24)
`acados_nmpc_node`	Predictive control, mission sequence, confirmation	payload frame, state → thrust and body-rate setpoint
`acc_release_detector`	Force-balance release evidence (4.34)–(4.36), inside the controller	rotor speed, inertial specific force → γ_{acc}
`payload_estimate`	Frame schema, evidence score (4.30)	shared by the online nodes and the offline replay
`proximity_gripper_node`	Grasp contact and release actuation	proximity, grasp enable, release command
`lifecycle_trace`	Machine-readable event trace for scoring	every state transition, with timestamps
`l1_adaptive`	Residual specific-force estimator (Section 4.4.3); in the campaign code base also the comparison adaptive controller	odometry, rotor speed → $\hat{\boldsymbol{d}}$
`px4_pid_benchmark`*	Bare-autopilot cascade baseline	position setpoints only

Every reported payload change is produced by the detachable-joint gripper,

which adds or removes a genuine second rigid body and therefore changes mass and inertia together. A retired run-time inertia plugin did not alter the physics engine and contributes no result to this dissertation.

Across twenty recorded batches, reconstructed roll and pitch torques track the intended values (median correlations 0.93 and 0.70; slopes 1.08 and 1.10), whereas executed yaw torque is only 0.32 of the intended value. This three-fold yaw discrepancy is a concrete reason to drive the estimator with the measured wrench (3.20), not the command.

5.1.1 Platform parameters and task

The reusable-vehicle constants are

$$m_B = 2.0643\,\mathrm{kg}, \qquad J_B = \mathrm{diag}(0.0142, 0.0142, 0.0210)\ \ \mathrm{kg\,m^2}, \qquad k_d = 0.05, \tag{5.1}$$

with rotor arm $0.174\,\mathrm{m}$, $k_f = 8.55 \times 10^{-6}\,\mathrm{N\,s^2}$ and $k_m = 0.016\,\mathrm{m}$. The collective-thrust box is the actuator envelope (4.22), $[1.27\,\mathrm{N}, 31.35\,\mathrm{N}]$, and the torque bounds are $0.5\,\mathrm{N\,m}$ in roll and pitch and $0.2\,\mathrm{N\,m}$ in yaw. None of these grows with the estimate.

The value m_B appears in the model, in the derived quantity $m_P = (m_T - m_B)^+$ and in offline scoring. The payload is a $0.15\,\mathrm{m}$ cube of $0.15\,\mathrm{kg}$ or $0.30\,\mathrm{kg}$, placed directly below the grasp point so that the grasp is centred. The gripper attaches when the parcel is within $0.03\,\mathrm{m}$ horizontally of the attachment point and its vertical distance lies in $[0.15\,\mathrm{m}, 0.45\,\mathrm{m}]$. The vehicle descends to $0.35\,\mathrm{m}$ above ground to grasp, which realises a vertical arm of about $0.27\,\mathrm{m}$; both optimisers use the nominal prior $r_z = -0.27\,\mathrm{m}$, and the realised arm varies between about $0.25\,\mathrm{m}$ and $0.30\,\mathrm{m}$ from flight to flight (Section 3.4.3). None of the payload values is supplied to the estimator or the controller.

The task sequence is: approach under position control; hand-off to the predictive controller and warm-up hover; alignment over the parcel; controlled descent; grasp enable; lift from 0.35 m to 6.0 m in 8.4 s; figure-eight carry (4.37) with $r = 10\,\mathrm{m}$, $z_h = 6.0\,\mathrm{m}$ and $d_z = 0.8\,\mathrm{m}$; release; recovery. The release command is issued after 45 s of loaded figure-eight, at the next tip of the figure-eight ($a = 3\pi/2$), so that every release occurs at the same geometric phase. The two speeds are set by the phase rate: $w = 0.283\,\mathrm{rad\,s^{-1}}$ for a peak of about $4\,\mathrm{m\,s^{-1}}$, and $w = 0.1415\,\mathrm{rad\,s^{-1}}$ with a 9.36 s speed ramp for about $2\,\mathrm{m\,s^{-1}}$. The scope of the controller is stated explicitly — it governs the grasp hover and the post-event adaptation phase, and the approach controller is not part of any claim. Batches run headless.

5.1.2 Real-time performance

On one desktop core shared with the simulator and the autopilot, the 13-state controller ($N = 20$, 20 Hz) solves in 1.3 ms at the median and 2.2 ms at the 99th percentile over $n = 1009$ solves: 2.6 % and 4.4 % of its 50 ms period. The controller's problem dimensions are unchanged by the estimator configuration.

The full-state estimator ($N = 20$, 10 Hz) logs its solve time as a mean and a maximum over consecutive 2 s windows, not per solve. Over the 120 cohort flights, with the first two windows of each flight discarded ($n = 5800$ windows), the window mean had a median of 2.8 ms, a 90th percentile of 5.4 ms and a 99th percentile of 14.3 ms; the largest single solve recorded in any window was 77.3 ms. Against the 100 ms period the estimator never overran in the cohort, but the worst case used about three quarters of it, and because only window aggregates were recorded, the per-solve percentiles are not available. The reduced estimator logs every solve; its timing is reported with the deployed-system validation (Section 5.3) and the comparison of Section 5.8.2.

The deadline is met in simulation. We nevertheless make *no* claim about head-

room on an onboard computer; that requires measurement on the target hardware.

5.2 Evaluated Configuration

Throughout this dissertation, *evaluated configuration* means the full-state (14-state) mass-only MHE of Section 5.2.1 at 10 Hz ($N = 20$), driven by odometry and the rotor-speed wrench, with stage weights that were uniform in effect (Section 4.3.6), the $0.95\,m_B$ lower bound, lagged inertia (5.6), $r_z = -0.27\,\mathrm{m}$, window-balance seeding and, in the cohort, the earlier product form of the evidence score (4.30) (see *Historical scoring* below). It feeds the 13-state NMPC at 20 Hz. The cohort additionally sets five non-default choices explicitly:

1. the residual specific force $\hat{\boldsymbol{d}}$ of Section 4.4.3 ($a = 10$, $\omega_c = 0.5\,\mathrm{rad\,s^{-1}}$);
2. the attach-time inertia floor held until the release command (Section 4.6.3);
3. alignment relative to the parcel before grasp, to 0.012 m with at most 4 s of waiting (Section 5.9.2);
4. a smoothed initial yaw reference at the start of the figure-eight;
5. the shortened arm described above.

Offset compensation is off and every grasp is centred. Inner-loop scaling and lifecycle thresholds remain at their documented defaults; switches belonging to the unused moment/offset formulation are inactive. Appendix A gives the complete resolved configuration. All 120 flight records were checked and contain the 14-state selection, lagged inertia, residual, inertia-floor hold, relative alignment, yaw smoothing and stated r_z prior.

Deployed system. The deployed system differs from the evaluated configuration in two places: its release rule is the force-balance rule, arm D of Section 5.7, and its estimator is the reduced velocity–mass estimator of Section 4.2.1. Everything else — controller, residual specific force, inertia-floor hold, alignment, yaw smoothing, arm prior, inner-loop scaling and lifecycle thresholds — is the configuration above. On 5 November 2026 the deployed system was extracted from the campaign code base into a separate mainline code base in which these settings are fixed rather than switchable (Section 5.9.5).

Deployed estimator. After the cohort and the release campaign, the experiment entry points were switched to the reduced velocity–mass estimator of Section 4.2.1. Every flight of Sections 5.6–5.7, of the ablations other than Section 5.8.2, and of the failure-mode analysis was flown with the full-state estimator from the campaign code base; those results describe the historical configuration, not the deployed estimator. The exceptions are the R arm of Section 5.8.2 and the 180 flights of the deployed-system validation (Section 5.3); the latter were flown with the deployed system itself, from the mainline code base. The evidence for the replacement is Section 5.8.2. Results that depend on the estimator's model — the mass bias and its manoeuvre-induced collapse in particular — should not be transferred to the reduced estimator without that evidence.

Historical scoring. The cohort of Sections 5.5–5.6 was flown with an earlier product score, which multiplied the mass score of (4.30) by an inertia score derived from the same mass estimate. Its results are therefore reported as results of that historical configuration. They are not a closed-loop validation of the mass-only score (4.30), and no closed-loop rerun with the mass-only score was made: the deployed system does not release on the mass score at all, but on the force balance of arm D (Section 5.7). Offline replay of the cohort with the mass-only score found no change in acceptance time in 68 of 71 score-

confirmed flights and a 0.1 s advance in the other three. This indicates that the two scores would have behaved alike in these flights, but it is a replay of recorded data, not a closed-loop result.

5.2.1 The full-state estimator and the mass-evidence lifecycle

Neither belongs to the system of Chapters 3 and 4; they are stated here because every flight of Sections 5.6–5.7 and of the failure-mode analysis used the first, and the cohort used the second.

Full-state estimator F. F integrates the complete vehicle model and augments the composite mass into its state,

$$x_{\text{aug}} = \left[x;\, m_T\right] \in \mathbb{R}^{14}, \qquad \dot{m_T} = 0, \tag{5.2}$$

with process noise on the thirteen physical states only. Its known parameters are the reconstructed wrench of (3.20) and a geometry slot,

$$p_{\text{MHE}} = \left[T_{\text{phys}};\, \boldsymbol{\tau}_{\text{phys}};\, \Delta J^{\text{est}};\, c_x^{\text{est}};\, c_y^{\text{est}}\right] \in \mathbb{R}^7, \tag{5.3}$$

and its stage residual is the full measurement residual

$$\boldsymbol{e}_i \;=\; \boldsymbol{y}_i - x_i \;=\; \left[\boldsymbol{e}_i^{p};\;\; \boldsymbol{e}_i^{v};\;\; \boldsymbol{e}_i^{q};\;\; \boldsymbol{e}_i^{\omega}\right] \in \mathbb{R}^{13}, \tag{5.4}$$

with a component-wise quaternion difference in $\boldsymbol{e}^q$. F keeps the mass off the rotational channel by removing only the direct sensitivity,

$$\left.\frac{\partial \dot{\boldsymbol{\omega}}}{\partial m_T}\right|_{\text{estimator}} = 0, \tag{5.5}$$

and inserts the inertia increment as a known parameter computed from the previous solution $\hat{m}^-$ and held constant within the window,

$$\Delta J^{\text{lag}} = \frac{m_B \left(\hat{m}^- - m_B\right)^+}{\hat{m}^-} r_z^2, \qquad \left.\frac{\partial \Delta J^{\text{lag}}}{\partial m_T}\right|_{\text{window}} = 0, \tag{5.6}$$

refreshed at the 10 Hz window rate, with the positive part smoothed over 0.02 kg; the two offsets are zero. Both indirect couplings of Section 3.4.2 therefore remain in F. It is solved as (4.7) with C selecting the thirteen vehicle states and process noise on all of them, at the same horizon and rate as the deployed estimator, with

$$W_y = \operatorname{diag}\Big(\sigma_p^{-2} I_3,\ \sigma_v^{-2} I_3,\ \sigma_q^{-2} I_4,\ \sigma_\omega^{-2} I_3\Big), \tag{5.7}$$

$\sigma_p = 0.02\,\text{m},\ \sigma_v = 0.05\,\text{m s}^{-1},\ \sigma_q = 0.01,\ \sigma_\omega = 0.02\,\text{rad s}^{-1}$,

$$W_w = \operatorname{diag}\big(10^4 I_6,\ 10^5 I_4,\ 10^3 I_3\big), \tag{5.8}$$

$$W_0 = \operatorname{diag}\big(10^3 I_6,\ 10^4 I_4,\ 10^2 I_3,\ \sigma_{m,0}^{-2}\big), \tag{5.9}$$

whose velocity and mass entries are those of (4.9), so that the comparison of Section 5.8.2 isolates the model structure rather than a retuning. Its mass was held in a hard box $[0.95\,m_B,\ 5.0\,\text{kg}]$ with no health check, and after five consecutive solver failures it adopted the window-balance seed as its estimate.

Mass-evidence lifecycle. The cohort accepted a release on the estimator's mass score γ of (4.30), in the product form noted under *Historical scoring* in Section 5.2, instead of γ_{acc}: the persistence test of (4.31), $\gamma \geq 0.90$ held for 1.0 s, on healthy, fresh frames. It had two arming policies: *always-on* counted the persistence throughout loaded flight, *command-armed* only after the release command. There was no loss watchdog, the attach-time inertia floor was ended only by the release command, and an acceptance with no command outstanding did not reset the residual $\hat{d}$. The release arms of Section 5.4.1 are built on this

lifecycle.

5.3 Deployed-System Validation

This section is the primary evidence for the system of Chapters 3 and 4 as deployed: the reduced velocity–mass estimator R with the force-balance release rule (arm D), and the controller, residual specific force, inertia floor, alignment and lifecycle settings of Section 5.2, run from the mainline code base (Section 5.9.5). A campaign of 180 flights repeats with that system the release, fault and thrust-map experiments that the historical release campaign of Section 5.7 flew with the full-state estimator, and adds the combination that campaign did not fly: an under-reading thrust map with delayed detachment. It is a separate batch, from a different code base and on different days. It is not paired with, and not pooled with, any other result of this chapter; where its results are set beside those of the historical campaign, the comparison is descriptive.

5.3.1 Design and execution

The campaign uses four conditions: c1, 0.15 kg at $2\,\mathrm{m\,s^{-1}}$; c2, 0.15 kg at $4\,\mathrm{m\,s^{-1}}$; c3, 0.30 kg at $4\,\mathrm{m\,s^{-1}}$; and c4, 0.30 kg at $2\,\mathrm{m\,s^{-1}}$. Trajectory, grasp and release phase are those of Section 5.1.1. It has four segments (Table 5.2). Within each round every cell of a segment is flown once, in an order drawn from the segment's seed, and the whole schedule was written before the segment's first flight.

The thrust-map gain has the meaning it had in the historical campaign (Section 5.7.1): it scales the thrust reconstructed from rotor speeds in the estimator's thrust input and in the force-balance detector (4.34). The thrust used by the residual specific force, the thrust-to-throttle map, the actuator envelope and

Table 5.2: Design of the deployed-system validation. d0: detachment at the command; d4: detachment held 4 s after the command. Gain: thrust-map gain on the estimation side (see text).

Segment	Flights	Factors	Seed
Release	72	d0 / d4 × c1–c4 × 9 rounds	20261106001
Faults	48	uncommanded loss / jammed gripper × c1–c4 × 6 rounds	20261106002
Thrust map	48	gain 0.90 / 1.10 × c1–c4 × 6 rounds; d0	20261106003
Thrust map, delayed	12	gain 0.90 × c1, c2 (0.15 kg) × 6 rounds; d4	20261106004
Total	180		

the plant's rotor coefficient keep the nominal k_f, so the error reaches the control input only through the mass estimate. An uncommanded loss detaches the payload at the release phase with no command and no notification; a jammed gripper receives the normal command and never releases.

The four segments were flown back to back, from 20:15 on 6 November to 06:33 on 7 November 2026. The flight code was frozen at tag `deployed-r-armd-180-v1` (commit `59236f7`) of the mainline code base. A fingerprint of the flight-stack sources and configuration, the launch and stop scripts and the batch driver was checked before every flight and was `186022e6c95cfa38` throughout. During the release segment a commit that changed only the sources of this dissertation (`9881e78`) moved the repository head, so the three later segments record that commit instead of the tag; the unchanged fingerprint shows that the flight code was the same. The toolchain and solver identities are listed in Appendix A.1. Eligibility and reruns followed rules written into the batch driver before the first flight: only an infrastructure failure — stack not started, logs missing, configuration not matching the schedule, physical attachment failed, or no trigger before the timeout — would have been rerun, once, with the original flight kept in the record. Divergence, solver failure, early or missed confirmation and UNRESOLVED are outcomes. All 180 flights attached and were eligible, and none was rerun. A six-flight smoke batch, one flight per path, preceded the campaign to check the driver and the logs; it is not part of the results.

Repeated flights of a cell receive no seeded random input. They differ through the simulator's unseeded sensor noise and through the timing jitter of the simulation step and of the multi-process stack (Section 5.5.4). Intervals therefore describe the repeat-to-repeat variation of this simulation stack on this machine, not a population of real flights. No significance test is made in this section, so no correction for multiple comparisons arises.

5.3.2 Acceptance criteria

The campaign had no pre-registration of its own. Its four acceptance criteria were inherited verbatim from the pre-specification of 5 November 2026 for the comparison of F and R (Section 5.8.2), namely items 1, 2, 9 and 10, the items that concern R alone, with the parts that require a comparison with F removed. They were fixed during the run: they were written to the campaign's analysis directory after 11 of the 180 flights, when only the terminal-state lines of those 11 flights in the campaign log had been seen and no summary had been opened, and before the three later segments started. They were not changed afterwards. Item 8 of the earlier specification (no runs of non-zero solver status and no re-initialisation under a $-10\,\%$ thrust map) was not adopted, and the corresponding observations are reported descriptively below. Every other result in this section is descriptive.

All four criteria were met (Table 5.3). Unlike the F/R comparison, in which no flight entered UNRESOLVED, this campaign exercised A4 in 48 flights: the 24 jammed releases and the 24 flights with an over-reading thrust map.

5.3.3 Outcomes

Nominal release. Every one of the 72 releases was confirmed ($[0.950, 1]$), none before the payload left, none was missed and none ended in UNRESOLVED;

Table 5.3: Acceptance criteria of the deployed-system validation, inherited from items 1, 2, 9 and 10 of the pre-specified F/R comparison and fixed after 11 **of** 180 **flights. Events: flights violating the criterion. CP: upper limit of the two-sided** 95 % **Clopper–Pearson interval for the event rate.**

	Criterion	Scope	Events	CP	Met
A1	No divergence (more than 100 controller solve failures) in any phase	release	0/72	0.050	yes
A2	No estimator re-initialisation after five consecutive solve failures	release	0/72	0.050	yes
A3	Controller model mass constant throughout every interval in which the estimate was unhealthy or stale	all	0/180	0.020	yes
A4	After UNRESOLVED: speed below $0.5\,\mathrm{m\,s^{-1}}$ from 5 s later, and no divergence	every UNRESOLVED	0/48	0.074	yes

no release was accepted during loaded carry, although the loss watchdog was armed in every flight before the command, a median of 3.2 s after the start of the lift. After every release the controller's model returned to the bare airframe. The confirmation came 2.17 s after physical detachment at the median, from 1.86 s to 2.52 s (Table 5.5); under delayed detachment it came 6.20 s after the command and the same 2.17 s after detachment, so holding the joint changed nothing but the wait. The delay depends mainly on the payload, 2.04 s for 0.15 kg and 2.40 s for 0.30 kg at the median, as in the historical campaign.

The mass estimate was accurate in loaded hover (median MAE 2.7 g) and in manoeuvre was biased low by 0.66 % at the median at $4\,\mathrm{m\,s^{-1}}$. It did not collapse: in no flight did the estimated payload fall below half the true payload during the loaded figure-eight. The estimate returned to within 0.03 kg of the empty value 1.7 s after detachment at the median. Loaded tracking error was about 0.03 m at $2\,\mathrm{m\,s^{-1}}$ and 0.11 m at $4\,\mathrm{m\,s^{-1}}$, and the mean vertical error in the figure-eight was 0.6 mm. The collective thrust never reached its bound; the yaw torque was at its box limit in 2.5 % of loaded samples at $4\,\mathrm{m\,s^{-1}}$, and no motor command saturated in more than 1.2 % of loaded samples in any flight of the campaign. These figures are of the same order as those of the 48 R flights of the F/R comparison; the two batches come from different code bases

Table 5.4: Binary outcomes of the deployed-system validation, all 180 **eligible flights. Early: release accepted before the physical detachment (for loss: loss declared before it). Carry: release accepted during loaded carry before any command or injected loss. Armed: loss watchdog armed before the command or the loss. Two-sided** 95 % **Clopper–Pearson intervals:** n/n **has lower limits** 0.90**,** 0.86 **and** 0.74 **for** $n = 36$**,** 24 **and** 12**;** $0/n$ **has upper limits** 0.10**,** 0.14 **and** 0.26**. No flight diverged, logged a controller solve failure, ended an estimator solve with non-zero status or re-initialised the estimator.**

Cell	n	Confirmed	Early	UNRESOLVED	Carry	Armed	Loss detected
Release, d0	36	36/36	0/36	0/36	0/36	36/36	—
Release, d4	36	36/36	0/36	0/36	0/36	36/36	—
Uncommanded loss	24	—	0/24	0/24	0/24	24/24	24/24
Jammed gripper	24	0/24	—	24/24	0/24	24/24	—
Gain 0.90, d0	24	24/24	0/24	0/24	0/24	12/24	—
Gain 1.10, d0	24	0/24	—	24/24	0/24	24/24	—
Gain 0.90, d4, 0.15 kg	12	12/12	**12/12**	0/12	0/12	0/12	—

and are not compared statistically.

Faults. Every uncommanded loss was detected ($24/24$, $[0.86, 1]$), 2.15 s after detachment at the median (1.94 s to 2.50 s), never before it; in every case the payload geometry was zeroed, the inner-loop scale returned to the bare airframe, no planned release was issued afterwards and no flight diverged. No jammed release was accepted ($0/24$, $[0, 0.14]$) and the payload model was never cleared; every jam reached UNRESOLVED, braked and hovered, with a largest speed from 5 s after UNRESOLVED of $0.11\,\mathrm{m\,s^{-1}}$ at the median ($0.05\,\mathrm{m\,s^{-1}}$ to $0.18\,\mathrm{m\,s^{-1}}$).

Over-reading thrust map (1.10)**.** The empty vehicle read 9.8 % heavy after release, so no release was confirmed ($0/24$); all 24 flights reached UNRESOLVED and hovered safely, with a largest speed from 5 s after it of $0.11\,\mathrm{m\,s^{-1}}$ at the median ($0.05\,\mathrm{m\,s^{-1}}$ to $0.18\,\mathrm{m\,s^{-1}}$). The estimator's mass was 9.4 % high in loaded carry. With the heavier payload this put the estimate above the health band for 94 % to 95 % of loaded flight, and the controller held its last accepted model mass throughout (criterion A3); with the lighter payload the estimate stayed inside the band and was used. The loss watchdog armed in every

Table 5.5: Nominal release in the deployed system, all 72 flights of the release segment (d0 and d4 pooled; the delay from detachment did not differ between them). Median [minimum, maximum] over flights; medians only by speed. Mass windows: loaded, from 5 s after lift to the command; post-release, from 5 s after detachment to the end of the flight, as in Table 5.19; hover and figure-eight, and the collapse fraction, as in Section 5.5.3. Position RMSE: loaded from 5 s into the figure-eight to the command, post-release from detachment to 25 s after it. Yaw at bound: share of loaded samples with the yaw torque at its box limit. Solve times are per-flight percentiles; the controller's is sampled (below). Except for the confirmation delay, these metrics were defined after the campaign started and are exploratory.

Metric	All ($n = 72$)	$2\,\mathrm{m\,s^{-1}}$ ($n = 36$)	$4\,\mathrm{m\,s^{-1}}$ ($n = 36$)
Confirmation after detachment [s]	2.17 [1.86, 2.52]	2.30	2.10
Loaded mass MAE [kg]	0.0127 [0.0069, 0.0259]	0.0094	0.0153
Loaded-hover mass MAE [kg]	0.0027 [0.0008, 0.0036]	0.0026	0.0027
Figure-eight mass MAE [kg]	0.0136 [0.0070, 0.0289]	0.0099	0.0166
Figure-eight mass bias [%]	−0.58 [−1.34, −0.28]	−0.41	−0.66
Collapse fraction	0 [0, 0]	0	0
Post-release mass MAE [kg]	0.0091 [0.0048, 0.0171]	0.0067	0.0103
Loaded RMSE [m]	0.069 [0.024, 0.134]	0.030	0.113
Post-release RMSE [m]	0.062 [0.021, 0.127]	0.033	0.102
Post-release peak error [m]	0.141 [0.062, 0.429]	0.124	0.179
Yaw torque at bound, loaded [%]	0.9 [0, 2.9]	0	2.5
Estimator solve P50 / P99 [ms]	0.31 / 0.37	0.25 / 0.35	0.35 / 0.39
Estimator solve maximum [ms]	0.41 [0.36, 2.28]	0.38	0.44
Controller solve median, sampled [ms]	1.7 [1.5, 2.1]	1.5	2.0

flight, so an uncommanded loss would also have gone undetected; that was not flown.

Under-reading thrust map (0.90) with immediate detachment. Every release was confirmed (24/24), none early, 1.20 s after the command at the median: 0.95 s with the lighter payload and 1.50 s to 1.55 s with the heavier. The force-balance mass read 10.45 % low. With the lighter payload the loaded vehicle therefore read as empty, and the loss watchdog never armed (0/12): protection against uncommanded loss was silently absent. The estimator's mass was 10.5 % low, a loaded MAE of 0.24 kg. The estimate, which the controller uses whenever it is healthy, therefore lay below the bare airframe with the lighter payload and recognised only about a sixth of the heavier one; the estimator ended no solve with non-zero status and never re-initialised. No flight

diverged.

Under-reading thrust map with delayed detachment: early confirmation. In all 12 flights with gain 0.90, the lighter payload and detachment held for 4 s, the release was accepted while the payload was still attached ($12/12$, $[0.74, 1]$): 0.95 s after the command and 3.09 s before the physical detachment at the median (3.08 s to 3.13 s). The dual enabling still prevented any acceptance during carry ($0/12$), because the watchdog never armed and the rule consumed the evidence only once the command was pending. The mechanism follows from (4.34)–(4.36) and is confirmed by the logged values. With gain $1+\varepsilon$ the loaded vehicle reads $m_{\text{acc}} = (1+\varepsilon)(m_B + m_P)$, and the score treats it as empty when $m_{\text{acc}} - m_B \leq 0.015\,\text{kg}$, that is for every payload with

$$m_P \leq \frac{m_B + 0.015}{1+\varepsilon} - m_B, \tag{5.10}$$

which is 0.25 kg at $\varepsilon = -0.10$. The 0.15 kg payload is 6.8 % of the composite mass, less than the 10.4 % under-reading measured in these flights, so the loaded reading, 1.98 kg, lay below the bare-airframe mass itself. The score was at the empty level throughout carry, the watchdog had no evidence of load to arm on, and when the command opened the rule the only thing left to wait for was the 1 s persistence hold. Under the same error the payload is read clearly as loaded ($m_{\text{acc}} - m_B \geq 0.060\,\text{kg}$) only above about 0.30 kg, which is why the heavier payload, just at that limit, armed the watchdog in all 12 of its gain-0.90 flights, but later than nominally (4.4 s against 3.2 s after the start of the lift). This is a working limit of the deployed system: with an under-reading thrust map the force balance cannot tell a light payload from none, and under delayed detachment the system then declares a delivery complete, and resets the residual specific force, while the payload is still on the hook. In these flights the controller's model already held a mass at or just below the bare airframe, and the vehicle neither diverged nor lost the track (post-release peak error 0.10 m at the median); the error is in the lifecycle. The rule is correct

only while the thrust map is accurate to better than the payload's share of the composite mass, and the 0.015 kg threshold must be read against that calibration requirement.

Real time. The reduced estimator logs every solve. Its per-flight median, 95th and 99th percentiles and maximum were 0.31, 0.35, 0.37 and 0.41 ms at the median over the release flights, and the largest single solve in the campaign took 2.28 ms, 2.3 % of the 100 ms period. The controller's solve time is recorded only in sampled diagnostic lines at 0.1 ms resolution; its per-flight median was 1.7 ms (1.5 ms at $2\,\mathrm{m\,s^{-1}}$, 2.0 ms at $4\,\mathrm{m\,s^{-1}}$). Its 99th percentile, maximum and deadline overruns cannot be obtained from these logs and are not reported.

5.3.4 Relation to the historical campaign

The historical release campaign (Section 5.7) flew the same release rule with the full-state estimator, from the campaign code base. Set side by side, descriptively, the lifecycle results agree: confirmation 1.86 s to 2.52 s after detachment here against 1.90 s to 2.51 s there, no early acceptance under nominal thrust, every uncommanded loss detected and no jammed release accepted, the same missed releases with the over-reading map and the same unarmed watchdog with the lighter payload and the under-reading map. That is expected, because rule D does not read the mass estimate. What differs is the estimator: here the loaded estimate never collapsed, where the full-state estimate did at $4\,\mathrm{m\,s^{-1}}$. The early confirmation under an under-reading map with delayed detachment, predicted but not flown in the historical campaign, is observed here for the first time.

5.3.5 Scope

The campaign shares the boundaries of every simulation result in this chapter: one airframe and gripper geometry, centred grasps, no wind, a plant that generates thrust with the static law the estimator and the detector invert, and an inertial unit at the body origin (Section 4.7.5). The thrust-map error was applied on the estimation side only and at $\pm 10\,\%$ only. Delayed detachment was combined with the under-reading map for the lighter payload only; the heavier payload, the over-reading map and smaller errors were not flown with delayed detachment, and uncommanded loss was not flown with a thrust-map error. Zero counts are bounded by the cell sizes in Table 5.4. The acceptance criteria were fixed during the run, not before it, as stated above. The data, per-flight table and report are in `results/campaign_180_analysis/` of the mainline code base, and Section 5.10.4 gives the command that reproduces the campaign.

5.4 Experimental Questions and Hypotheses

The historical cohort, with the full-state estimator and the mass-evidence lifecycle, is organised around five questions. For each, the design of Chapters 3 and 4 implies an expected outcome, stated here so that the results can be read against it.

Q1 **Mass identifiability.** Is the composite mass identified through grasp, carry and release from the measured wrench and odometry alone? *Expected:* accurate in loaded hover; biased low in sustained manoeuvre, because rotational model error reaches the mass through the attitude (Section 3.4.2).

Q2 **Release completion.** Is every physical release accepted, and how long after the payload has left? *Expected:* completion, at a delay of at least the 1 s persistence hold plus the time the estimate needs to fall.

Q3 Delayed detachment. When the command precedes the physical detachment, does an evidence-gated arm avoid accepting a release while the payload is still attached? *Expected:* the command-triggered arm accepts early by the length of the hold; the evidence-gated arms do not.

Q4 Arming policy. What does running the confirmation before the command cost in false acceptances during carry, and what does command-arming cost in delay? *Expected:* the always-on rule inherits the manoeuvre-induced collapse of the estimate (Section 4.7.6).

Q5 Tracking. Does the release arm change tracking, and what does the residual specific force contribute? *Expected:* no difference between arms, since the controller's mass follows the estimate in all of them; an improvement from the residual, which compensates the mass bias.

Failure modes and operating limits are examined separately in Section 5.9.

5.4.1 Release-acceptance arms

The release-acceptance logic of the campaign code base is parameterised, so that the arms differ in exactly one factor wherever possible. Table 5.6 lists them. Arm B is the rule originally proposed for this interface and was the default of the implementation during the cohort; arm C differs from it only in when the persistence count may start. Arm D, the deployed rule, was added after the cohort and is evaluated in Section 5.7.

What *accepting* the release does should be stated precisely, because in the continuous interface it is less than the name of arm A suggests. Accepting marks the release complete in the lifecycle and resets the residual specific force $\hat{\boldsymbol{d}}$ of Section 4.4.3, which was compensating the loaded vehicle. It does *not* replace the controller's mass or inertia: in every arm those follow the published estimate throughout, so a vehicle whose payload is still attached keeps a loaded

Table 5.6: Release-acceptance arms, switchable settings of the campaign code base. In every arm the controller's mass and inertia follow the estimate continuously. Arms A–C differ only in when a release is accepted; arm D, added after the cohort and the only rule of the deployed controller, differs also in its evidence.

Arm	Name	Release-acceptance rule
B	Always-on confirmed	evidence score (4.31) counted throughout loaded flight; the command does not accept the release
A	Command-immediate	accepts the release at the command itself, with no confirmation
C	Command-armed confirmed	the same confirmation, but persistence counted only after the command arrives
D	Force-balance, dual-enabled	(4.31) on γ_{acc}, opened by the command or the loss watchdog; the deployed rule; release campaign only (Section 5.7)

mass in its model whatever the arm has declared. The attach-time inertia floor is ended by the release command in all three arms (Section 4.6.3), after which the inertia, too, is whatever the estimate implies. The arms therefore differ only in *when* the release is declared complete and *when* $\hat{\boldsymbol{d}}$ is reset.

5.4.2 Conditions and scenarios

Three nominal conditions are used: 0.15 kg at $2\,\mathrm{m\,s^{-1}}$, 0.15 kg at $4\,\mathrm{m\,s^{-1}}$ and 0.30 kg at $4\,\mathrm{m\,s^{-1}}$, labelled c1, c2 and c3. The last is deliberately a stress condition, with hover at 74 % of maximum thrust, for reasons quantified in Section 5.9.3.

Two scenarios are crossed with those conditions in the frozen cohort:

1. **Nominal grasp–carry–release** (d0), arms A and B.
2. **Delayed detachment** (d4), arms A, B and C. A fault injector accepts the normal release command but holds the physical joint for 4 s. This separates "the command was issued" from "the payload left".

One further scenario was implemented but is outside the cohort. **Unsignaled loss**, in which the payload is detached at the release phase with no command and no notification, emulates a gripper that opens by itself or a load torn off. It was *not* run in the cohort, because in the evaluated configuration the attach-time inertia floor is released only by the release command (Section 4.6.3): after a loss without a command the empty airframe keeps flying with the envelope inner-loop scale, and smoke flights of this scenario showed pitch-moment saturation in 8–31 % of samples. A partial-loss scenario and a thrust-map mismatch scenario were also designed but were not run in the cohort; Section 5.10.2 states what is and is not known about them. Uncommanded loss and a thrust-map error, applied to the thrust shared by the estimator and the release detector, were later flown under the force-balance rule, which ends the inertia floor on detected loss (Section 5.7).

5.5 Cohort Design, Eligibility and Metrics

The cohort has five arm–scenario cells per condition, 15 cells in all. Each block runs every cell once, in an order drawn from a fixed random seed, and eight blocks give 120 flights. Every flight launched enters the statistics, and none is re-run: a flight that crashes or fails to grasp is reported as such, under the eligibility rule below. The arms are implemented by the release-baseline switch of the controller (A: accept at the command; B: always-on confirmation; C: command-armed confirmation). The cohort was launched on 30 October 2026 with block seed 1793358067; all 120 scheduled flights ran, and the attempts and exclusions are listed in Table 5.8.

Eligibility is declared before scoring: a flight enters the release statistics if the payload physically attached, according to the gripper's own attach record, and the vehicle survived to the release command. Physical attachment is used rather than the estimator's own mass evidence, because the latter would silently ex-

clude flights whose estimator failed to recognise an attached payload — exactly the cases most worth reporting.

5.5.1 Sources of evidence

Table 5.7 lists every source of evidence used in this chapter. The first row, the deployed-system validation, is the primary evidence for the system as deployed (Section 5.3). The frozen cohort supports the historical results of Section 5.6; the historical release campaign, of the same standing, is analysed on its own in Section 5.7, and no two of these batches are pooled. The residual ablation answers one question in one configuration; the horizontal-offset ablation was run in an earlier, eccentric configuration; and development flights are cited only for mechanisms, never pooled with frozen results or used to estimate an effect size.

The deployed system is the combination of R and rule D. Its primary evidence is the 180-flight validation of Section 5.3, flown with the deployed system itself from the mainline code base under nominal release, delayed detachment, uncommanded loss, a jammed gripper and a thrust-map error. The R arm of the estimator comparison — 48 flights with a nominal thrust map and no injected fault, plus four thrust-map smoke flights — flew the same combination from the campaign code base; it is the evidence for choosing R, and it is a different batch, neither paired nor pooled with the validation. The fault, jam and thrust-map results of the historical release campaign, and every result of the cohort, were obtained with F. Where the validation repeats them, its results are the ones that describe the deployed system; the historical results are kept because they document how the design was reached, and because the cohort's mass-evidence rules were flown only there. The mass-estimation results of the cohort and the historical campaign do not transfer to R at all.

Table 5.7: Sources of evidence in this chapter, in decreasing order of standing, with the estimator and release rule each was flown with. F: full-state estimator of Section 5.2.1; R: reduced estimator of the deployed system. Release rules: A, accept at the command; B, C, estimator mass evidence (always-on, command-armed); D, force balance with loss watchdog.

Source	Flights	Est.	Rule	Use	Sec.
Deployed-system validation	180 (180 eligible)	R	D	the deployed system, mainline code base; nominal and delayed release, uncommanded loss, jam, thrust-map error	5.3
Frozen cohort	120 (118 eligible)	F	A/B/C	mass-evidence lifecycle; d0 and d4; three conditions; primary results for that rule	5.6
Release campaign	312 (312 eligible)	F	D	force-balance rule; uncommanded loss, jam, thrust-map error; fixed and $\mathscr{L}_1$ controllers	5.7
Residual ablation	48	F	B	residual on/off, nominal release	5.8.1
Estimator comparison	96 (48 pairs)	F, R	D	F against R, nominal thrust map, no fault	5.8.2
of which R	48 + 4 smoke	R	D	the deployed combination in the campaign code base; nominal thrust map, no fault	5.8.2
Horizontal-offset ablation	8 (4 + 4)	earlier	—	eccentric configuration, 0.15 kg	5.8.3
Development flights	single to 8	earlier	—	changes confounded; mechanisms only	5.9.1

5.5.2 Attempts and exclusions

Table 5.8 accounts for every flight launched. Two were ineligible. In one, the parcel was 0.47 m below the gripper at the grasp, just outside the attachment window — most plausibly the vertical drift of the grasp height described in Section 5.9.2 — and the grasp was never made. In the other, the simulator stopped advancing before take-off and the flight produced no estimator or controller data; this is an infrastructure failure, and it is reported rather than re-run, in accordance with the no-re-run rule of Section 5.5. No flight that produced data crashed or diverged in any phase.

Table 5.8: Attempts and exclusions in the frozen cohort. Eligible: physically attached according to the gripper's record, and survived to the release command. Crashed: before the release command. Conditions as in Section 5.4.2.

Cond.	Attempted	Attached	Eligible	Crashed	Excluded
c1	40	40	40	0	—
c2	40	39	39	0	1 missed grasp (A, d0)
c3	40	39	39	0	1 simulator stall (C, d4)
Total	120	118	118	0	2

5.5.3 Metrics

Position tracking is summarised by

$$\mathrm{RMSE}_p = \sqrt{\frac{1}{K}\sum_{k=1}^{K} \left\| \boldsymbol{p}_k - \boldsymbol{p}_k^{\mathrm{ref}} \right\|_2^2}. \tag{5.11}$$

Estimator metrics are the composite-mass error in loaded hover, in loaded manoeuvre and after release; the fraction of loaded figure-eight samples in which $\hat{m}_P$ falls below half the true payload, called the *collapse fraction*; the fraction of samples resting on a bound; and the number of window re-anchorings. The collapse threshold is relative because an absolute one does not transfer between payloads: a fixed 0.15 kg threshold, used in earlier development work at 0.30 kg where it coincides with the relative definition, would classify every sample of a 0.15 kg flight with any negative bias as collapsed. Hover is the interval from 2 s after the end of the lift ramp to the start of the figure-eight; loaded figure-eight runs from 5 s after its start to the release command; post-release runs from 10 s to 40 s after physical detachment. Position RMSE in loaded figure-eight uses the same interval, and post-release RMSE the interval from 5 s to 35 s after detachment. Lifecycle metrics are the completion rate, the confirmation delay measured from the release command to the end of the 1.0 s evidence-score hold, and the count of confirmations occurring during loaded carry before any command or injected loss. Computational metrics are the median and upper percentiles of the solve time for both optimisers.

Ground truth is used only in offline scoring. The composite-mass truth is the airframe mass plus the payload mass while the simulator's joint record shows the payload attached; it never reaches the estimator.

5.5.4 Experimental unit and statistical reading

The experimental unit is one flight, or one pair of flights where two arms are compared. No experimental factor is randomised within a cell: there is no wind, no randomised initial state, payload mass or grasp position, and the recorded seeds determine only the order of cells within blocks and the pairing. Two sources make repeated flights of one cell differ. The simulator adds small Gaussian noise to its sensors (inertial unit: $8.7 \times 10^{-4}\,\mathrm{rad\,s^{-1}}$ on the rates and $6.4 \times 10^{-3}\,\mathrm{m\,s^{-2}}$ on the specific force; barometer and magnetometer), from a generator whose seed is not recorded. And the multi-process stack — simulator, autopilot, bridge, estimator and controller, each on its own clock and thread — does not schedule its messages identically from run to run. The resulting variation is visible in every table, for example in the grasp height, the timing of the release relative to the figure-eight phase and the solver iteration counts. It is not a sample of the physical variation a fielded vehicle would meet, and the rotor speeds and odometry the estimator uses are close to noise-free.

The statistics are read accordingly. Wilcoxon tests, bootstrap intervals and Clopper–Pearson intervals describe the *repeatability* of an effect across replicates of the same simulated conditions; they do not license inference to other airframes, sensors or environments. Effect sizes — paired medians with their intervals and better/same/worse counts — are the primary quantities; p values are secondary. Two further limits apply. With n pairs the smallest attainable two-sided Wilcoxon p is 2^{1-n}, so $p = 0.031$ with six pairs and $p = 0.008$ with eight mean only that every pair moved in the same direction; and a zero count in n flights bounds the rate only by the Clopper–Pearson limit, 0.37 for

$n = 8$ and 0.14 for $n = 24$. No correction for multiple comparisons is applied. Endpoints fixed before a batch are called *pre-specified*: they were written to a dated file in the batch directory before the first flight, but not deposited in a public, time-stamped registry. Subgroup results and every metric not fixed before its batch are exploratory, and are reported as such.

5.6 Primary Results

5.6.1 Mass estimation

The behaviour the design intends is that $\hat{m}$ rises with no attach notification, holds near truth in loaded hover, and returns to the bare-airframe value after release without any event resetting it. The behaviour it does *not* guarantee is accuracy in sustained manoeuvre, where the estimator's approximate rotational model (Section 3.4.2) leaks into the mass. Figure 5.1 shows both on the nominal flights of the cohort, and Table 5.9 summarises every physically attached flight.

Table 5.9: Composite-mass error by flight phase, all physically attached cohort flights (all arms and scenarios; the estimator is identical across arms). Errors are medians over flights of the per-flight mean, with the range of per-flight values in brackets; post-release error is relative to m_B. Collapse fraction: share of loaded figure-eight samples with $\hat{m}_P < \frac{1}{2} m_P$. Bound: share resting on $m_T = 0.95\, m_B$.

Cond.	n	Hover [%]	Figure-eight [%]	Post [%]	Collapse	Bound
c1	40	−0.21	−0.97 [−2.50, −0.47]	−0.94	0.00 [0, 0.12]	0.000
c2	39	−0.22	−4.09 [−6.27, −2.37]	−1.70	0.38 [0.21, 0.73]	0.010
c3	39	−0.29	−3.30 [−8.09, −1.62]	−1.68	0.03 [0, 0.40]	0.014

Three findings follow. First, in loaded hover the estimate is accurate in every flight: the per-flight mean error lies between −0.44 % and −0.10 % of the composite mass, which for the lighter payload is within about 10 g. No physically attached flight failed to recognise its payload. Second, in the figure-eight

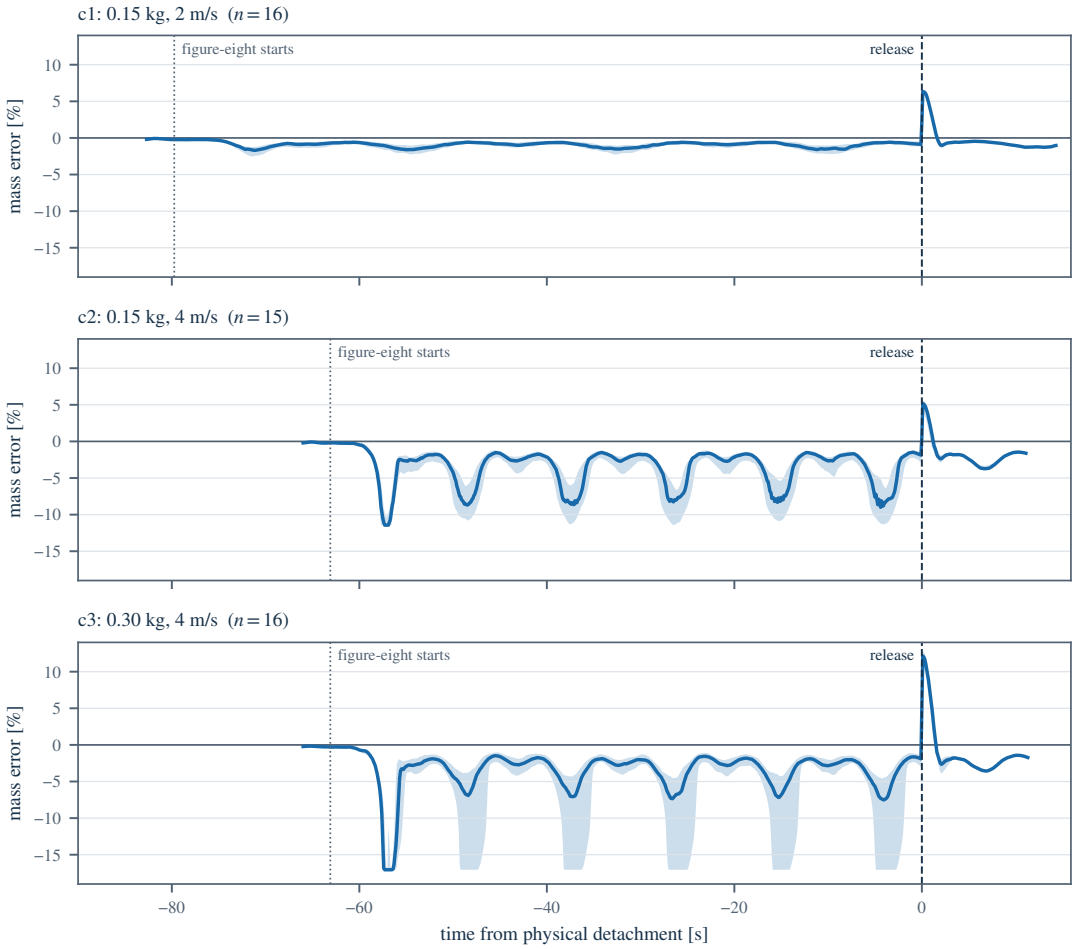


Figure 5.1: Composite-mass error $(\hat{m} - m)/m$ of the nominal (d0) cohort flights, each aligned on its own physical detachment and scored against its own truth. Line: median over flights; band: 10th–90th percentile. Traces start at the end of the lift ramp, because before lift-off the parcel rests on the ground and the joint record is not a usable truth. The positive transient at release is the estimator's lag behind an instantaneous truth step. The floor of the c3 band at $-17\,\%$ is the mass lower bound $0.95\,m_B$.

the bias is small at $2\,\mathrm{m\,s^{-1}}$ (median $-1.0\,\%$) and roughly four times larger at $4\,\mathrm{m\,s^{-1}}$, and it is not constant: the error dips about once every 11 s, half the figure-eight period, to $-6\,\%$ to $-9\,\%$ in the median trace. The first dip, at the entry to the figure-eight while the speed ramps up, takes the median of both $4\,\mathrm{m\,s^{-1}}$ conditions to the lower mass bound. Because the dip is a fixed fraction of the composite mass, it is a larger fraction of the *payload* for the lighter one, which is why the collapse fraction is highest at c2 (median 0.38) although c3's dips reach the bound more often. Third, after release the estimate settles slightly below the bare airframe, $-0.9\,\%$ to $-1.7\,\%$ of m_B; since

$\hat{m}_P = (\hat{m} - m_B)^+$ is then zero, the release confirmation is unaffected.

The manoeuvre-induced dips of the second finding are the collapse that Section 5.9 discusses as the dominant failure of the interface. In the evaluated configuration the collapse fraction had medians of 0, 0.38 and 0.03 at c1, c2 and c3, with the per-flight maximum reaching 0.73 at c2 (Table 5.9); at $4\,\mathrm{m\,s^{-1}}$ the estimate touched the mass lower bound at the entry to the figure-eight in at least half the flights, and in some c3 flights at every dip thereafter (Figure 5.1). Its lifecycle consequence is reported in Section 5.6.4. The collapse no longer cost a flight: the inertia floor held the inner-loop scale at 2.34 through every loaded figure-eight (Section 5.9.3), the residual kept the vertical tracking error below 1 mm on average (Section 5.8.1), and no eligible flight diverged. But it was not prevented, and its effect on the lifecycle is undiminished. Development flights that show the mechanism of the collapse, in earlier configurations, are reported separately in Section 5.9.1.

5.6.2 Release completion and delay

Two delay modes are possible by construction and would have to be reported separately rather than pooled: confirmation while the vehicle is still flying the figure-eight, and confirmation after the 12 s timeout has declared UNRESOLVED and the vehicle has braked to hover. In the cohort only the first occurred. Table 5.10 lists completion and delay for every eligible flight.

Every one of the 118 eligible flights completed its release (CP interval $[0.969, 1]$); none reached UNRESOLVED, and no flight diverged. Pooled over conditions and scenarios, arm B completed 48 of 48 ($[0.926, 1]$). Arm B's confirmation arrived 2.3 s to 3.2 s after the payload physically left, in manoeuvre, with medians of 3.0 s at $2\,\mathrm{m\,s^{-1}}$ and 2.5 s to 3.1 s at $4\,\mathrm{m\,s^{-1}}$. About 1 s of this is the persistence hold of (4.31); the rest is the time the estimate takes to fall through the evidence-score transition band (Figure 5.1). That delay is the price

Table 5.10: Release completion and confirmation delay, all eligible cohort flights. Delay is from the release command to acceptance; under delayed detachment (d4) it is also given from the physical detachment. CP: two-sided 95 % Clopper–Pearson interval on the completion rate. Ranges are over flights.

Cond.	Arm	Scen.	Done	CP	From command [s]	From detachment [s]
c1	A	d0	8/8	[0.63, 1]	0	0
	A	d4	8/8	[0.63, 1]	0	−4.08
	B	d0	8/8	[0.63, 1]	2.96 (2.76–3.00)	2.96
	B	d4	8/8	[0.63, 1]	7.16 (7.10–7.26)	3.07
	C	d4	8/8	[0.63, 1]	7.13 (7.10–7.16)	3.07
c2	A	d0	7/7	[0.59, 1]	0	0
	A	d4	8/8	[0.63, 1]	0	−4.07
	B	d0	8/8	[0.63, 1]	2.60 (2.30–2.70)	2.60
	B	d4	8/8	[0.63, 1]	6.56 (6.30–6.76)	2.51
	C	d4	8/8	[0.63, 1]	6.65 (6.40–6.76)	2.62
c3	A	d0	8/8	[0.63, 1]	0	0
	A	d4	8/8	[0.63, 1]	0	−4.03
	B	d0	8/8	[0.63, 1]	3.03 (2.90–3.16)	3.03
	B	d4	8/8	[0.63, 1]	7.10 (6.96–7.26)	3.06
	C	d4	7/7	[0.59, 1]	7.00 (6.96–7.20)	2.97

of confirmation under nominal conditions, where the command is true.

The completion rate does not by itself show that the confirmations were correct. Arm B also confirmed releases that had not happened: 114 times during loaded carry, all at $4\,\mathrm{m\,s^{-1}}$, before any command. These are reported with arm C in Section 5.6.4; arm C confirmed real detachments in the same 2.4 s to 3.2 s range (Table 5.10).

5.6.3 Delayed detachment

A command-triggered arm accepts the release at the command and therefore declares, for the whole of the hold, that a payload has gone which is still attached. As Section 5.4.1 explains, in the continuous interface this does not put an empty model on the loaded vehicle: the controller's mass and inertia keep following the estimate, which still sees the payload. What the early acceptance

does change is the lifecycle's report, which is wrong for the duration of the hold, and the residual $\hat{d}$, which is reset while the mass bias it was compensating is still present. An evidence-gated arm should accept in none of these flights.

That is what the cohort shows. Arm A accepted before the joint was removed in all 24 of its delayed-detachment flights, about 4.0 s early in each; arms B and C accepted early in none of their 47 (Table 5.10). Measured from the physical detachment, arm B's confirmation came 2.3 s to 3.2 s later (medians 2.5 s to 3.1 s), the same as under nominal release: holding the joint for 4 s neither produced a confirmation during the hold nor slowed the one after it. What this experiment shows is a difference in *lifecycle correctness*. It does not show a difference in tracking or survival, and none was expected with the hold this short and the mass model unaffected; a permanently jammed gripper is a separate test.

5.6.4 Command-armed versus always-on confirmation

A delayed-detachment result reflects *persistent confirmation*: arm C, which starts counting only after the command, also never accepts early. The difference between arms B and C lies elsewhere. Arm B's confirmation rule runs throughout loaded flight and can fire during carry if the mass estimate collapses (Section 4.7.6); arm C's cannot, because it is not armed until the command. The comparison therefore measures what running the rule before the command costs in false confirmations.

Table 5.11 gives the result. Arm C confirmed no release during loaded carry in any of its 23 flights, a per-flight rate whose 95 % upper bound is 0.15. Arm B confirmed 114 releases that had not happened, in 20 of its 32 flights at $4\,\mathrm{m\,s^{-1}}$ and in none of its 16 at $2\,\mathrm{m\,s^{-1}}$. They are not isolated events: an affected flight typically confirmed about six times, spaced about 11 s apart, each time

Table 5.11: Release confirmations during loaded carry (after physical attachment, before the release command), all physically attached cohort flights. Arm A does not run the confirmation rule.

Cond.	B: flights	B: confirmations	C: flights	C: confirmations
c1	0 of 16	0	0 of 8	0
c2	14 of 16	78	0 of 8	0
c3	6 of 16	36	0 of 7	0

at an evidence score above 0.99. The spacing is the half-period of the figure-eight and coincides with the dips of Figure 5.1: each dip carries $\hat{m}_P$ below the confirmation threshold for longer than the 1 s hold. At c2 the light payload makes this nearly certain; at c3 the same relative dip leaves more payload mass above the threshold, and it happened in fewer flights.

In the evaluated configuration these false confirmations had no effect on control. With no release command pending, an acceptance neither resets $\hat{\boldsymbol{d}}$ nor ends the inertia floor, and the controller's mass follows the estimate in either case; the release command re-arms the rule, so the genuine release was still confirmed in every flight. The error is in the lifecycle's report: a delivery system that acted on *release confirmed* would, at $4\,\mathrm{m\,s^{-1}}$, have been told repeatedly that a parcel still on the hook had been delivered. Arm C avoids this by construction, at no cost in delay (Table 5.10); its price is that it cannot detect a release that was never commanded.

The comparison settles the arming policy. The always-on rule of arm B is a negative result: it shows that, in this configuration, mass evidence alone does not separate a loaded vehicle in manoeuvre from an empty one, and a rule that admits only that evidence will accept releases that have not happened. The command-armed rule of arm C keeps the property the delayed-detachment experiment established — it does not accept a release while the payload is still attached — without the false acceptances, and without a longer delay after detachment. It is the mechanism the present evidence supports. Its scope is correspondingly narrower: it distinguishes a commanded release from a physical

detachment, and it cannot by itself detect a load lost without a command.

5.6.5 Tracking performance

Under nominal conditions the command is true, so waiting for confirmation can gain nothing and costs a delay; arm A has zero confirmation delay by construction. The purpose of the nominal pairs is to establish that the always-on confirmed arm completes and tracks as well as the command-triggered one, not that it is better: in the continuous interface the controller's mass and inertia follow the estimate in both arms, so no tracking difference is expected.

Pairs are formed within a block at the same condition. Both arms completed every pair. Post-release position RMSE (median over pairs, arm A against arm B) was 0.028 m against 0.026 m at c1 ($n = 8$, Wilcoxon $p = 0.38$), 0.108 m against 0.088 m at c2 ($n = 7$, $p = 0.078$) and 0.104 m against 0.102 m at c3 ($n = 8$, $p = 0.74$). Loaded figure-eight RMSE, before any difference between the arms can act, was 0.029 m to 0.033 m at c1 and 0.118 m to 0.126 m at $4\,\mathrm{m\,s^{-1}}$ in both arms. No difference in post-release stability was detected; the c2 pair count is seven because one arm A flight missed its grasp (Table 5.8).

Tracking is therefore governed by the condition rather than by the release arm: loaded figure-eight RMSE is about 0.03 m at $2\,\mathrm{m\,s^{-1}}$ and about 0.12 m at $4\,\mathrm{m\,s^{-1}}$. What does change tracking is the residual specific force, whose contribution is isolated in Section 5.8.1.

5.7 Historical Release Campaign with Force-Balance Evidence

This campaign was flown with the full-state estimator F from the campaign code base, before the reduced estimator was adopted. Its release, fault and thrust-map experiments were repeated with the deployed system in Section 5.3, whose results supersede these for the deployed system; the controller comparison of Section 5.7.5 exists only here. The cohort leaves three questions open: whether release evidence that does not pass through the mass estimate removes the carry-phase false acceptances of the always-on rule; whether such a rule can detect a load lost without a command and remain safe when a commanded release fails; and how the interface compares with fixed-parameter and lumped-adaptive control. A second frozen campaign of 312 flights, run on 3–4 November 2026, addresses them with the force-balance rule of Section 4.7.5, here called arm D. Its results are reported separately from the cohort and are not pooled with it; the earlier cohort's logs were used only for offline replay before the campaign was frozen.

5.7.1 Design

Arm D is the dual-enabled force-balance rule: the score (4.36), consumed only when a release command is pending or the uncommanded-loss watchdog is armed. Arm A is the command-immediate arm of Table 5.6. The two arms share every other setting of the evaluated configuration of Section 5.2, and they differ only in the release lifecycle. A fourth condition, c4 (0.30 kg at $2\,\mathrm{m\,s^{-1}}$), completes a two-by-two mass–speed design with c1–c3 as defined in Section 5.4.2. Table 5.12 lists the four experiments.

In E1 each block contains one round of eight flights with nominal release and one with delayed detachment, both rounds covering the four conditions under

Table 5.12: Experiments of the force-balance release campaign. All four conditions c1–c4 appear in every experiment. d0: nominal release; d4: detachment held 4 s **after the command.**

Experiment	Flights	Factors	Arm
E1 Release semantics	144	arm A/D × d0/d4 × 4 conditions; 9 paired blocks	A and D
E2 Controller comparison	72	Fixed / $\mathscr{L}_1$ / MHE × 4 conditions; 6 rounds; d0	D
E3 Thrust-map error	48	detector thrust scaled by 0.90 / 1.10 × 4 conditions; 6 rounds; d0	D
E4 Faults	48	uncommanded loss / jammed gripper × 4 conditions; 6 rounds	D

both arms, with the order of the rounds and of the flights within them drawn from a recorded seed. In E2 the three controllers share the arm-D lifecycle, the inertia floor and the inner-loop scheduling, and differ only in their translational compensation. *Fixed* keeps the bare-airframe mass in the prediction model with no translational adaptation. $\mathscr{L}_1$ keeps the same mass and compensates with the residual $\hat{\boldsymbol{d}}$ alone. *MHE* is the evaluated configuration, with the estimated mass and $\hat{\boldsymbol{d}}$. E3 models a miscalibrated thrust coefficient k_f on the estimation side: the thrust reconstructed from rotor speeds is scaled by a fixed factor in both of its consumers, the estimator's thrust input and the thrust (4.34) of the force-balance detector. The plant's rotor coefficient, the controller's thrust-to-throttle map and the thrust used by the residual $\hat{\boldsymbol{d}}$ keep the nominal k_f. The payload mass and thresholds are not altered, and the factor actually applied is recorded in the estimator's and the controller's log of every flight (0.900 and 1.100 in 24 flights each). Because the estimated mass enters the controller's model, the error also reaches the control input, through the mass estimate. In E4 an *uncommanded loss* detaches the payload at the release phase with no command, no pending release and no notification to the estimator. A *jammed gripper* receives the normal command and never releases.

Thresholds, filters, arming and confirmation times, controller and trajectory parameters were frozen before the first flight. A fingerprint of the flight code was recorded at the start and checked before every flight; it did not change.

A flight was eligible if the payload physically attached, the vehicle survived to the release command or the injected fault, the estimator, controller, gripper, inertial, rotor-speed and residual logs were complete, and the recorded configuration matched the schedule. False or missed confirmations, divergence, solver failures, large tracking errors, an UNRESOLVED jam and a failed recovery after loss count as results, never as exclusions; in E2 a divergence before the command counts as a controller failure. Ineligible flights would have been replaced, and only those; none occurred. All 312 scheduled flights attached and were eligible, so every cell has its planned size (nine per E1 cell, six per cell elsewhere). No flight in the campaign diverged. Post-release metrics here use a window from physical detachment (or the injected loss) to 25 s after it, which differs from the cohort's window; post-release numbers are not comparable across the two.

5.7.2 Release semantics and carry-phase acceptance

Table 5.13: Release acceptance under the force-balance rule (E1). Median (range) over nine flights per cell. det.: time of acceptance relative to physical detachment (negative = before the payload left); cmd: relative to the release command. Every flight in every cell completed its release.

	A, d4	D, d0	D, d4	
Cond.	**det. [s]**	**det. [s]**	**cmd [s]**	**det. [s]**
c1	−4.05 (−4.09 to −4.04)	2.06 (2.00–2.15)	6.20 (6.10–6.26)	2.12 (2.05–2.21)
c2	−4.09 (−4.10 to −4.04)	1.94 (1.90–2.05)	6.10 (6.00–6.16)	2.01 (1.91–2.12)
c3	−4.09 (−4.09 to −4.00)	2.30 (2.20–2.44)	6.40 (6.25–6.45)	2.31 (2.20–2.44)
c4	−4.05 (−4.09 to −4.04)	2.40 (2.30–2.46)	6.46 (6.34–6.60)	2.41 (2.34–2.51)

Table 5.13 reproduces the delayed-detachment result of the cohort with the new evidence. Arm A accepted before the payload left in all 36 of its delayed-detachment flights, by 4.05 s at the median. Arm D accepted early in none of its 36 (95 % CP upper bound 0.10) and completed all 72 of its releases ($[0.950, 1]$). Its confirmation came 1.90 s to 2.51 s after physical detachment (median 2.20 s, 95th percentile 2.44 s). Holding the joint for 4 s did not change

the delay measured from detachment. About 1 s of it is the persistence hold; the rest is the two low-pass stages of (4.34)–(4.36).

The delay depends on the mass and hardly at all on the speed. In an effect-coded two-by-two regression on the nominal flights, the heavier payload lengthened the delay from detachment by 0.35 s (± 0.02, $p < 10^{-15}$), the higher speed shortened it by 0.08 s ($p = 0.0006$), and the interaction was not detectable ($+0.02$, $p = 0.64$); under delayed detachment the effects were $+0.30$, -0.14 and $+0.01$ s ($p = 0.76$ for the interaction). A larger payload leaves a larger step for the filtered score to traverse before it crosses 0.90.

The comparison that matters for the arming policy is carry. In every arm-D flight the watchdog armed after the lift and before the release command, a median of 3.2 s after the start of the lift (range 1.9 s to 3.9 s). From that point, the rule was able to accept a release at any time during carry, which is the always-on property that produced the cohort's 114 false acceptances. Across all 240 flights of the campaign flown with the arm-D lifecycle, the watchdog was armed in 228, and no release was accepted during loaded carry in any flight (95 % upper bound 0.016 per armed flight), including every armed flight at c2, the condition at which the mass evidence had failed in 14 of 16 cohort flights. The force balance therefore separates a loaded vehicle in manoeuvre from an empty one where the mass estimate did not. The twelve flights in which the watchdog did not arm are the thrust-map cases of Section 5.7.4.

Arm D's delay after detachment, 1.9 s to 2.5 s, is also shorter than the 2.3 s to 3.2 s of the cohort's mass-evidence arms. The two come from different batches and are not a paired comparison.

Tracking did not differ consistently between the arms. Of the eight within-block pairings of post-release RMSE (four conditions, two scenarios), seven showed no difference ($p \geq 0.07$, Wilcoxon, nine pairs each); at c3 with delayed detachment arm D's was higher by 0.014 m at the median ($p = 0.004$). As in the

cohort, the result concerns the lifecycle, not tracking.

5.7.3 Uncommanded loss and a jammed gripper

Table 5.14: Faults under the force-balance rule (E4), six flights per cell. Loss: payload detached with no command. Armed: watchdog armed before the loss. Delay: from detachment to the declared loss, median (range). Reset: geometry zeroed and inner-loop scale returned to the bare airframe. Jam: command issued, payload never released. False acc.: release accepted. Hover: UNRESOLVED, braked, then hovering below $0.5\,\mathrm{m\,s^{-1}}$.

	Uncommanded loss				Jammed gripper	
Cond.	**Armed**	**Detected**	**Delay [s]**	**Reset**	**False acc.**	**Hover**
c1	6/6	6/6	2.03 (1.76–2.10)	6/6	0/6	6/6
c2	6/6	6/6	1.94 (1.90–2.00)	6/6	0/6	6/6
c3	6/6	6/6	2.34 (2.20–2.50)	6/6	0/6	6/6
c4	6/6	6/6	2.38 (2.30–2.46)	6/6	0/6	6/6

Table 5.14 gives the fault results. The watchdog was armed before the loss in all 24 uncommanded-loss flights, and every loss was detected ($24/24$, $[0.86, 1]$), 1.8 s to 2.5 s after the payload left; none was declared before it. In every case the payload geometry was zeroed, the inner-loop scale returned from the floor value to that of the bare airframe, no planned release was issued afterwards, and no flight diverged; the largest post-loss position error had a median between 0.08 m and 0.20 m by condition. This is the scenario the cohort could not fly, because under the mass-evidence rules nothing other than the command ended the inertia floor (Section 5.4.2).

The jammed gripper is the converse test: the command is true and the detachment never comes. No release was accepted in any of the 24 flights ($0/24$, $[0, 0.14]$), and the payload model was never cleared. Every flight reached UNRESOLVED at the 12 s timeout, armed the braking reference and came to hover, with a largest speed in hover of $0.18\,\mathrm{m\,s^{-1}}$ over all flights.

5.7.4 Thrust-map error

Table 5.15: Estimation-side thrust-map error (E3), applied to the estimator and the force-balance detector together; detector quantities shown, six flights per cell. m_{acc} bias: median error relative to the true composite mass, in loaded carry and after release. γ_{acc} max: median over flights of the largest score in loaded carry. Delay from the command.

Gain	Cond.	Bias, carry	Bias, post	γ_{acc} max	Armed	Done	Delay [s]
0.90	c1	−10.4 %	−10.3 %	1.00	0/6	6/6	0.96
	c2	−10.7 %	−10.5 %	1.00	0/6	6/6	0.95
	c3	−10.7 %	−10.6 %	0.62	6/6	6/6	1.48
	c4	−10.4 %	−10.4 %	0.18	6/6	6/6	1.56
1.10	c1	+9.3 %	+9.8 %	0.00	6/6	0/6	—
	c2	+9.1 %	+9.8 %	0.00	6/6	0/6	—
	c3	+9.0 %	+9.8 %	0.00	6/6	0/6	—
	c4	+9.3 %	+9.8 %	0.00	6/6	0/6	—

The force-balance mass moved one-for-one with the injected error (Table 5.15), and the two directions failed differently, as the analysis of Section 5.10.1 predicts for the mass channel. A 10 % error in the composite mass is 0.22 kg to 0.24 kg, larger than the lighter payload and close to the heavier one.

*Overestimated thrust (*1.10*).* After release the empty vehicle read about 10 % heavier than it was, the score never rose, and none of the 24 releases was confirmed. Every flight reached UNRESOLVED, braked and hovered, without divergence. The failure is a miss, and it ends in the safe state the timeout was designed for. By the same arithmetic, an uncommanded loss would also go undetected; that was not flown.

*Underestimated thrust (*0.90*).* No release was accepted during carry and every release completed, faster than nominally. That outcome hides two failures. With the lighter payload the loaded vehicle already read as empty — the score was 1.00 throughout carry — so the watchdog never armed in any of the 12 flights: protection against uncommanded loss was silently lost. Confirmation followed the command by the length of the hold alone, because the score was already high; with delayed detachment, which E3 did not include, the same

rule would be expected to accept before the payload left. The deployed-system validation flew that combination, and the rule did accept early in every flight (Section 5.3). With the heavier payload the watchdog armed, but the largest carry score reached 0.85 in one flight, close to the 0.90 threshold. The dual enabling prevented every false acceptance in E3; it cannot restore evidence the thrust map has removed.

The estimator's mass. The same error reached the controller's model through the estimate. During loaded carry (from 5 s after lift to the command), the median mass in the controller's model was 7.4 % to 9.2 % above truth with the map 1.10. With the map 0.90 it stayed at the bare-airframe value in all four conditions (cell medians 2.064 kg to 2.076 kg, against 2.214 kg and 2.364 kg of truth): the payload was never recognised in the controller's model. This is the first of the two failures predicted in Section 5.10.1, and the replay of Section 5.8.2 shows the full-state estimate pressed against its $0.95\,m_B$ bound in this case. None of the 48 flights of E3 diverged.

5.7.5 Fixed, $\mathscr{L}_1$ and MHE controllers

All 72 flights completed their release without divergence or saturation of the collective thrust, with one solver failure in one flight, and the confirmation delay did not depend on the controller (Table 5.16). Flights were paired within rounds; with six pairs the smallest attainable two-sided Wilcoxon p is 0.031.

Against the fixed-parameter controller the benefit of adaptation is clear in loaded carry: the MHE controller had the smaller loaded RMSE in all 24 pairs ($p = 0.031$ in each condition), by 0.04 m to 0.14 m at the median, largest with the heavier payload. After release, when the fixed model is again correct, the difference in RMSE shrank to 0.002 m to 0.033 m and was consistent only at c1 ($p = 0.031$).

Table 5.16: Controller comparison under the force-balance rule (E2), nominal release. Medians over six flights per cell. Loaded RMSE: from 5 s into the figure-eight to the command. Post: detachment to 25 s after it. Delay from detachment.

Cond.	Controller	Loaded RMSE [m]	Post RMSE [m]	Post peak [m]	Delay [s]
c1	Fixed	0.087	0.031	0.090	2.00
	$\mathscr{L}_1$	0.032	0.030	0.077	2.03
	MHE	0.033	0.026	0.072	2.08
c2	Fixed	0.151	0.118	0.203	1.94
	$\mathscr{L}_1$	0.099	0.085	0.136	2.02
	MHE	0.123	0.102	0.163	1.97
c3	Fixed	0.218	0.130	0.211	2.25
	$\mathscr{L}_1$	0.114	0.107	0.198	2.29
	MHE	0.117	0.096	0.206	2.32
c4	Fixed	0.168	0.036	0.170	2.40
	$\mathscr{L}_1$	0.030	0.045	0.149	2.40
	MHE	0.028	0.033	0.134	2.40

Against the $\mathscr{L}_1$ controller, which compensates the same payload with the residual alone, no consistent difference in loaded tracking was found ($p \geq 0.16$ in every condition); at c2 the $\mathscr{L}_1$ controller was nominally the better in five of six pairs. After release the MHE controller had the smaller peak error at both $2\,\mathrm{m\,s^{-1}}$ conditions in all six pairs ($p = 0.031$), by 0.006 m to 0.017 m, and the smaller RMSE at c4 ($p = 0.031$); at $4\,\mathrm{m\,s^{-1}}$ there was no consistent difference. With the residual present in both, estimating the mass is therefore not shown to improve tracking in carry, and its measurable effect in this experiment is a small one in the transient after release at low speed. The three controllers also share the inertia floor and the inner-loop scheduling, so E2 does not test the rotational benefit of the payload model.

5.7.6 Scope of the campaign

The campaign has the limitations of the cohort — simulation only, one airframe and gripper geometry, centred grasps, no wind — and some of its own. The thrust-map error was applied to the estimator and the detector together.

The plant, the controller's thrust-to-throttle map and the residual were not perturbed, so E3 does not represent an error that also affects the actuation side. E3 also used only nominal release; delayed detachment with an under-reading map was flown later, with the deployed system (Section 5.3). Every zero-count result is bounded by the cell sizes: an upper bound of 0.10 for early acceptance and 0.14 for false acceptance under a jam. The interface code of the campaign was not committed; the record holds the base revision, a patch of the uncommitted changes and an archive of untracked files, as for the cohort (Section 5.10.4). The campaign's data and scoring scripts are in `release_paper_20261103_210022/`.

5.8 Ablations

5.8.1 The residual specific force

The residual $\hat{\boldsymbol{d}}$ of Section 4.4.3 is part of the evaluated configuration, and its contribution was isolated in a separate frozen batch of 48 flights run on 1 November 2026 (block seed 1793469773). Arm B of the cohort was flown with the residual on and off, everything else in the evaluated configuration unchanged; the flight records confirm that the residual switch is the only setting that differs between the two arms. Nominal release (d0) was crossed with the three conditions, eight blocks each, in an order drawn from the seed. The endpoints were fixed before the batch: primary, loaded figure-eight position RMSE and mean vertical error; secondary, post-release RMSE, completion, divergence, mass bias and confirmations during carry. All 48 flights attached, completed their release and did not diverge.

Table 5.17 gives the result. Without the residual the vehicle flies the loaded figure-eight below its reference, by 4.5 mm at c1 and by 28 mm and 44 mm at the two $4\,\mathrm{m\,s^{-1}}$ conditions; with it the mean vertical error is under 1 mm at

Table 5.17: Residual specific force on against off (arm B, nominal release). Medians over eight within-block pairs per condition; "better" counts the pairs in which the residual-on flight had the smaller value (for the vertical error, the smaller magnitude); p: Wilcoxon signed-rank. Vertical error is $z - z^{\mathrm{ref}}$ averaged over loaded figure-eight.

Metric	Cond.	On	Off	Better	p
Loaded figure-eight RMSE [m]	c1	0.033	0.047	8/8	0.008
	c2	0.116	0.148	7/8	0.016
	c3	0.106	0.145	8/8	0.008
Mean vertical error [mm]	c1	$+0.4$	-4.5	7/8	0.008
	c2	-0.8	-28.4	8/8	0.008
	c3	$+0.2$	-44.3	8/8	0.008
Post-release RMSE [m]	c1	0.023	0.044	8/8	0.008
	c2	0.089	0.132	7/8	0.016
	c3	0.073	0.115	8/8	0.008

every condition. Loaded figure-eight RMSE falls by 22 % to 31 % and post-release RMSE by 33 % to 48 %, in 23 of the 24 pairs on each metric. The sag grows with the manoeuvre-induced mass bias of Section 5.6.1, which is the error the residual is there to absorb: an estimate that is low by a few percent asks for a thrust that is low by the same fraction.

Equally informative is what the residual did not change. The loaded figure-eight mass bias was the same in both arms (medians -0.9 % against -0.8 %, -3.6 % against -3.1 %, -3.1 % against -3.5 %; all $p > 0.19$), and so were the confirmation delays (all $p > 0.45$) and the confirmations during carry, which occurred in both arms at c2. The residual compensates the *consequence* of the mass bias for tracking; it does not reduce the bias, and it does not touch the lifecycle.

5.8.2 Full-state versus reduced estimator

Every historical flight reported so far in this chapter was flown with the full-state estimator F of Section 5.2.1. The deployed default is now the reduced

velocity–mass estimator R of Section 4.2.1. This subsection gives the evidence for that change in three parts: an offline replay over the recorded release campaign, a four-flight thrust-map smoke test of R in closed loop, and a paired closed-loop comparison of F and R. The two estimators share horizon, sampling, alignment of inputs and measurements, the velocity, noise and arrival weights (4.9), initialisation and event logic; they differ only in model structure and in the mass bound and health check of Section 4.3.3.

Offline replay. R was solved offline on the odometry, the interval-averaged reconstructed thrust and the 10 Hz solve instants of all 312 flights of Section 5.7, and compared with the estimate F produced online in the same flights. The replay used F's lower bound $0.95\,m_B$, so that only the model differs. Loaded windows run from 5 s after lift-off to the command or detachment, post-release windows from 5 s after detachment; convergence is entry into a ± 0.03 kg band held for 1 s. Table 5.18 gives the flight-level medians for the 264 flights with a nominal thrust map.

Table 5.18: Replay of the reduced estimator on the 264 nominal-thrust flights of the release campaign, against the full-state estimate recorded online in the same flights. Flight-level medians; paired Wilcoxon signed-rank tests.

Metric	F (online)	R (replay)	R better	p
Loaded mass MAE [kg]	0.0359	0.0127	264/264	5×10^{-45}
Post-release mass MAE [kg]	0.0291	0.0093	240/240	4×10^{-41}
Convergence after lift-off [s]	4.21	4.22	—	—
Return to the empty value within the flight	183/240	240/240	—	—
Solve time, median [ms]	2.8	0.16	264/264	—

The reduced estimate was more accurate in every loaded and every post-release window. Convergence after lift-off was unchanged. After release, 57 of the 240 full-state estimates never re-entered the band within the flight. All 57 were at $4\,\mathrm{m\,s^{-1}}$, and they had not stalled: they settled 2.15 % below the empty value (median; interquartile range -2.21 % to -2.09 %) in the continuing figure-eight, which is outside the band. The reduced estimate re-entered it in every flight. The solve times are indicative only: the replay ran on an otherwise idle core

and the online figure is a window mean.

Under the thrust-map error of Section 5.7.4 the replay behaved as the analysis of Section 4.3.3 predicts. With a map 10 % high the reduced estimate was biased high by the same proportion (loaded MAE 0.207 kg, against 0.198 kg for F). With a map 10 % low it was driven onto the $0.95\,m_B$ bound: 107 of its solves, in 22 of the 24 flights, ended at the iteration limit, all at or near the bound. These 107 solves are 0.03 % of all replay solves, but they are the reason the deployed reduced estimator replaces the hard physical bound by a wide numerical box and an explicit health check.

The replay has three limitations. It cannot measure tracking, because the trajectories were generated with F in the loop. It uses the simulator's near-ideal odometry. And it does not reproduce the grasp gap of Section 4.3.4: it treats the logged solve instants as consecutive, so the windows that span the gap join samples taken about 6 s apart, and its lift-off convergence times are not a model of the online ones. It supports the reduced model as a mass estimator; it does not by itself justify putting it in the loop.

Thrust-map smoke test. Before the closed-loop comparison, R was flown once in each of four conditions — thrust map -10 % and 10 %, payload 0.15 kg and 0.30 kg, at $2\,\mathrm{m\,s^{-1}}$ with immediate detachment — with the pre-specified rule that any divergence, any model update during an unhealthy interval or any failed safety fallback would stop the comparison. None occurred. No flight recorded a non-zero solver status or a re-initialisation, so the iteration-limit failures of the replay did not recur with the wider box. In both -10 % flights the estimate was 10.2 % to 10.6 % low and was flagged below the health band whenever the airframe was empty; the controller then held its model, and its model mass did not change at all during any unhealthy interval. Both releases were confirmed, and the controller's model returned to within 0.01 kg of the empty value. With 10 % both releases ended in UNRESOLVED, the expected

outcome because the over-reading thrust map makes the empty airframe look loaded, and both flights braked to a safe hover. With the heavier payload the estimate left the band upward during lift-off and the controller held, for the whole carry, the last accepted value of 2.272 kg against a true 2.364 kg. That value is closer to the truth than the 2.60 kg the estimator itself reported, but it is whatever the estimate happened to be when it crossed the band edge during a transient: the check prevents an implausible model, it does not supply a correct one. With the lighter payload the 10 % estimate stayed inside the band and was used.

Paired closed-loop comparison. The deciding experiment flies F and R in the same conditions with everything else frozen: release arm D, the deployed controller, trajectory, gripper geometry, lifecycle and safety settings of Section 5.7, nominal thrust map, no injected fault. The design crosses payload (0.15 kg, 0.30 kg), speed ($2\,\mathrm{m\,s^{-1}}$, $4\,\mathrm{m\,s^{-1}}$) and detachment (immediate, 4 s after the command) with six paired replicates, $2 \times 2 \times 2 \times 6$ pairs and 96 flights. Within each of six blocks the eight cells are flown in random order with the two estimators of a pair adjacent, in random order. Only infrastructure failures are excluded; divergence, solver failure, early or missed confirmation and UNRESOLVED are outcomes. Metrics, statistics and the acceptance criteria were fixed before the first flight: paired differences R−F with bootstrap confidence intervals of the median and Wilcoxon signed-rank tests, by payload, speed and detachment as well as overall.

Execution. The batch was flown on 5 and 6 November 2026 from a recorded seed. Three deviations are recorded with it, none of which touches the comparison. After flight 12 the experiment entry points changed their default estimator; no flight reads that default, because every flight sets the variant explicitly, but the change altered the code fingerprint, so the batch stopped itself by design and was resumed under the new fingerprint. After flight 32, at a block bound-

ary, the batch was paused, and it was resumed the next day with the fingerprint unchanged. All 96 flights attached and were eligible, no infrastructure rerun was needed, and all 48 pairs are complete.

Binary outcomes. Neither arm diverged, confirmed a release early, missed one or entered UNRESOLVED; every release was confirmed and the controller's model returned to the empty airframe afterwards, 48 of 48 in each arm. F ended 14 solves at the iteration limit and re-initialised its window twice; R had no non-zero solver status. Under the nominal thrust map R's mass-health check never flagged an estimate. The controller held its model for 8.7 s with R and 9.1 s with F per flight at the median, almost all of it in the grasp gap of Section 4.3.4, and its model mass did not change during any hold.

Table 5.19: Full-state (F) against reduced (R) estimator in closed loop, pre-specified metrics, all 48 pairs. Medians per arm; Δ is the median paired difference R$-$F with its bootstrap 95 % confidence interval; p: Wilcoxon signed-rank; B/S/W: pairs in which R was better, the same or worse. Loaded window: from 5 s after lift-off to the command; post-release: from 5 s after detachment to the end of the flight.

Metric	F	R	Δ	95% CI	p	B/S/W
Loaded mass MAE [kg]	0.0347	0.0132	-0.0187	$[-0.0275, -0.0115]$	$< 10^{-12}$	46/0/2
Post-release mass MAE [kg]	0.0298	0.0095	-0.0201	$[-0.0303, -0.0096]$	$< 10^{-12}$	48/0/0
Convergence after lift-off [s]	4.21	4.17	-0.01	$[-0.06, +0.02]$	0.11	18/20/10
Loaded RMSE [m]	0.068	0.073	$+0.000$	$[-0.002, +0.004]$	0.29	20/6/22
Post-release RMSE [m]	0.069	0.064	$+0.001$	$[-0.004, +0.002]$	0.84	22/3/23
Post-release peak error [m]	0.148	0.138	-0.006	$[-0.011, -0.001]$	0.12	31/1/16
Confirmation after detachment [s]	2.20	2.16	-0.00	$[-0.05, +0.01]$	0.29	18/19/11
MHE solve time P50 [ms]	2.76	0.31	-2.46	$[-2.63, -2.13]$	$< 10^{-12}$	48/0/0
MHE solve time P95 [ms]	4.89	0.35	-4.57	$[-6.22, -3.29]$	$< 10^{-12}$	48/0/0

Table 5.19 gives the pre-specified continuous metrics. The mass advantage of R grows with speed: the loaded mass MAE fell from 0.0179 kg to 0.0120 kg at $2\,\mathrm{m\,s^{-1}}$ (R better in 22 of 24 pairs) and from 0.0535 kg to 0.0190 kg at $4\,\mathrm{m\,s^{-1}}$ (24 of 24), and the post-release mass MAE from 0.0167 kg to 0.0092 kg and from 0.0469 kg to 0.0136 kg, R better in every pair at both speeds. The advantage held for both payloads and both detachment conditions. Tracking showed no consistent difference in any group: the confidence interval of the loaded

RMSE difference contains zero in every group. The post-release peak error was smaller with R at $2\,\mathrm{m\,s^{-1}}$ (by 7 mm, 18 of 24 pairs, $p = 0.016$) and with the heavier payload (by 8.5 mm, 19 of 24, $p = 0.011$), not overall.

Acceptance. All pre-specified acceptance criteria were met: no divergence with R; no sustained solver failure, and fewer non-zero solver statuses than F; no more early confirmations, missed releases or UNRESOLVED outcomes than F; a loaded mass MAE lower with a confidence interval below zero, $p < 0.05$ and R better in at least two thirds of the pairs; no consistent worsening of loaded RMSE, post-release RMSE or post-release peak error; solve-time P50 and P95 lower with confidence intervals below zero; no iteration-limit runs and no re-initialisation under a $-10\,\%$ thrust map (smoke test above); and no model update while the estimate was unhealthy. The last criterion, safe hover after every UNRESOLVED, was not exercised, because no flight entered that state; it was exercised in 48 flights of the deployed-system validation and met (Section 5.3.2).

Exploratory metrics. Table 5.20 adds metrics that were *not* pre-specified, computed on the windows of Section 5.5.3 so that they can be read against the cohort; they are reported as exploratory.

The collapse is the clearest difference. At $4\,\mathrm{m\,s^{-1}}$ the full-state estimate spent a median $22\,\%$ of the loaded figure-eight below half the true payload, and touched its lower bound; the reduced estimate did neither in any of the 24 flights. This is the closed-loop counterpart of the argument of Section 3.4.2: the indirect path through the estimated attitude, which the reduced estimator does not have, is where the manoeuvre-induced bias came from. The mean vertical error in the figure-eight did not differ between the arms (26 of 48 pairs smaller with R, $p = 0.76$).

Table 5.20: Full-state (F) against reduced (R) estimator, exploratory metrics (not pre-specified), on the windows of Section 5.5.3. Notation as in Table 5.19; collapse fraction: share of loaded figure-eight samples with $\hat{m}_P$ below half the true payload; bound fraction: share of loaded samples within 2 g of the estimator's lower mass bound; post-release: 10–40 s after detachment. Pairs: all 48, or the 24 at $4\,\mathrm{m\,s^{-1}}$.

Metric	Pairs	F	R	Δ [95% CI]	B/S/W
Loaded-hover mass MAE [kg]	all	0.0055	0.0025	-0.0031 $[-0.0037, -0.0026]$	48/0/0
Loaded figure-eight mass MAE [kg]	all	0.0387	0.0141	-0.0209 $[-0.0325, -0.0130]$	46/0/2
	$4\,\mathrm{m\,s^{-1}}$	0.0614	0.0210	-0.0444 $[-0.0635, -0.0326]$	24/0/0
Collapse fraction	$4\,\mathrm{m\,s^{-1}}$	0.220	0.000	-0.220 $[-0.284, -0.024]$	24/0/0
Bound fraction	$4\,\mathrm{m\,s^{-1}}$	0.0055	0.000	-0.0055 $[-0.0083, -0.0021]$	17/7/0
Post-release mass MAE [kg]	all	0.0308	0.0093	-0.0192 $[-0.0308, -0.0099]$	48/0/0
MHE solve time P99 [ms]	all	6.40	0.38	-6.02 $[-8.10, -4.30]$	48/0/0
MHE solve time maximum [ms]	all	9.41	0.44	-9.02 $[-13.6, -6.0]$	48/0/0
NMPC solve time, median [ms]	all	1.75	1.70	$+0.00$ $[+0.00, +0.00]$	14/21/13

What the comparison supports. With the release rule, the controller, the residual specific force and the inertia floor held fixed, the reduced estimator identified the mass more accurately in hover, in manoeuvre and after release, removed the manoeuvre-induced collapse, solved more than an order of magnitude faster, and changed neither the safety of the lifecycle nor the confirmation delay. It did not improve tracking. That is consistent with Section 5.8.1: the residual specific force already compensates the translational consequence of a mass bias, and the held inertia floor its rotational consequence during carry, so a more accurate mass has little left to correct in the loop. The comparison is therefore evidence for the reduced estimator as an estimator — accuracy, health and computation — not for a tracking benefit.

Replay ablations and simple estimator baselines. Three questions about the reduced estimator can be answered on the recorded data of the 48 R flights without flying again: whether the reference point of the translational state matters (Section 3.2.2), whether the newest measurement y_N, which carries no cost in (4.7), should, and whether a simpler estimator on the same signals would do as well. All variants were run on the odometry, interval-averaged reconstructed thrust and 10 Hz ticks of each flight, with the event de-weighting triggered at

the recorded physical attach and detach. The replay of R itself reproduced the online estimate to within $7 \times 10^{-6}\,\mathrm{kg}$ in every flight. Like the replay of Table 5.18, it joins the samples across the grasp gap, so its convergence times after lift-off are shorter than the online ones and only differences between variants are meaningful. The variants are: *CoM*, R fed the composite-centre velocity of (3.9) with the true payload and arm; y_N, R with the velocity residual of the newest measurement added at the terminal node, at the weight of the other stages; *window*, the window force balance (4.14) re-evaluated at every tick; *low-pass*, the ratio of the low-passed vertical thrust to the low-passed $g + a_z$ (time constant 1 s); *RLS*, recursive least squares on $1/m_T$ in (3.7) with forgetting factor 0.95; and *EKF*, an extended Kalman filter on $[\boldsymbol{v}; m_T]$ with the model (4.3), a random-walk mass ($0.05\,\mathrm{kg\,s^{-1/2}}$), process noise $0.3\,\mathrm{m\,s^{-2}}$ and the estimator's velocity noise. The four baselines use exactly the inputs of R — reconstructed thrust and measured attitude — and differ from it only in how the mass is extracted. Their tuning was fixed once, before the replay, and not optimised.

Table 5.21: Replay ablations of the reduced estimator (R) and simple estimators on the same signals, 48 R flights of the paired comparison. Medians over flights; loaded and post-release windows as in Table 5.19; figure-eight: the 24 flights at $4\,\mathrm{m\,s^{-1}}$. Δt: median paired difference of the convergence time from R (B/S/W: variant faster, same, slower). Convergence: entry into $\pm 0.03\,\mathrm{kg}$ held for 1 s.

	Mass MAE [kg]			Δt to converge [s]	
Estimator	**Loaded**	**Figure-eight**	**Post-release**	**After lift-off**	**After detachment**
R (replay)	0.0132	0.0209	0.0095	—	—
R, CoM velocity	0.0132	0.0210	0.0095	0.00 (0/48/0)	0.00 (0/48/0)
R, with y_N	0.0132	0.0209	0.0095	−0.30 (48/0/0)	−0.20 (46/0/2)
Window balance	0.0137	0.0225	0.0102	+0.75 (0/0/48)	+0.50 (8/4/36)
Low-pass ratio	0.0141	0.0224	0.0101	+1.90 (0/0/48)	+0.30 (15/5/28)
RLS	0.0149	0.0208	0.0086	+3.95 (0/0/48)	+1.70 (3/0/45)
EKF	0.0132	0.0210	0.0096	+0.50 (0/0/48)	−0.40 (46/1/1)

Table 5.21 gives three answers. *The reference point does not matter here.* Replacing the origin velocity by the composite-centre velocity changed the figure-eight mass error by 0.05 g and the bias not at all (−0.88 against −0.88 %), so the transport term neglected in Section 3.2.2 is not the source of the residual manoeuvre-induced bias. *The newest measurement is worth adding.* With y_N in the

cost the accuracy was unchanged, but the estimate converged 0.30 s sooner after lift-off in every flight and 0.20 s sooner after detachment in 46 of 48: omitting y_N costs about one sample of delay, as the structure of (4.7) implies. The omission was an implementation convention, not a design choice. It does not affect release acceptance, which uses the force balance, but it delays the controller's model update by the same amount; the change is a candidate for the next revision of the deployed estimator and has not been flown. *A moving horizon is not what makes the estimate accurate.* An extended Kalman filter on the same model and inputs matched R to within 0.2 g in loaded flight and 0.1 g in the figure-eight; it converged 0.5 s later after lift-off and 0.4 s sooner after detachment, a trade set by its random-walk tuning. The algebraic window balance and the low-pass ratio were 0.5 g to 1.6 g less accurate and slower to converge, and RLS was as accurate in manoeuvre but several seconds slower. The accuracy gain over the full-state estimator of Tables 5.18 and 5.19 therefore comes from the model structure that all of these estimators share — reconstructed thrust as a known input, measured attitude, no rotational channel — and not from the moving-horizon solution. What a moving horizon adds in this design — the explicit window that the event de-weighting acts on, and the numerical box with the health check that the $-10\,\%$ thrust-map replay and smoke test exercised — has not been compared with an equivalent mechanism in a filter, and these baselines were not run on the thrust-map flights or in closed loop. They are open-loop replays on trajectories flown with R.

5.8.3 The horizontal-offset decision

The decision to hold the controller's horizontal centre-of-mass offset at zero (Section 3.3) rests on a control-side ablation performed with eccentric grasps, before the offset was removed from the model. The cohort used 0.15 kg, a nominal 0.10 m grasp offset and the $r = 10\,\mathrm{m}$, $\omega = 0.283\,\mathrm{rad/s}$ figure-eight, with four time-paired trials per arm in alternating order. The ablated arm set the

controller's offset parameters to zero, removing both the predicted thrust-offset moment and its trim-cost reference; the estimator and every other part of the controller were identical in both arms. All eight attempts physically attached and completed, with no solver failures.

Mean loaded figure-eight position RMS was 0.1230 m with compensation and 0.1213 m without it (-1.4%); the mean per-flight peak was 0.2760 m versus 0.2705 m, and lift RMS 0.4032 m versus 0.4039 m. Even at a grasp offset three times the attachment tolerance of the present configuration, modelling the offset in the controller bought nothing measurable. This is consistent with an analysis of flight logs from an earlier eccentric-load configuration, which found that the autopilot's rate integrator, not the predictive controller's torque, carried about 99 % of the trim moment even when the controller modelled the offset explicitly. This small, single-condition cohort is not an equivalence demonstration, and it does not address offsets large enough to consume most of the torque authority; the present configuration avoids that regime by aligning before grasp. The frozen logs, configurations and scoring scripts are in `../data/ecc_ablation_20260924_202511/`.

5.9 Failure Modes and Operating Limits

5.9.1 Manoeuvre-induced mass collapse: development evidence

The dominant failure of the mass-only interface is that the loaded mass estimate collapses toward the bare airframe during the figure-eight. Its size in the evaluated configuration is measured on the frozen cohort in Section 5.6.1. The evidence below is of a lower standard and serves a different purpose: it shows the mechanism and the consequences of the collapse. It comes from development flights at 0.30 kg on a figure-eight of radius 5 m at $w = 0.283\,\mathrm{rad\,s^{-1}}$, predates both the final cohort and the lagged default — every flight below ran

with the payload inertia omitted from the estimator — and mixes configuration changes noted where they occur. It is not pooled with the cohort.

Table 5.22 condenses the three development batches.

Table 5.22: Development evidence for the mass-collapse mechanism. Configuration changes are cumulative and the first two changes in the second row are confounded.

Configuration	Attempts	Completed	Main observation
Early baseline	8	2	two missed-grasp failures; four loaded flights lost roll control after $\hat{m}$ and the inner-loop scale collapsed
Relative alignment + yaw smoothing	8	7	survival improved ($p = 0.041$), but collapse persisted and one flight failed during carry
Shortened arm, with both changes above	8	8	collapse fraction fell from 0.05–0.90 to 0.03–0.24 ($p = 0.028$), but was not eliminated

Across the sixteen flights used for the arm comparison, collapse fraction rose with measured arm length (Spearman $\rho = 0.66,\ p = 0.005$), implicating the omitted inertia without making it the sole cause. Separate high-speed development flights recorded seven and nine carry-phase acceptances; holding the inertia floor preserved control authority but did not restore the mass estimate or the rule's discriminating power. The frozen cohort measures the current frequency of these failures in Sections 5.6.1 and 5.6.4.

These observations bound what the mass-only interface can claim. Its release confirmation is sound only while the mass estimate is: where the estimate collapses under manoeuvre, a confirmation rule that admits no other evidence will confirm a release that has not happened. The remedies available within the present architecture are the inertia-floor hold and the residual specific force, both in the evaluated configuration, which address two of its consequences (loss of inner-loop authority and loss of thrust feed-forward) without touching its cause; and the command-armed arm C, which removes the false confirmation by not running the rule before the command.

5.9.2 Grasp alignment

The centred-grasp assumption of Section 3.3 depends on the vehicle actually being over the parcel when the grasp is enabled. In development flights the autopilot's local position estimate drifted 2 cm to 4 cm per flight relative to the simulator's world frame, while the controller believed its horizontal error to be under 2 cm; with the 0.03 m attachment tolerance, roughly one flight in four or five missed the grasp. Tuning the controller cannot fix an error in the position it is given. An alignment step relative to the parcel, emulating a downward-looking camera, aligns the vehicle on the parcel's relative position to within 0.012 m, waiting at most 4 s, before the grasp is enabled; it is part of the evaluated configuration. The same drift acts vertically on the grasp height, which is why the realised arm varies from flight to flight (Section 3.6).

5.9.3 The operating limit

0.30 kg at $4\,\mathrm{m\,s^{-1}}$ is the stress condition. Hover requires 74 % of maximum thrust. At the shortened arm the inertia ratio (3.16) is 2.34 nominally and at most about 2.7 over the realised arm range, well inside the body-rate scaling cap of 5.0; with the earlier 0.47 m arm it was 5.07, above the cap, and the inner loop could not be given the authority the load implied. The arm was shortened for that reason, so what remains at this condition is a thrust and torque margin, not an inner-loop one. This is the quantitative form of the feasibility boundary predicted in Chapter 3.

The cohort confirms that the remaining margin was not exhausted. The controller logs the inner-loop scale and the share of torque commands at their bounds over consecutive 5 s windows. In loaded figure-eight the inner-loop scale had a median of 2.34 and its window minimum never fell below 2.34 in any physically attached flight: the inertia floor, held until the command, kept the

inner loop at the envelope scale however far the estimate dipped. Roll torque was never at its bound; pitch was at its bound in at most 0.3 % of samples on average; yaw was at its bound in about 2.5 % of samples at $4\,\mathrm{m\,s^{-1}}$ and never at $2\,\mathrm{m\,s^{-1}}$. The controller's solver reported at most one failure in any flight. These are window-level diagnostics, not per-sample statistics, and they say nothing about margin under disturbances the cohort did not include.

5.9.4 Estimator health

In every physically attached cohort flight the published mass left the empty value and settled within 0.44 % of truth in loaded hover (Table 5.9). The one flight whose estimate stayed at the empty value is the missed grasp of Table 5.8, where that was the correct answer: the parcel was 0.47 m below the gripper, outside the 0.45 m attachment window, and the 90th percentile of the estimated payload was 4 g. That flight is also the cohort's only instance of the bootstrap's known failure (Section 4.6.3): the empty airframe flew the $4\,\mathrm{m\,s^{-1}}$ figure-eight with the envelope inertia floor, at a 2.34-fold inner-loop scale, and did not diverge. Estimator health at the moment of the release command was not scored separately.

One structural gap is known independently of the cohort: the phase-timed release command is *not* gated on estimator health. A release issued while the estimate is frozen leaves a model that no longer matches the vehicle after detachment. Gating release on estimator health is an obvious remedy and is not implemented.

5.9.5 Configuration and default-safety

Two code bases are involved, and a statement about a default must name one. Every historical flight reported in this chapter, including the paired F/R com-

parison, was run from the *campaign code base*, in which the configuration is selected by switches. The *mainline code base*, extracted from it on 5 November 2026, contains the deployed system only; the deployed-system validation of Section 5.3 was run from it.

In the campaign code base the estimator model and the evidence form are selected by environment switches whose library defaults differ from those of the experiment entry points, so a node started without an entry point does not run the evaluated configuration. The cohort and the campaign ran the full-state model; the cohort driver set the evaluated configuration explicitly (below).

The mainline code base removes that hazard by removing the switches. Its estimator node implements the reduced estimator only. Its controller node has no setting for the release rule, the controller type, offset compensation or the inertia-floor hand-over, and its parameter defaults are the deployed values, so a node started without any argument runs the deployed system; the launch script passes only the experiment variables (payload, speed, detachment delay, thrust-map gain, injected loss and jam). Whether the extraction preserved the controller's behaviour was checked rather than assumed (Section 5.10.4). A configuration that does not match the one described here must be detectable from the run record alone, which is why every run records the resolved value of every behaviour-affecting switch (Appendix B).

The cohort driver sets the five non-default settings of Section 5.2 explicitly for every flight, so the cohort does not depend on the entry-point defaults, which for those settings remain off. A flight launched from an entry point without the cohort driver therefore does *not* run the evaluated configuration. The 14-state selection is both an entry-point default and set explicitly by the cohort driver, and each flight's record marks it as explicitly set.

5.10 Threats to Validity, Missing Baselines and Reproducibility

All system-level evidence is software-in-the-loop. The leading threats are:

- **Thrust-map fidelity.** The plant generates thrust with the same square law the estimator inverts, and rotor speeds are exact. Neither a mis-calibrated thrust coefficient (Section 5.10.1) nor speed noise, transport delay or induced inflow [31] was tested in the evaluated configuration.
- **Clean state estimates.** The odometry available in simulation is better than a real vehicle's, and the measurement weights (5.7) are set from representative rather than measured noise levels. The horizontal drift of the local position estimate (Section 5.9.2) is the one deviation from clean odometry that has already been seen to matter.
- **Rigid attachment and centred grasp.** Gripper compliance and pendulum modes are not modelled, and every reported grasp is within the attachment tolerance of the thrust axis. Larger offsets are outside the claim. The realised vertical arm differs from the nominal prior by up to about 10 % per flight and is not measured per flight.
- **Single gripper geometry.** Every cohort flight uses the shortened arm. The inertia ratio grows with the square of the arm, and the earlier 0.47 m geometry, at which the heavier payload exceeds the inner-loop scaling cap, is outside the claim.
- **Confounded disturbances.** A sustained vertical updraft or downdraft is intrinsically indistinguishable from a payload change on a collective-thrust-only residual, and the mass-only interface has no second channel to separate them. Wind is not evaluated.

5.10.1 Thrust-map calibration: an analytical sensitivity

The mass channel recovers m_T from an *absolute* thrust, so it inherits the calibration error of k_f directly — nothing in the formulation cancels it. The relation is analytical: a relative error ε in the estimator's thrust coefficient scales the reconstructed thrust by $1+\varepsilon$, and through (3.7) the identified mass by the same factor. At $\varepsilon = \pm 5\%$ and the 2.2 kg composite mass of the lighter condition this is ± 0.11 kg — three-quarters of the 0.15 kg payload, and between four and five times the 0.024 kg confirmation threshold. The arithmetic predicts two failures: a coefficient reading low may leave the payload unrecognised, and one reading high may leave a phantom payload after release, so that confirmation never arrives. The direction matters: the second degrades toward UNRESOLVED, which is safe, the first toward an unrecognised payload, which is not. The force-balance detector of Section 4.7.5 has the same one-for-one sensitivity. A $\pm 10\,\%$ error was flown in the thrust shared by the estimator and the detector (Section 5.7.4), and both predicted failures occurred. With the map high, the release was missed and the flight ended in UNRESOLVED, and the controller's model mass in carry was 7–9 % heavy. With the map low, the lighter payload read as absent in the detector, which silently disabled the uncommanded-loss watchdog, and the controller's model held the bare-airframe mass with either payload. The deployed system, flown with the same errors (Section 5.3), failed in the same two directions, and under the under-reading map with delayed detachment it accepted every release with the lighter payload about 3.1 s before the payload left. The designed $\pm 5\,\%$ level has not been flown.

5.10.2 Experiments designed but not run

Four experiments were designed for this dissertation but not run, or run only in part. They are listed so that their absence is explicit; no result is claimed for the parts not run.

- **Small thrust-map mismatch** (k_f scaled by 0.95 and 1.05). Its expected effect is the analytical sensitivity of Section 5.10.1. It is defined in the batch driver of the mainline code base but was not flown; only the $\pm 10\,\%$ level was, in the estimator and the detector together (Sections 5.3 and 5.7.4).

- **Thrust-map error with delayed detachment** (gains 0.90 and 1.10, all four conditions, detachment held 4 s). Only gain 0.90 with the lighter payload was flown, in the deployed-system validation, where it produced early acceptance in every flight (Section 5.3); the heavier payload and the over-reading map were not flown with delayed detachment.

- **Partial payload loss** (four 0.05 kg boxes, three removed one at a time without a signal, against a no-loss control arm). This is the case a binary attached/detached abstraction cannot represent at all, and the one where a continuous interface would be expected to differ in kind.

- **Window-seed comparison** (the window force balance (4.14) against the instantaneous seed (4.11) at $4\,\mathrm{m\,s^{-1}}$). The only evidence for the window seed is the development observation of Section 4.3.5.

Unsignaled loss, the fourth scenario outside the cohort, was flown under the force-balance rule, in the historical release campaign (Section 5.7.3) and with the deployed system (Section 5.3); it has not been flown under the mass-evidence rules, for the reason given in Section 5.4.2. The open experiments are taken up in Chapter 6.

5.10.3 Missing baselines

The release arms of Table 5.6 and arm D isolate release semantics. A fixed-parameter predictive controller and an $\mathscr{L}_1$-type controller were compared with the MHE controller in the release campaign (Section 5.7.5), under one lifecycle

and with the inertia-floor and inner-loop scheduling shared by all three. Because the $\mathscr{L}_1$ arm uses the same residual law as the evaluated configuration, that comparison isolates the payload model rather than the presence of adaptation; it found no consistent tracking benefit in carry. The evaluation still does *not* include incremental nonlinear dynamic inversion [19], the full $\mathscr{L}_1$-adaptive NMPC of [18] with its rotational channel, or a change-detection procedure such as CUSUM [29] applied to the same signals under a matched false-alarm constraint. The last is the largest remaining gap for the release question: the force-balance rule is compared with the mass-evidence rules and with command-triggered acceptance, not with an optimised detector.

5.10.4 Reproducibility

Reproducibility is handled by construction rather than by discipline. Every run records its code identity, the resolved value of every environment switch that affects behaviour, the problem dimensions, horizons, weights, bounds, event timestamps, random seed and solver statistics. Runs terminated by numerical failure remain in the denominator. The evidence-score function used online and the one used in offline replay are the same function, so a recorded flight can be re-scored without re-implementing the logic.

The code identity is recorded as a repository revision *plus* a frozen patch of uncommitted changes and an archive of untracked files, not as a single commit. The cohort was not run from a clean revision: the working tree contained uncommitted edits to this dissertation's sources, which do not affect flight code, and the autopilot source tree contained modifications belonging to an unrelated project. Each flight's record accordingly marks an exact-commit claim as prohibited. Every flight can nevertheless be rebuilt exactly from the recorded revision, patch and archive, and that is the claim made here; no cohort result should be cited as having been produced by a particular commit.

The flight code of every reported batch also carried three further controller parameters, the slot of a rotational lumped-disturbance channel that had been tried in development and rejected; they were held at zero in every flight. They were removed from the code after the campaign. On twelve fixed test problems, including two that end at the iteration limit, the controller's solution with and without them was bitwise identical, iteration counts included, so the removal changes the description of the controller in Section 4.2.2 and not its behaviour.

The mainline code base was produced from the campaign code base by removing every controller branch that the deployed configuration never executes, and by making the deployed parameter values the defaults; the controller node shrank from 3712 to 1707 lines. Because the controller runs in closed loop, there is no recorded input stream to replay through it, so equivalence was tested with an offline harness: both versions of the controller node ran in-process against simulated clocks, publishers, subscriptions, timers and services, closed around one deterministic synthetic plant, through a complete mission from approach to confirmed release. In seven scenarios — nominal release, detachment delayed by 4 s, uncommanded loss, a jammed gripper reaching UNRESOLVED, a thrust-map gain of 0.9, $4\,\mathrm{m\,s^{-1}}$ and 0.15 kg — all 19556 published messages and all 5597 run-time log lines of the two versions were bitwise identical, both when the mainline node was given the campaign's full parameter list and when it was given only the experiment variables. One software-in-the-loop flight of the mainline system at 0.30 kg and $2\,\mathrm{m\,s^{-1}}$ completed the mission, with the release confirmed 2.4 s after the command and no controller solver failure. This is a regression check of the extraction, not an evaluation result.

The deployed-system validation of Section 5.3, unlike the historical batches, was run from a tagged commit with no uncommitted change to the flight code, which the fingerprint recorded before every flight confirms (Appendix A.1). It is reproduced from the mainline repository by

```
cd /home/clear/MHE_NMPC && git checkout deployed-r-armd-180-v1 \
  && colcon build && i=1; \
```

```
for e in release faults thrust thrust_low_d4; do \
  EXP=$e SEED=2026110600$i bash scripts/run_batch.sh; \
  i=$((i+1)); \
done && python3 scripts/tools/campaign_eval.py
```

which flies the four segments with the seeds of Table 5.2 (`20261106001` to `20261106004`), so that the schedules are regenerated exactly, and then scores them with the analysis script `scripts/tools/campaign_eval.py`. The schedules are reproduced exactly; the flights themselves are reproduced only up to the run-to-run variation of Section 5.5.4. The analysis script reads the batch directories only, and the acceptance criteria are kept with the results in `results/campaign_180_analysis/acceptance_criteria.md`.

5.11 Chapter Summary

This chapter addressed the fifth research objective. Its primary evidence is a validation of the deployed system — the reduced estimator with the force-balance release rule — in 180 software-in-the-loop flights from a tagged commit of the mainline code base (Section 5.3). The four acceptance criteria, inherited from the pre-specified F/R comparison and fixed after 11 of the 180 flights, were all met: no divergence and no estimator re-initialisation in 72 nominal releases, no change of the controller's model mass while the estimate was unhealthy in any of the 180 flights, and a safe hover after each of the 48 UNRESOLVED outcomes. Every nominal release was confirmed, none early and none during carry, 2.17 s after physical detachment at the median (1.86 s to 2.52 s), with a loaded mass error of 0.013 kg and no collapse of the estimate in manoeuvre; 24 of 24 uncommanded losses were detected and none of 24 jammed releases accepted. The estimator solved in 0.31 ms at the median and never in more than 2.3 ms. The thrust map bounds the system: with it 10 % high every release was missed and ended in a safe hover; with it 10 % low the lighter

payload read as absent, so the loss watchdog never armed, and under delayed detachment the release was accepted in all 12 flights about 3.1 s before the payload left. The rule is therefore correct only while the thrust map is accurate to better than the payload's share of the composite mass.

The historical sources, flown with the full-state estimator, explain how the design was reached. In the frozen cohort of 120 flights the mass was accurate in loaded hover and collapsed periodically in sustained manoeuvre at $4\,\mathrm{m\,s^{-1}}$ (Q1); every one of the 118 eligible releases was accepted, 2.3 s to 3.2 s after physical detachment (Q2); under delayed detachment the evidence-gated arms never accepted a release while the payload was attached, whereas the command-triggered arm always did (Q3); the always-on confirmation on the mass evidence accepted 114 releases that had not happened during high-speed carry, and the command-armed confirmation none, at no cost in delay (Q4); and the release arm did not change tracking, while the residual specific force improved it without changing the mass bias (Q5). The historical campaign of 312 flights introduced the force-balance rule, which removed the carry-phase false acceptances, and showed that adaptation clearly beat a fixed-parameter controller in carry while estimating the mass gave no consistent tracking benefit over the residual alone. The paired closed-loop comparison of 96 flights then chose the reduced estimator: it met every pre-specified acceptance criterion, lowered the loaded mass error from 0.035 kg to 0.013 kg, removed the manoeuvre-induced collapse and solved about ten times faster, with neither tracking nor the lifecycle changed (Section 5.8.2). Replayed on the same flights, an extended Kalman filter with the reduced estimator's inputs was as accurate, so the gain belongs to the model structure rather than to the moving horizon; penalising the newest measurement would shorten convergence by 0.2 s to 0.3 s; and correcting the velocity to the composite centre of mass changed nothing. These replays are open-loop analyses of recorded flights, not closed-loop results. No actuation-side thrust-map, small thrust-map, partial-loss or hardware result exists. Chapter 6 draws the conclusions these results support, and only those.

Chapter 6

Conclusion and Future Work

6.1 Summary

This dissertation treated payload grasp and release on a quadrotor as one estimation-and-control problem. Chapter 3 reduced a centred rigid payload to a single online parameter, the composite mass, from which the roll and pitch inertia increment follows, and assigned the identification of that mass to the translational channel. Chapter 4 built a moving-horizon estimator driven by a wrench reconstructed from rotor speeds, a predictive controller updated continuously from one health-gated estimate, and a release lifecycle in which acceptance of a release is gated on evidence — in the deployed form an independent force balance, in the historical cohort the estimator's mass evidence. Chapter 5 evaluated the deployed system — the reduced estimator with the force-balance rule — in a validation campaign of 180 PX4/Gazebo software-in-the-loop flights, and documented how the design was reached with historical batches flown with a full-state estimator: a frozen cohort of 120 flight attempts, a smaller frozen ablation, a release campaign of 312 flights and a paired estimator comparison of 96. The findings below answer the five research objectives against that evidence, and say which batch each statement rests on.

6.2 Principal Findings

Objective 1 — a single-parameter payload structure.

- **Answer.** For a rigid payload on the thrust axis, the roll and pitch inertia increment is μr_z^2, a function of the composite mass and a nominal arm, so the composite mass is the only payload quantity that needs online identification; the inertia follows from it and vanishes with it.
- **Evidence.** The derivation of Section 3.3; the structure is implemented in both optimisers and used in every reported flight.
- **Boundary.** A centred grasp within the alignment tolerance, a point-mass payload, and a nominal arm whose realised value varies by up to -14 to $+23$ percent in inertia (Section 3.6).
- **Not established.** Eccentric grasps; and whether estimating the inertia directly would perform better or worse, which was not compared.

Objective 2 — identifying the mass.

- **Answer.** The estimator identifies the composite mass accurately in loaded hover, but under sustained manoeuvre the estimate is biased low and periodically collapses toward the bare airframe. Both statements describe the full-state estimator with which the campaigns were flown. The deployed reduced estimator, which does not integrate the rotational dynamics, removes most of that bias and the collapse, in replay and in closed loop, at about a tenth of the computation. The improvement belongs to its model structure — reconstructed thrust and measured attitude as known inputs, no rotational channel — rather than to the moving horizon: replayed on the same flights, an extended Kalman filter with that structure was as accurate.

- **Evidence.** In the deployed-system validation the reduced estimator's loaded mass error was 0.013 kg at the median over 72 nominal releases (2.7 g in loaded hover, a figure-eight bias of $-0.66\,\%$ at $4\,\mathrm{m\,s^{-1}}$), its estimate never collapsed in any flight, it never re-initialised, and it solved in 0.31 ms at the median (Section 5.3). In the historical frozen cohort the per-flight loaded-hover error lay between $-0.44\,\%$ and $-0.10\,\%$; the loaded figure-eight bias had medians of $-1.0\,\%$ at $2\,\mathrm{m\,s^{-1}}$ and $-4.1\,\%$ and $-3.3\,\%$ at $4\,\mathrm{m\,s^{-1}}$, with dips to the mass lower bound and a median collapse fraction of 0.38 for the lighter payload (Section 5.6.1). Replayed on the 312 campaign flights, the reduced estimator lowered the median loaded mass error from 0.036 kg to 0.013 kg, smaller in every one of the 264 nominal flights, and returned to the empty value after every release, where the full-state estimate stayed about $2\,\%$ low in 57 flights at $4\,\mathrm{m\,s^{-1}}$. In closed loop, over 48 pairs flown with the same controller and release rule, the reduced estimator lowered the loaded mass error from 0.035 kg to 0.013 kg (better in 46 pairs) and the post-release error from 0.030 kg to 0.010 kg (better in all 48); at $4\,\mathrm{m\,s^{-1}}$ the full-state estimate spent a median $22\,\%$ of the loaded figure-eight below half the payload and the reduced estimate none, in any flight. The median solve time fell from 2.8 ms to 0.3 ms. Tracking and the lifecycle did not change. In replay on the 48 reduced-estimator flights, the extended Kalman filter came within 0.2 g of the reduced estimator's loaded mass error, converging 0.5 s later after lift-off and 0.4 s sooner after detachment; the window force balance was 0.5 g to 1.6 g less accurate. Penalising the newest measurement, which the flown estimator leaves without cost, shortened its convergence by 0.2 s to 0.3 s at unchanged accuracy, and replacing the body-origin velocity by the composite-centre velocity changed nothing (Section 5.8.2).
- **Boundary.** Simulated rotor speeds are exact and the plant uses the thrust law the estimator inverts; the shortened arm; payloads of 0.15 kg to 0.30 kg.

- **Not established.** The full cause of the collapse; the reduced estimator under realistic attitude noise, which the simulator's odometry does not exercise; the effect of a thrust-coefficient error on the actuation side, and of an error smaller than 10 % (Section 5.10.1); whether the moving horizon is needed at all, since the filter baseline was compared only in open-loop replay with nominal thrust; and the advantage of the window seed, whose comparison was not run.

Objective 3 — a continuously updated controller.

- **Answer.** The controller completed every eligible flight without divergence while consuming the estimate continuously. The residual specific force improves tracking but does not improve the mass estimate or change the lifecycle. The held inertia floor protects the inner loop through the collapse, but for that reason the cohort cannot show the effect of estimate-driven inner-loop scheduling on its own.
- **Evidence.** None of the 180 flights of the deployed-system validation diverged or logged a controller solve failure, including the jammed, lost-load and thrust-map flights (Section 5.3). No eligible cohort flight diverged (Table 5.8). In the 48-flight ablation the residual reduced loaded figure-eight RMSE by 22 % to 31 % and removed a mean vertical sag of up to 44 mm, with no difference in mass bias, confirmation delay or carry-phase confirmations (Section 5.8.1). The inner-loop scale stayed at the floor value of 2.34 through every loaded figure-eight (Section 5.9.3). In the release campaign, with the lifecycle and inner loop shared, the MHE controller had a smaller loaded RMSE than a fixed-parameter controller in all 24 pairs, by 0.04 m to 0.14 m at the median, and no consistent difference in carry from an $\mathscr{L}_1$ controller that uses the residual alone (Section 5.7.5).
- **Boundary.** The inertia ratio at the shortened arm stays below the scaling

cap; with the earlier, longer arm it would not.

- **Not established.** A tracking benefit of estimating the mass over the lumped residual alone — the only consistent difference was a smaller post-release peak at $2\,\mathrm{m\,s^{-1}}$, of 6 mm to 17 mm; any benefit over incremental dynamic inversion or full $\mathscr{L}_1$-adaptive NMPC, which were not run; and the contribution of the continuous inner-loop scaling, which the held floor masked during carry and which all three controllers of the comparison shared.

Objective 4 — separating a release command from a physical detachment.

- **Answer.** Gating release acceptance on evidence separates "the command was issued" from "the payload has left". On the mass evidence, only consulting it after the command — *command-armed, estimator-confirmed detachment* — is sound; consulting it throughout carry is not, because the mass evidence fails in manoeuvre. With independent force-balance evidence, which does not pass through the mass estimate, the rule can in addition be enabled by a watchdog armed on airborne load: it then also detects a load lost without a command, without false acceptances in carry. That rule's weakness is the thrust map, not the manoeuvre: when the map under-reads by more than the payload's share of the composite mass, a loaded vehicle reads as empty, and the rule then accepts a delayed release before the payload has left.

- **Evidence.** In the deployed-system validation the force-balance rule confirmed 72 of 72 nominal releases, none early, none during carry, 2.17 s after physical detachment at the median; it detected 24 of 24 uncommanded losses, 2.15 s after detachment, and accepted none of 24 jammed releases, each of which ended in a braked hover. With a 10 % over-reading thrust map it missed all 24 releases and hovered safely. With a 10 % under-reading map the 0.15 kg payload, 6.8 % of the composite

mass, read as absent: the loss watchdog never armed, and with detachment held 4 s the release was accepted in 12 of 12 flights, 3.1 s before the payload left (Section 5.3). In the historical cohort every eligible release was accepted, 2.3 s to 3.2 s after physical detachment. Under delayed detachment the command-triggered arm accepted early in 24 of 24 flights, about 4 s early, and the evidence-gated arms in none of 47. The always-on rule accepted 114 releases that had not happened, in 20 of 32 flights at $4\,\mathrm{m\,s^{-1}}$; the command-armed rule accepted none in its 23 flights and confirmed real detachments no later (Sections 5.6.2–5.6.4). In the historical release campaign the force-balance rule accepted no release during carry in any of 240 flights (228 with the watchdog armed), none early in 36 delayed-detachment flights, against 36 of 36 for command-triggered acceptance, and confirmed real detachments 1.9 s to 2.5 s after they occurred. It detected 24 of 24 uncommanded losses 1.8 s to 2.5 s after detachment, resetting the payload geometry and inner-loop scale each time, and accepted none of 24 releases with a jammed gripper, all of which ended in a braked hover (Section 5.7).

- **Boundary.** For the command-armed rule, a release command must exist. For both rules, the detachment delay tested was 4 s. The force-balance rule assumes a thrust map accurate to better than 10 % and, more precisely, accurate to better than the payload's share of the composite mass: overestimated by 10 % it missed every release, safely; underestimated by as much it read the lighter payload as absent, so that the loss watchdog never armed and a delayed release was accepted early (Sections 5.3 and 5.7.4). For the deployed system this is a calibration requirement on k_f, not a tuning choice.

- **Not established.** A tracking or survival advantage of evidence-gated acceptance — none was detected, and none was expected, since the model's mass follows the estimate in every arm; what the delayed-detachment experiment shows is lifecycle correctness. Also not established: uncom-

manded loss under the mass-evidence rules, which was not run; the force-balance rule under delayed detachment with an over-reading map or with the heavier payload, and under a thrust-map error smaller than 10 %; detection of partial loss; and its performance relative to an optimised change detector.

Objective 5 — evaluation and its scope.

- **Answer.** The deployed system was validated in a campaign of its own, from a tagged commit, with randomised rounds, complete reporting and acceptance criteria inherited from an earlier pre-specification; the historical configuration was evaluated with randomised blocks, complete reporting of every attempt, targeted ablations and an explicit account of its failure modes.
- **Evidence.** The validation: 180 flights, all eligible, none rerun, the four inherited acceptance criteria met; the criteria were fixed during the run, after 11 flights, not before it (Section 5.3.2). The cohort: 120 attempts, 118 eligible, two exclusions both explained (Table 5.8); the historical campaign: 312 attempts, all eligible, with no replacement flight needed (Section 5.7.1); the evidence sources and their standing are listed in Table 5.7.
- **Boundary.** All system-level evidence is software-in-the-loop.
- **Not established.** Hardware validity; a comparison with incremental dynamic inversion or with a change detector under a matched false-alarm constraint; and the designed small thrust-map, partial-loss and window-seed experiments, which were not run, and the parts of the thrust-map and delayed-detachment crossing that were not flown (Section 5.10.2).

6.3 Limitations

The limitations that bound the findings are collected here once. All system-level evidence is software-in-the-loop, with exact rotor speeds and a plant that generates thrust by the same law the estimator inverts. The attachment is rigid and the grasp centred; eccentric grasps, gripper compliance, pendulum modes and flexible loads are not modelled, and a sustained vertical gust is intrinsically confounded with a payload change on a collective-thrust residual. Every flight uses one gripper geometry, whose nominal arm differs from the realised one by an unmeasured amount. The attach-time inertia floor is armed whether or not the grasp succeeds. Under the mass-evidence rules it is ended only by the release command, so a load lost without a command leaves it in place; under the force-balance rule a detected loss ends it, but detection depends on the thrust map. The release command is not gated on estimator health. The evaluated configuration includes settings that are not the defaults of the campaign code base's experiment entry points, and the results are claims about that configuration; in the mainline code base those settings are the defaults, and the deployed system differs from the evaluated configuration only in its release rule and its estimator (Section 5.2). The deployed system was validated in 180 flights, which cover nominal and delayed release, uncommanded loss, a jam and a $\pm 10\,\%$ estimation-side thrust-map error; its acceptance criteria were fixed during that run, not before it. Its decisive limit is the thrust map: under a $10\,\%$ under-reading map it cannot tell the lighter payload from none, so the loss watchdog does not arm and a delayed release is accepted about 3 s before the payload leaves (Section 5.3).

Three further limits concern what the evidence can show. The force-balance detector was validated in a plant whose thrust follows exactly the static law it inverts, with near-ideal rotor speeds and an inertial unit at the body origin; rotor-speed noise and latency, accelerometer bias and vibration, the lever arm between the inertial unit and the composite centre of mass, induced inflow and

an actuation-side thrust error were not present, and the thrust-map error was applied to the estimation side only. Repeated flights of one cell differ through unseeded sensor noise and scheduling, not through randomised physical conditions, so the statistical tests describe repeatability in simulation, not inference to a fielded vehicle (Section 5.5.4). And the release rules were compared with each other, never with an optimised change detector at a matched false-alarm rate; that is the largest gap for the release question. The controller's torque bounds are a surrogate, not the realisable input set, and the torque is a virtual input that the autopilot never receives (Section 4.4.4).

6.4 Future Work

The following directions are ranked by how much they would change what can be claimed.

1. **Release evidence that knows its own calibration.** The validation showed that the deployed rule accepts a delayed release early when the thrust map under-reads by more than the payload's share of the composite mass. A rule that combines m_{acc} with an explicit bound on the thrust-map uncertainty — refusing to treat a vehicle as empty, or reporting that it cannot tell, when the expected payload is smaller than that bound, and setting the thresholds of (4.36) from the calibrated uncertainty rather than as absolute masses — should be designed and tested against the crossing of thrust-map error and delayed detachment, of which only one corner was flown. Evidence insensitive to a constant coefficient error, such as the relative step in m_{acc} against the value recorded after lift, or an online calibration of k_f in empty hover, belongs to the same test. None of these has been implemented or validated.

2. **The estimator the deployed system needs.** The extended Kalman filter that

matched the reduced estimator in replay should be flown in closed loop against it, under nominal and erroneous thrust maps, to settle whether a moving horizon is needed; the same batch can add the terminal measurement to the estimator and replace the controller's torque box by the four per-rotor force limits.

3. **Manoeuvre-robust mass estimation.** The reduced estimator removed the collapse that dominated the historical configuration, but a manoeuvre bias of about $-0.7\,\%$ remains at $4\,\mathrm{m\,s^{-1}}$, and the reference-point correction does not explain it. Calibrating the airframe's rotational model — inertia, rotor-drag moment and torque bias — would reduce the error that reaches the mass through the attitude, and might eventually let the rotational channel contribute information instead of error. The designed but unrun partial-loss and window-seed experiments belong here: both test the estimator rather than the lifecycle.

4. **Health-gated release.** The release command should not be issued while the estimator is unhealthy, which the deployed system does not enforce.

5. **Change-detection and rotational baselines.** A comparison against CUSUM or a related procedure on the same signals, under a matched false-alarm constraint, would show whether persistence on a filtered score is near the achievable detection performance. Since the mass estimate gave no consistent tracking benefit over the lumped residual in carry, its value should next be tested where the residual cannot substitute for it — in the rotational channel, with the inner-loop scaling no longer shared between the arms.

6. **Thrust-map robustness and hardware transfer.** The one-for-one sensitivity to the thrust coefficient was measured at $\pm 10\,\%$, for the release detector and the estimator together. It should next be measured at the designed $\pm 5\,\%$, with the error also applied to the actuation side, and then against motor-stand calibration of k_f and k_m. Transfer should proceed through

tethered hover with the estimator running open loop, centred pickup, and only then dynamic flight, with electronic-speed-controller telemetry replacing simulated rotor speeds and a supervisor gating every stage.

6.5 Closing Remark

The result that generalises beyond this vehicle concerns the relation between an estimate and a decision. A continuously estimated mass can drive the model of a predictive controller without any event, and doing so kept every flight of the frozen cohort and of the deployed-system validation stable. Deciding that a payload has gone is a different matter: the same estimate that is accurate enough to adapt a model in hover is not reliable enough to declare a release in manoeuvre. In this work that decision was safe either when the estimate was asked a narrower question — whether a release that has already been commanded has physically happened — or when it was not asked at all, and the release question was put to a measurement that does not depend on it. The second form also detects a load lost without a command; it moves the weak point from the manoeuvre to the calibration of the thrust map, and the validation located that weak point exactly: an under-reading map larger than the payload's share of the vehicle's mass makes a loaded vehicle look empty.

References

[1] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 edition, 2017.

[2] David Q. Mayne. Model predictive control: Recent developments and future promise. *Automatica*, 50(12):2967–2986, 2014.

[3] Lars Grüne and Jürgen Pannek. *Nonlinear Model Predictive Control: Theory and Algorithms*. Springer, 2 edition, 2017.

[4] Mina Kamel, Michael Burri, and Roland Siegwart. Linear vs nonlinear MPC for trajectory tracking applied to rotary wing micro aerial vehicles. *IFAC-PapersOnLine*, 50(1):3463–3469, 2017.

[5] Christopher V. Rao, James B. Rawlings, and David Q. Mayne. Constrained state estimation for nonlinear discrete-time systems: stability and moving horizon approximations. *IEEE Transactions on Automatic Control*, 48(2):246–258, 2003.

[6] Moritz Diehl, Hans Georg Bock, and Johannes P. Schlöder. A real-time iteration scheme for nonlinear optimization in optimal feedback control. *SIAM Journal on Control and Optimization*, 43(5):1714–1736, 2005.

[7] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

[8] Robin Verschueren, Gianluca Frison, Dimitris Kouzoupis, Jonathan Frey, Niels van Duijkeren, Andrea Zanelli, Branimir Novoselnik, Thivaharan Albin, Rien Quirynen, and Moritz Diehl. acados—a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 14(1):147–183, 2022.

[9] Gianluca Frison and Moritz Diehl. HPIPM: a high-performance quadratic programming framework for model predictive control. In *IFAC-PapersOnLine (21st IFAC World Congress)*, volume 53, pages 6563–6569, 2020.

[10] D. Mellinger, Q. Lindsey, M. Shomin, and V. Kumar. Design, modeling, estimation and control for aerial grasping and manipulation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2668–2673, 2011.

[11] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In *IEEE Conference on Decision and Control (CDC)*, pages 5420–5425, 2010.

[12] Philipp Foehn, Angel Romero, and Davide Scaramuzza. Time-optimal planning for quadrotor waypoint flight. *Science Robotics*, 6(56):eabh1221, 2021.

[13] Anil Aswani, Humberto Gonzalez, S. Shankar Sastry, and Claire Tomlin. Provably safe and robust learning-based model predictive control. *Automatica*, 49(5):1216–1226, 2013.

[14] Lukas Hewing, Kim P. Wabersich, Melanie Menner, and Melanie N. Zeilinger. Learning-based model predictive control: Toward safe learning in control. *Annual Review of Control, Robotics, and Autonomous Systems*, 3:269–296, 2020.

[15] Guillem Torrente, Elia Kaufmann, Philipp Foehn, and Davide Scaramuzza. Data-driven MPC for quadrotors. *IEEE Robotics and Automation Letters*, 6(2):3769–3776, 2021.

[16] Tim Salzmann, Elia Kaufmann, Juan Arrizabalaga, Marco Pavone, Davide Scaramuzza, and Markus Ryll. Real-time neural MPC: Deep learning model predictive control for quadrotors and agile robotic platforms. *IEEE Robotics and Automation Letters*, 8(4):2397–2404, 2023.

[17] Lukas Brunke, Melissa Greeff, Adam W. Hall, Zhaocong Yuan, Siqi Zhou, Jacopo Panerati, and Angela P. Schoellig. Safe learning in robotics: From learning-based control to safe reinforcement learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 5:411–444, 2022.

[18] Drew Hanover, Philipp Foehn, Sihao Sun, Elia Kaufmann, and Davide Scaramuzza. Performance, precision, and payloads: Adaptive nonlinear MPC for quadrotors. *IEEE Robotics and Automation Letters*, 7(2):690–697, 2022.

[19] Ewoud J. J. Smeur, Qiping Chu, and Guido C. H. E. de Croon. Adaptive incremental nonlinear dynamic inversion for attitude control of micro air vehicles. *Journal of Guidance, Control, and Dynamics*, 39(3):450–461, 2016.

[20] Bingheng Wang, Zhengtian Ma, Shupeng Lai, and Lin Zhao. Neural moving horizon estimation for robust flight control. *IEEE Transactions on Robotics*, 40:639–659, 2024. open source: RCL-NUS/NeuroMHE.

[21] C. G. Atkeson, C. H. An, and J. M. Hollerbach. Estimation of inertial parameters of manipulator loads and links. *The International Journal of Robotics Research*, 5(3):101–119, 1986.

[22] Teodor Tomić, Christian Ott, and Sami Haddadin. External wrench estimation, collision detection, and reflex reaction for flying robots. *IEEE Transactions on Robotics*, 33(6):1467–1482, 2017.

[23] James Svacha, James Paulos, Giuseppe Loianno, and Vijay Kumar. IMU-based inertia estimation for a quadrotor using Newton–Euler dynamics. *IEEE Robotics and Automation Letters*, 5(3):3861–3867, 2020.

[24] Valentin Wüest, Vijay Kumar, and Giuseppe Loianno. Online estimation of geometric and inertia parameters for multirotor aerial vehicles. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1884–1890, 2019.

[25] Christian Böhm, Martin Scheiber, and Stephan Weiss. Filter-based online system-parameter estimation for multicopter UAVs. In *Robotics: Science and Systems (RSS)*, 2021. paper 087, DOI 10.15607/RSS.2021.XVII.087.

[26] Koushil Sreenath, Nathan Michael, and Vijay Kumar. Trajectory generation and control of a quadrotor with a cable-suspended load — a differentially-flat hybrid system. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 4888–4895, 2013.

[27] Justin Thomas, Giuseppe Loianno, Joseph Polin, Koushil Sreenath, and Vijay Kumar. Toward autonomous avian-inspired grasping for micro aerial vehicles. *Bioinspiration & Biomimetics*, 9(2):025010, 2014.

[28] Anibal Ollero, Marco Tognon, Alejandro Suarez, Dongjun Lee, and Antonio Franchi. Past, present, and future of aerial robotic manipulators. *IEEE Transactions on Robotics*, 38(1):626–645, 2022.

[29] Michèle Basseville and Igor V. Nikiforov. *Detection of Abrupt Changes: Theory and Application*. Prentice-Hall, 1993.

[30] W. P. M. H. Heemels, Karl Henrik Johansson, and Paulo Tabuada. An introduction to event-triggered and self-triggered control. In *IEEE Conference on Decision and Control (CDC)*, pages 3270–3285, 2012.

[31] Moses Bangura and Robert Mahony. Nonlinear dynamic modeling for high performance control of a quadrotor. In *Australasian Conference on Robotics and Automation (ACRA)*, 2012.

[32] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. PX4: A node-based multithreaded open source robotics framework for deeply embedded plat-

forms. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6235–6240, 2015.

[33] Nathan Koenig and Andrew Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2149–2154, 2004.

Appendix A

Implemented Optimal Control Problem Configuration

Table A.1: NMPC configuration (evaluated configuration and deployed controller).

State dimension	13
Input dimension	4
Runtime parameter dimension	23 in the reported flights (three unused slots held at zero); 20 deployed, (4.5)
Prediction stages	$N = 20$
Sampling interval	$\Delta t = 0.05\,\mathrm{s}$
Prediction horizon	1.0 s
Solve rate	20 Hz; setpoints published at 50 Hz
Integrator	Explicit RK4
NLP method	Full SQP with merit backtracking
QP solver	Partial-condensing HPIPM
Hessian	Gauss–Newton
Thrust box	$[1.27\,\mathrm{N},\ 31.35\,\mathrm{N}]$ (actuator envelope)
Torque box	$\pm 0.5\,\mathrm{N\,m}$ roll/pitch, $\pm 0.2\,\mathrm{N\,m}$ yaw
Bryson tolerances (state)	0.1 m, 0.3 m/s, 0.05, 0.3 rad/s
Bryson tolerances (input)	2.0 N, 0.5 N m, 0.2 N m
Terminal scaling	$20\times$ pos., $1\times$ vel., $2\times$ att., $0.2\times$ rate

A.1 Identity of the deployed system in the validation campaign

Table A.4 identifies the code, toolchain and generated solvers with which the 180 flights of Section 5.3 were flown. The toolchain and solver identities were

Table A.2: MHE configuration. The full-state estimator is the evaluated configuration of the cohort and the release campaign; the reduced estimator is the deployed estimator and the only one in the mainline code base.

	Full-state (evaluated)	**Reduced (deployed)**
Estimator	full-state F	reduced R (the only estimator of the mainline code base)
State dimension	14 (x, m_T)	4 ($\boldsymbol{v}$, m_T)
Process-noise dimension	13 (physical states only)	3 (velocity)
Runtime parameter dimension	7 (wrench; lagged ΔJ; $c_x = c_y = 0$)	5 (measured unit quaternion; T_{phys})
Measurement residual	13 ((5.4))	3, velocity ((4.4))
Weights	(5.7)–(5.9)	(4.9) (velocity and mass blocks of the former)
Estimation stages / interval	$N = 20$ / 0.1 s	same
Mass box[b]	$[0.95\,m_B,\ 5.0\,\text{kg}]$	$[0.70\,m_B,\ 5.0\,\text{kg}]$
Mass-health band	none	(4.10), $[1.964\,\text{kg}, 2.464\,\text{kg}]$; recover after 10 solves
Mass arrival σ	3.16 kg	3.16 kg
Initial iterate	window force balance (4.14)	same
Estimator inertia increment	lagged, (5.6)	none
Stage weighting β_j	residual-triggered de-weighting enabled, uniform in effect (Section 4.3.6)	same, rebuilt on the reduced blocks
Solver failure	re-anchor after 5 consecutive failures; seed adopted as estimate	re-initialise after 5; last valid estimate held, frame unhealthy
Output filter	first order, $\tau = 0.25\,\text{s}$	same
NLP / QP method	Full SQP / partial-condensing HPIPM	same

[b] Reduced: adjustable by `MHE_REDUCED_M_MIN` and `MHE_REDUCED_M_MAX`.

captured at 20:54 on 6 November 2026, during the release segment; none of the listed components was rebuilt after the campaign started, as their file times show. Solver identities are the first 16 hexadecimal digits of the SHA-256 of the installed files.

Table A.3: Payload interface, lifecycle and task settings (evaluated configuration). Rows marked ‡ are where the deployed system differs or adds.

Frame fields	m_T, ΔJ_{diag}, J_{diag}, γ, health, solution age (offset fields unused)
Vertical-arm prior r_z	-0.27 m nominal, both optimisers (realised 0.25 m to 0.30 m)
Input freshness / solution freshness	0.30 s / 0.30 s
Consumer receive-age limit	0.35 s
Controller-side filter τ	0.20 s
Slew limits (m_T / ΔJ)	0.60 kg/s / 0.080 kg m^2/s
Evidence-score mode	mass only (no inertia factor)
Payload-mass score thresholds	$(0.015, 0.060)$ kg
Confirmation	cohort: $\gamma \geq 0.90$ held 1.0 s on healthy frames
Confirmation (campaign arm D, deployed)‡	$\gamma_{\text{acc}} \geq 0.90$ held 1.0 s on fresh evidence, opened by the release command or the loss watchdog
Force-balance score‡	(4.34)–(4.36); low-pass 0.5 s, output 0.25 s; fresh within 0.2 s; thresholds $(0.015, 0.060)$ kg
Loss watchdog‡	arms after lift start on $\gamma_{\text{acc}} < 0.75$ for 2.0 s; cleared at the next pickup
Presence entry / exit	$m_P > 0.09$ kg for 2 s / $m_P < 0.02$ kg for 5 s in quiet flight
Quiet-flight test	$\|\boldsymbol{\omega}\| \leq 0.15\,\text{rad s}^{-1}$, $\|\boldsymbol{v}_{xy}\| \leq 0.20\,\text{m s}^{-1}$
UNRESOLVED timeout / brake duration	12 s / 3 s
Body-rate scale cap / dead band	5.0 / 0.08
Body-rate scale τ / rise / recover	0.5 s / 4.0 s^{-1} / 1.5 s^{-1}
Body-rate headroom fraction / hard limit	0.95 / 2.0 rad s^{-1}
Attach inertia bootstrap	envelope 0.30 kg ($\Delta J = 0.0191$ kg m^2, $\kappa = 2.34$); fade 0.5 s; hold until command†, deployed also ended by a detected uncommanded loss‡
Residual specific force†	$a = 10$, $\omega_c = 0.5\,\text{rad s}^{-1}$, nominal mass $\hat{m}$
Relative alignment†	tolerance 0.012 m, wait ≤ 4 s
Figure-eight yaw smoothing†	on
Controller offset compensation	off ($\boldsymbol{c}_{xy} \equiv \mathbf{0}$)
Release	at the figure-eight tip after 45 s loaded carry

† Set explicitly by the cohort driver; off at the experiment entry points of the campaign code base, fixed on in the mainline code base. The geometry-release mode, moment-based offset and release options and moment over-range re-anchoring appear in launch records but are inert in the mass-only form; the residual event trigger is active but rarely changes the stage weights (Section 4.3.6).

Table A.4: Identity of the deployed system in the validation campaign, from the versions file `versions_20261106.txt` in the analysis directory and the `batch_info.txt` of each segment.

Code base	mainline (`MHE_NMPC`)
Tag / commit	`deployed-r-armd-180-v1` / `59236f7fecb0`
Flight-code fingerprint	`186022e6c95cfa38`, checked before each of the 180 flights; covers the flight-stack package sources and configuration, worlds and gripper plug-in, launch and stop scripts, the tools they call and the batch driver
Repository head	at the start of the release segment: the tag, no uncommitted change under `src/` or `scripts/`; later segments: `9881e78`, which changes only this dissertation, with four untracked offline analysis scripts in `scripts/tools/` outside the fingerprint
ROS 2	Jazzy; `rclpy` 7.1.11, `mavros` 2.14.0
Gazebo	Gazebo Sim 8.11.0 (Harmonic)
PX4	`v1.17.0-alpha1-1373-gc0633d89ff-dirty`; binary `05c4a2a876d724ba`
acados	`dc6668f85`
Python	3.12.3; NumPy 1.26.4; CasADi 3.7.2
Estimator solver (R)	library `4066ba077b882e0b`; problem description `0634387011f05db4`
Controller solver	library `0809ef2083bcfa7c`; problem description `d5965ff370213bb8`
Schedule seeds	`20261106001`–`20261106004` (release, faults, thrust map, thrust map delayed)
Flight data	`results/batch_release_20261106_201503`, `batch_faults_20261107_002042`, `batch_thrust_20261107_030655`, `batch_thrust_low_d4_20261107_055237`
Analysis	`scripts/tools/campaign_eval.py`; `results/campaign_180_analysis/` (acceptance criteria, per-flight table, report)

Appendix B

Experiment Record Template

Each run stores the following metadata with its log, so that any reported figure can be regenerated from retained data:

1. code identity — repository revision, a frozen patch of uncommitted changes and an archive of untracked files, with an explicit verdict on whether an exact-commit claim is permitted — together with the build timestamp and generated-solver identity;
2. the *resolved value* of every environment switch that affects behaviour — not only those explicitly set, since a default that is never written to disk is the commonest way for a batch to become unattributable;
3. airframe mass and inertia calibration, rotor coefficients and the fixed actuator limits;
4. payload mass and attachment geometry for offline scoring, together with evidence that neither reached the adaptive path;
5. trajectory parameters, grasp and release phase, and random seed;
6. both horizons, weights, bounds and solver options;
7. topic rates and measured end-to-end latency;
8. the complete lifecycle event trace, with timestamps for every state transition and every confirmation;

9. termination reason, solver status sequence and constraint activity.

Results tables carry one row per run in the retained data, not only an aggregate, so that failed runs cannot disappear during manual selection. Runs terminated by numerical failure remain in the denominator; any manual exclusion must name a pre-declared logging or infrastructure fault.